OptAttest: Verifying Multi-List Multi-Hop History via a Hybrid Zero-Knowledge Architecture

Joshua G. Stern
Research Engineer at iterabloom LLC
josh@iterabloom.com

May 28, 2025

Abstract

To prevent privacy-preserving digital assets from becoming instruments of despotism via unitary-executivist compliance regimes, we propose OptAttest, a hybrid zero-knowledge architecture. This system empowers users to *optionally* generate verifiable attestation history for the current (Hop 0) and immediately preceding (Hop 1) transactions involving their private commitments. For crucial 0-hop multi-list attestations, users employ Zero-Knowledge Proofs (ZKPs) of claims from selected Verifiable Credentials (VCs). Users achieve per-transaction efficiency with diverse VC types by pre-computing and caching proofs of their VC validity. This approach avoids mandated adherence to singular, fallible external standards. Opted-in lightweight updates create cryptographic accumulator summaries, verified by network infrastructure (e.g., Layer 2 scaling solutions using Zero-Knowledge Virtual Machines (L2 zkVMs) [Roy, 2024]), and are paired with user-managed Intermediate Attestation Data Packets (IADPs) containing detailed evidence. For comprehensive verification, users can then generate full recursive proofs from these IADPs for their attestation-enabled funds, leveraging native zkVM recursion. The protocol facilitates optional attestation generation, not enforcement, allowing downstream policy application. Aiming to cultivate a permissionless ethos, we propose a user-centric balance between privacy and verifiable accountability, distinct from models compelling broader data access. Folding schemes [Kothapalli and Setty, 2024] are noted as potential future enhancements for recursive proof efficiency.

1 Introduction

Privacy-preserving digital asset systems – from the traditional cryptocurrencies to the up-and-coming private Layer 2 (L2) rollups – reveal tensions between user privacy needs and intensifying regulator demands. Simultaneously satisfying both impulses is an open challenge, particularly when trust in the neutrality and justice of regulatory tools themselves comes into question.

1.1 The Privacy vs. Attestation Challenge: Sovereign Failures and the Need for Resilient Alternatives

Current approaches to bridging the gap between privacy and verifiable transaction history on blockchains often fall short. Systems relying on default transparency, where transaction details are publicly pseudonymous, facilitate blockchain ana-

lytics. However, this approach compromises user privacy, is ineffective against sophisticated obfuscation techniques (e.g., mixers), and becomes untenable as privacy-enhancing technologies like Zero-Knowledge Proofs (ZKPs) become more prevalent [Azgad-Tromer et al., 2023]. This challenge extends to L2 systems aiming to provide privacy on otherwise transparent L1s.

Meanwhile, the external lists against which systems might provide attestations, particularly governmental sanctions lists such as the U.S. Office of Foreign Assets Control (OFAC) Specially Designated Nationals (SDN) list, are growing less trustworthy. The SDN list has seen perverse alterations (early 2025), including the listing of human rights defenders[1] and the delisting of human rights violators[2].

[1]On February 6, 2025, President Donald Trump issued Executive Order 14203 titled "Imposing Sanctions on the International Criminal Court." [Human Rights Watch, 2025, Amnesty International, 2025, Associated Press, 2025] This order is retaliation against the ICC's issuing of arrest warrants for Israeli Prime Minister Benjamin Netanyahu and former Defense Minister Yoav Gallant. [Human Rights Watch, 2025, Amnesty International, 2025, Associated Press, 2025]

[2]On January 24, 2024, hundreds of Tzav 9 activists blocked more than 60 humanitarian aid trucks at Kerem Shalom crossing; hours later, only nine trucks managed to pass; the remaining trucks were either rerouted to Egypt's Rafah Crossing or unable to deliver aid that day [Lidor, 2024]. On May 6-7, 2024, at Latrun Junction, Tzav 9 activists blocked an aid convoy of a dozen trucks for four hours; the activists slashed tires, climbed onto trucks, ripped open sacks of food, and dumped flour on the highway [TOI Staff, 2024b]. On May 13, 2024, at the Tarqumiyah crossing, Tzav 9 activists blocked and looted a humanitarian aid convoy heading to Gaza, spilling food onto the road. Later that day, at least two trucks from the convoy were set on fire [TOI Staff, 2024a, democracynow.org, 2024, Magid, 2024, Israel National News Staff, 2024]. On June 14, 2024, Reuters reported that the Biden administration's OFAC added Tzav 9 to the SDN. State Department spokesperson Matthew Miller said at the time: "For months, individuals from Tzav 9 have repeatedly sought to thwart the delivery of humanitarian aid to Gaza, including by blocking roads, sometimes violently, along their route from Jordan to Gaza, including transiting the West Bank. They also have damaged aid trucks and dumped life-saving humanitarian aid onto the road" [Lewis, 2024].

On January 20, 2025, the first day of President Trump's second term, Tzav 9 leaders Reut Ben Haim and Shlomo Sarid sent an open letter to Trump urging him to halt all humanitarian aid to Gaza while Israeli hostages remain captive [Listratov, 2025, Kahana, 2025, Katz, 2025].

The 1949 Geneva Conventions and customary international humanitarian law (IHL) prohibit starving or collectively punishing a civilian population [Human Rights Watch, 2008, Polakow-Suransky, 2015]. EU foreign policy chief Josep Borrell has warned that cutting "water, electricity, [and] food to a mass of civilian people is against international law" [Gray, 2023]. Geneva IV (Art. 33) expressly bans collective penalties and measures of intimidation against protected persons [Polakow-Suransky, 2015]. Additional Protocol II likewise outlaws collective punishments and reprisals against civilians [Human Rights Watch, 2008]. The ICRC's Practical Guide notes, "the parties to the conflict have the obligation to allow and facilitate the free passage" of relief supplies and may not "hinder them" [Program for Humanitarian Policy and Conflict Research at Harvard University]. Similarly, Additional Protocol I (Art. 70) requires that all relief consignments destined for civilians be allowed "rapid and unimpeded passage" and not diverted or delayed [Program for Humanitarian Policy and Conflict Research at Harvard University]. The Rome Statute for the ICC codifies that deliberately depriving civilians of food, water or relief supplies is a war crime (Art. 8(2)(b)(xxv)) [International Criminal Court, 2002]. Human Rights Watch (HRW) notes that any "belligerent reprisals" or collective sanctions that target civilians (even indirectly) violate fundamental humane treatment rules [Human Rights Watch, 2008].

And yet: senior Israeli officials have publicly advocated just such a linkage of aid to hostages [Hall, 2023]. Under Protocol II, collective sanctions of any kind are banned, and violations may entail individual criminal responsibility [Human Rights Watch, 2008]. And any third-party state (like the U.S.) "knowingly making a substantial contribution" to such an aid blockade risks being "complicitly liable" in a war crime [Dannenbaum et al., 2023].

International law experts stress that one war crime (hostage-taking by Hamas) cannot be used to excuse another (starving Gazans). Professor Tom Dannenbaum (Tufts) declared "Aid must be allowed in unconditionally", and warned that hostage-taking cannot justify starving civilians [Dannenbaum et al., 2023]. Professor Janina Dill (Oxford) similarly notes that "withholding consent [to relief] [...] violates other rules of international law" and may itself breach the Rome Statute's starvation ban [Dannenbaum et al., 2023]. Aid must flow regardless of hostages.

The World Food Programme reported in April 2025 that 400,000 Gaza residents depend on UN aid and that "food stocks [are] completely depleted," with dozens already dying of hunger [Al Jazeera Staff, 2025]. UN agencies report that Gazans are surviving on a single meal a day, stretching dwindling food rations as humanitarian aid remains blocked. Food stocks are now exhausted, with families relying on canned goods, lentils, and rice. Meanwhile, healthcare services are deteriorating sharply, with critical shortages of trauma supplies, antibiotics, and essential medical equipment [UN News Staff, 2025].

Also on Jan 20, 2025 (the same day as the letter from Tzav 9 to Trump), Trump issued EO 14148 [Executive Office of the President, 2025] reversing former President Biden's EO 14115 [Executive

Such policy shifts not only erode trust in the ethical or beneficial nature of a specific sanctions list, but also signal a deeper crisis in the stewardship of such critical informational infrastructure by centralized authorities. This erosion exemplifies how systems can be simultaneously *compliant*—adhering to prosocial standards—and *complicit*, sharing responsibility for antisocial harms when the underlying mandates are unjust or applied perversely. A system that automates adherence to a singular, uncontestable list, particularly one susceptible to such arbitrary alteration by unitary powers, could with minimal changes automate industrial-scale oppression. This concern that digital financial systems could become instruments of despotism, when the presumed sovereign guarantors of rights and due process themselves act erratically or fail in their protective duties, is substantiated by legal scholars and advocates. They warn that new digital currency infrastructures, if not deliberately designed for privacy and resilience against such failures, risk replicating or exacerbating an existing "surveillance status quo" [Carrillo, 2023], and they call for robust, privacy-preserving alternatives, such as digital cash, to safeguard civil liberties [Grey, 2021].

The demonstrable fallibility and potential for misuse of centrally controlled lists thus motivates not merely a technical workaround, but a proactive search for more resilient, independent, and ethically-governed foundations for verifiable claims. Rather than embedding a singular, potentially compromised enforcement regime at the protocol level, our objective is to cultivate a "permissionless"[3] ecosystem capable of providing verifiable attestations of historical interactions involving discrete private representations (such as UTXOs or private notes). Such a system allows diverse entities and communities to independently define and enforce chosen policies based on the verifiable properties of these attestations.

Some systems, like Zcash, employ cryptographic mechanisms enabling selective disclosure for shielded UTXO-like notes, most notably through *viewing keys* [Hopwood et al., 2024]. Zcash's primary mechanism—the Full Viewing Key (FVK)—offers comprehensive access to the full transaction history tied to an address (including details such as senders, recipients, and values for all transactions), thereby limiting per-transaction disclosure flexibility. Zcash additionally provides a payment disclosure feature for selectively revealing transaction-specific details [Hopwood et al., 2024]. While intended to empower users, the very availability of such keys, which expose wholesale transactional data, can inadvertently create an expectation for their surrender as a default measure for regulatory interaction, potentially making full historical visibility and the ceding of informational sovereignty the anticipated norm.

Office of the President, 2024]; EO 14148 caused the new administration's OFAC to *stop* sanctioning (on Jan 24) the following violent extremist groups: Tzav 9, Hilltop Youth, Hashomer Yosh, Meitarim Farm, and associated individuals and groups.

On Jan 21, 2025, dozens of Hilltop Youth protesters blocked traffic at the entrance to Jerusalem, decrying a police shooting in the Nablus-Jenin area (West Bank) the day prior, which had injured four activists and three police. A protester stated to a reporter: "We demand that the security apparatus [...] distinguish between the Arab enemy [...] and Jewish brothers" [Ettinger, 2025]. On Jan 24, 2025, Avichai Suissa and other Hashomer Yosh activists built livestock corrals and planted olive saplings on a new unauthorized agricultural outpost near the Shilo settlement (West Bank) [WAFA News Agency, 2025]. In early March, 2025, settlers from Yinon Levi's Meitarim Farm entered Palestinian grazing lands near Khalet al-Mafatih (South Hebron Hills, West Samu', West Bank) on an ATV, "violently enter[ed] the herd in the pasture next to their land," and angrily cursed and threatened the local shepherd (Muhammad Mahariq), demanding he, his wife, and their four married sons leave the area, where the family has lived since the 1990s. "The settlers arrived with the army, some of whom were also settlers dressed in IDF uniforms" [Becker and Dabsan, 2025].

As a point of principle, and without excusing or justifying atrocious, reprehensible, and potentially genocidal conduct by anyone, we note that hostage-taking is a war crime, and all hostages must be released immediately and unconditionally.

[3] We are not aware of any digital currency systems that are both useful and truly free of all permissions. As far as we know, a digital currency may not *permit* double-spending, if it is to be useful.

Similarly, Railgun creates UTXO-like private notes within Ethereum's account-based environment, maintaining privacy for internal transactions and, like Zcash, enabling 'viewing-key' based disclosure of full wallet history [RAILGUN Project, e]. Its 'Private Proofs of Innocence' (PPOI) system allows users to generate attestations, primarily at the point of shielding funds into Railgun, proving non-interaction with addresses on lists from multiple external providers [RAILGUN Project, a]. This attestation, reflecting the state at the *initial shield event*, is then cryptographically propagated through subsequent private transactions within Railgun using recursive SNARKs [RAILGUN Project, d]. While PPOI offers a valuable historical link to an initial compliance check and supports multi-list attestations at entry, OptAttest differs fundamentally: our proposal enables users to optionally generate *new, independent attestations from their own Verifiable Credentials (VCs) at each chosen transaction hop*. These VCs can reference a user-specified set of lists pertinent to that specific interaction. The OptAttest approach leads to a disclosure model focused on VC-derived claims (e.g., "non-membership on list L1 via a VC from Issuer Y, for attestation event Nonce123, at Hop 0") rather than full transaction metadata or solely propagated entry-point attestations. The OptAttest architecture deliberately avoids a global viewing key for the attestation history data itself, seeking instead to provide users with more granular control over the claims they make. OptAttest aims to shift the paradigm towards user-generated, optional, and bounded attestations where targeted verifiability of specific claims about historical attestation *events* (identified by nonces and characterized by VC issuer DIDs, lists, and claims), not wholesale transparency of transaction history, addresses information requirements.

The mere existence of any user-controlled selective disclosures, whether comprehensive viewing keys or specific attestation generations, creates potential avenues—and possibly expectations—for authorities to compel their use.

This paper introduces privacy-preserving attestations of historical interactions (or non-interactions) by private representations against user-specified external data sources. By leaving the enforcement of external standards outside of the protocol, and by promoting a model of targeted, user-driven attestations (focused on the properties of attestation events rather than historical attester identities) rather than broad, key-based disclosures for historical data, we attempt to minimize our complicity in causing harm and to offer a more liberty-oriented approach to balancing privacy with accountability. Our goal is to curtail antisocial behaviors not by mandating universal surveillance, but by making it difficult for predatory actors to generate convincingly "clean" and deep attestation histories within an ecosystem that values verifiable, yet user-controlled, claims.

1.2 The Impracticality of Naive N>0 Recursive Proofs (if Mandated for Each Discrete Commitment)

The concept of *N>0 attestation*, where a system cryptographically verifies that a specific discrete private commitment (e.g., a UTXO or an L2 private note) has not interacted with prohibited entities within N previous transaction hops involving its ancestral commitments, addresses the potential need for provenance. If such attestations were to be generated for every transaction, the most direct cryptographic approach would involve *recursive ZKPs* (like zk-SNARKs or zk-STARKs). In such a mandatory scheme, each transaction generating a new private commitment would include a ZKP verifying not only the validity of the current transaction and its 0-hop attestation, but also the validity of the ZKP associated with the input commitment(s).

However, the computational cost of proof generation (*prover time*) frustrates this naive and universally-applied recursive approach. Verifying a ZKP (es-

pecially a SNARK involving pairing-based cryptography) *within* another ZKP circuit imposes substantial computational overhead [Kothapalli et al., 2022]. For resource-constrained environments like smartphones, mandating the generation of such complex recursive proofs for every transaction creating a new private commitment would result in prohibitively long computation times, excessive energy consumption, and potentially insurmountable memory requirements. This complexity would render such a naive, mandatory N>0 recursive model impractical for widespread adoption. This challenge motivates our optional and hybrid architecture, which avoids this intensive recursion for standard private transactions and for the lightweight path of attestation-enabled transactions.

1.3 Our Approach: Optional Hybrid Attestation for Hop 0 & Hop 1 History

We propose OptAttest, a *hybrid architecture*—one combining quick, optional checks with more intensive, less frequent verification—that enables users to *optionally* generate and verify Hop 0 & Hop 1 attestation history. This capability signifies a user's choice to create verifiable claims, for example, about non-interaction with entities on *user-specified lists* for the current transaction and its immediate predecessor transaction involving a given private digital asset. This architectural choice to prioritize user agency ensures that standard private transactions maintain fast performance, as those not engaging this optional layer remain unencumbered by attestation metadata. The architecture is applicable to both traditional UTXO-based systems and L2 rollups that manage privacy through internal, private assets. The core idea of OptAttest is to separate the optional, efficient generation of attestation summaries from the less-frequent, more thorough user-driven verification of the full historical attestation chain for those assets whose owners have proactively chosen to create such a history. Our aim is an accessible system that offers verifiable information as a tool for users who choose to create and utilize it, thereby promoting a resilient and pluralistic mode of accountability.

In this model, users exercise their agency on a per-transaction basis, deciding whether or not to enable a *lightweight attestation path*. While a primary motivation for a user to engage this optional path might be interaction with counterparties requiring verifiable attestation history (such as regulated Virtual Asset Service Providers), the choice remains fundamentally theirs. If enabled, this path involves the user performing an immediate (0-hop) check for the current transaction using their private digital identity and pre-obtained Verifiable Credentials (VCs) for a **set of specified lists {L1, L2,...}** relevant to their interaction. Per-transaction efficiency, even with diverse or cryptographically complex VCs, is achieved through a user-managed process: the wallet periodically performs a background task to generate a comprehensive ZKP of their VCs' validity (a "cached VC validity proof"). The 0-hop ZKP for each OptAttest transaction then efficiently verifies this cached proof and links its attested claims to the current transaction. The transaction logic subsequently updates a compact cryptographic summary, the Acc, linked to any newly created, attestation-enabled output assets. This accumulator securely condenses the verified claims from the last two transaction hops relevant to its creation. The correct execution of this update logic is verified by network facilitators (L2 operators) or by the user's own ZKP (for L1 UTXO systems). Concurrently, detailed evidence files, *Intermediate Attestation Data Packets (IADPs)*, are created. These IADPs, containing necessary details for future verification, are encrypted by the sender for the recipient and enable secure *on-device storage*. Importantly, the protocol processes all transactions based on standard privacy rules, irrespective of attestation choices or outcomes, thereby upholding the permissionless ethos of the underlying system.

For private assets that possess an attestation history (i.e., were created via this user-chosen lightweight path), a more computationally intensive *heavyweight path* can be invoked less frequently by the asset holder. This path involves the user generating a complete *recursive ZKP* using their locally stored IADPs. This proof verifies the integrity of the specific asset's Acc by computationally "unrolling" its summary, using the detailed evidence in the IADPs to reconstruct and validate the sequence of past multi-list attestation checks. The primary approach envisioned for this user-generated heavyweight proof relies on the native recursive capabilities of the chosen general-purpose zkVM, with the user's software executing the logic to verify historical 0-hop attestations and the preceding ZKP. The output is a verifiable report of historical attestations for that specific asset, still under the user's control.

While specialized cryptographic methods like *folding schemes* offer potential efficiencies for this recursive process, their integration with general zkVMs presents complexities. Thus, the current emphasis is on the practical utility of native zkVM recursion for these user-driven proofs.

Although generating the heavyweight proof is computationally intensive, its infrequent requirement—and only for attestation-enabled funds when such proof is actively sought by the user or a trusted counterparty—makes the hybrid model practical. This practicality is maintained by background processing on the user's device for cached VC validity proofs and potentially for the heavyweight proof itself, or by user-controlled offloading, thereby preserving responsiveness for everyday transactions.

By focusing on Hop 0 & Hop 1 for private assets whose owners proactively opt into attestation, we target a meaningful level of historical visibility beyond immediate checks, while keeping the chain of proofs bounded. This hybrid approach furnishes verifiable historical attestation data as a resource for those who choose to generate it, without burdening all transactions or users with infeasible computational costs or mandatory metadata generation.

1.3.1 Core User Assurances: Verifiable Claims through Opt-In Attestation

To understand the tangible benefits of this architecture, consider what a user — both as a sender exercising their agency to provide attestations and as a recipient gaining insight — gains when the OptAttest Lightweight Path is proactively engaged for a transaction, compared to a standard private transfer. While a standard transaction ensures basic validity and privacy, choosing to opt into the Lightweight Path provides distinct, verifiable assurances regarding the transaction's context and the history of the assets involved, all under user control.

As a **sender choosing to opt-in**, one actively furnishes cryptographic proof related to the current (0-hop) transaction, thereby taking control of the narrative surrounding their interaction. The sender's process of furnishing cryptographic proof involves utilizing their Decentralized Identifier (DID) to authorize the use of their pre-obtained Verifiable Credentials (VCs). A core element of this capability is leveraging a periodically pre-computed and cached ZKP of their VCs' full validity (the "cached VC validity proof," $\pi_{\text{VC\_validity}}$). The per-transaction 0-hop ZKP then efficiently verifies this cached proof and demonstrates that the claims attested by it apply to the current transaction for a set of lists or criteria *they or their counterparty deem relevant*, not based on a unilateral external mandate. The sender also generates a unique random nonce (`Nonce_Current`) for this specific 0-hop attestation event, further anchoring their agency in this specific act of claim-making. They prepare an Intermediate Attestation Data Packet (IADP) containing the details of this 0-hop attestation event (including this `Nonce_Current`, the specified lists, a summary linking to the attested claims from the cached VC validity proof along with references to the VC *issuer's* DID/key, and the verified claims themselves)

and an updated Acc for the output asset(s). This IADP is then encrypted for the recipient, ensuring the privacy of this user-generated attestation.

Consequently, the **recipient of such an attestation-enabled asset** gains several assurances. Firstly, upon decrypting the received IADP, they gain immediate, verifiable insight into the sender's 0-hop attestation event for that specific transaction. This insight includes knowing the unique `Nonce` that identifies the sender's specific act of attestation, the lists referenced, the precise verified claims that were cryptographically proven (e.g., non-membership on a designated sanctions list at the time of transfer, or possession of a particular professional certification), and importantly, references to the *VC issuer's* DID/key. The sender's subject DID used to authorize their VCs for that attestation remains private and is not revealed to the recipient. Secondly, the received assets are now "attestation-enabled," carrying an Acc that cryptographically summarizes this 0-hop attestation event and, if the input funds were also attestation-enabled, a summary of the prior (Hop 1) attestation event. The `OPT_OUT` value within the Acc structure transparently indicates if any segment of this immediate prior history was not attested under OptAttest, respecting the choices of previous actors.

The attestation-enabled nature of the assets provides the recipient with the **future capability** to generate a user-initiated Heavyweight Proof if they need to verify the Hop 0 and Hop 1 attestation history of these specific assets, further enabling them to assess provenance on their own terms. This proof uses the locally stored IADP(s) as witness data to verify the sequence of historical nonce-identified attestation events and their properties (claims, VC issuer DIDs/keys, lists checked). The `MaxFanIn` constraint is a pragmatic measure to ensure the computational feasibility of these subsequent user-initiated Heavyweight Proofs. While the chain of IADPs provides a traceable evidence path of attestation events, the Heavyweight Proof offers a succinct, easily verifiable cryptographic summary of this N-hop history for third parties, should the user choose to share it, without requiring direct access to the raw IADPs.

The nature and strength of these assurances are further influenced by the practices of VC issuers and the choices users make in selecting them. Issuers, ideally curated through a robust and pluralistic governance framework (as speculated in Section 6), might range from those performing real-world identity verification against external datasets to those attesting to properties of on-chain addresses. Users obtain these native VCs using their own DIDs, and their wallets manage the periodic background generation of cached VC validity proofs. Users, in turn, select which VCs to obtain (and thus which VC issuers to rely on) that align with their desired balance of privacy, disclosure, and the specific contextual requirements of their interactions, thereby exercising their sovereignty in the attestation process.

1.4 Core Contributions: Enabling User Agency and Resilient Accountability

The contributions of the OptAttest architecture extend beyond mere technical novelty, aiming to provide foundational tools for more user-centric and ethically resilient digital interactions:

- The concept of an optional hybrid lightweight/heavyweight ZKP model that provides users with the option to enable attestation verification on a per-transaction basis, thereby choosing their desired level of verifiable disclosure.
- The design of the `AttestationAccumulator` mechanism, employed only for attestation-enabled commitments, to compactly represent the hop 0 & hop 1 history. This mechanism includes a "merge-hash binding" strategy

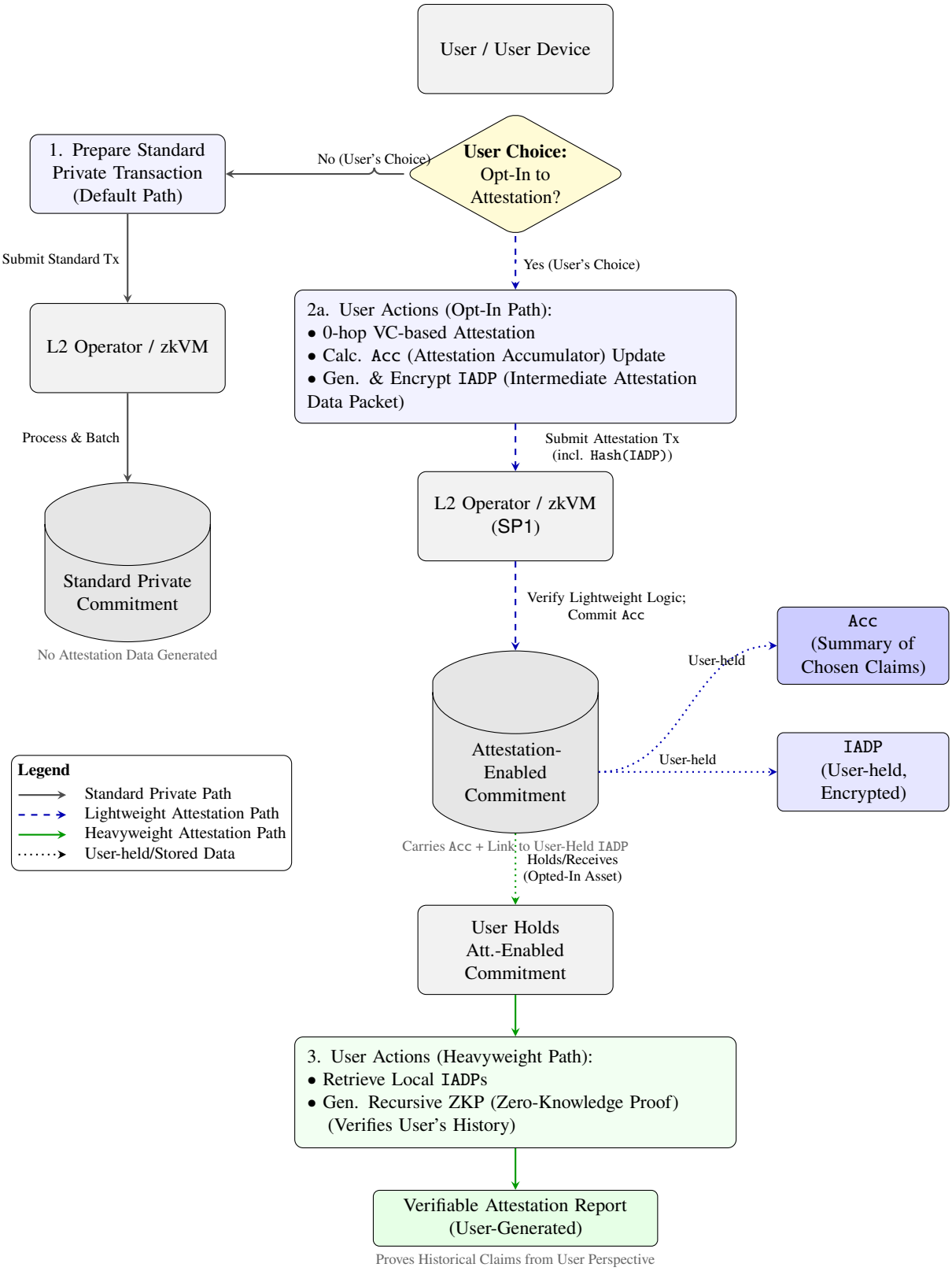

Figure 1: OptAttest Hybrid Architecture: Users exercise their choice. They can use standard private transactions (Path 1, Default). If opting into attestations (Path 2, Lightweight), they generate an attestation-enabled commitment with a compact `AttestationAccumulator` (`Acc`) summarizing their chosen claims, and an encrypted, user-held Intermediate Attestation Data Packet (`IADP`). The user-initiated Heavyweight Path (Path 3) employs local `IADP`s (user-controlled evidence) to produce a Verifiable Attestation Report for Hop 0 & 1 history, reflecting the user's chosen attestation lineage.

that efficiently handles transactions consuming multiple input commitments (some potentially attestation-enabled, others reflecting a user's choice to opt-out of attestation), all while respecting user decisions if the lightweight path is engaged.

- The integration of Intermediate Attestation Data Packets (IADPs) and asynchronous on-device storage, for users who opt-in to attestations. The integration of Intermediate Attestation Data Packets (IADPs) and asynchronous on-device storage allows users to privately manage the detailed historical data required for subsequent user-generated heavyweight proofs, further reinforcing their control over their attestation narratives.
- An outline of how modern cryptographic primitives can be combined to realize this optional attestation layer. The combination of modern cryptographic primitives includes: general-purpose zkVMs capable of arbitrary computation and native recursion; and a strategy for efficient 0-hop multi-list Verifiable Credential (VC) attestations where users periodically pre-compute ZKPs of their VC validity ("cached VC validity proofs") via background processing, allowing per-transaction ZKPs to verify these cached proofs rapidly. This design facilitates the use of diverse, potentially community-vetted, VC issuers.
- An initial framework (speculated in Section 6) for the **pluralistic and ethically-grounded governance of Verifiable Credential issuers** within the OptAttest ecosystem. This framework is a contribution towards ensuring the trustworthiness and vitality of the attestations themselves, moving beyond purely technical assurances to address the socio-political dimensions of digital trust.
- An analysis of the security, privacy, and performance trade-offs inherent in this opt-in attestation approach, arguing for its potential to promote an accountable yet privacy-preserving digital asset ecosystem, particularly when traditional systems of trust are inadequate or compromised.

2 Background

2.1 Zero-Knowledge Proofs (ZKPs) & Recursive Composition

Zero-Knowledge Proofs (ZKPs) are cryptographic protocols allowing one party (the prover) to convince another party (the verifier) that a statement is true, without revealing any information beyond the truth of the statement itself [Goldwasser et al., 1985]. Modern ZKP systems, such as zk-SNARKs (Zero-Knowledge Succinct Non-Interactive Argument of Knowledge) and zk-STARKs (Zero-Knowledge Scalable Transparent Argument of Knowledge) [Ben-Sasson et al., 2018], offer properties crucial for privacy-preserving systems:

- **Completeness:** An honest prover with a valid witness can always convince the verifier.
- **Soundness:** A cheating prover cannot convince the verifier of a false statement (except with negligible probability).
- **Zero-Knowledge:** The verifier learns nothing beyond the statement's validity.
- **Succinctness (often associated with SNARKs):** Proofs can be small and fast to verify, regardless of the computation size.

These properties make ZKPs well-suited for verifying computations privately, such as validating blockchain transactions while hiding sensitive details like amounts or identities [Hopwood et al., 2024].

A critical technique for verifying historical properties, such as the N-hop attestation (N>0) targeted by this work, is *recursive proof composition*. In this technique, a ZKP (π_n) proves the correctness of a computation step n and *also* includes logic to verify a previous proof (π_{n-1}) related to step $n-1$. This recursive approach allows constructing a single proof that attests to the validity of an entire sequence of computations, essential for tracking historical provenance. However, naively implementing recursion by embedding a full ZKP verifier (especially a SNARK verifier) within another ZKP circuit imposes substantial computational overhead for the prover [Kothapalli et al., 2022]. This high cost presents a major challenge for practical systems requiring historical checks and directly motivates both the development of more efficient recursive techniques, like the *folding schemes* discussed in Section 2.5, and the design of our proposed hybrid architecture, which aims to avoid this expensive recursive step for most common transactions.

2.2 General-Purpose zkVMs (e.g., SP1) for Verifiable Computation

While ZKPs can be constructed for specific, fixed computations (circuits), *Zero-Knowledge Virtual Machines (zkVMs)* have significantly advanced usability by enabling proofs for the correct execution of general-purpose programs. A zkVM allows developers to write applications in standard languages (like Rust), compile them to a standard Instruction Set Architecture (ISA) (like RISC-V), and then generate a ZKP attesting that the program executed correctly according to its compiled bytecode, without needing to manually design cryptographic circuits [Succinct Labs, b]. This approach facilitates the use of existing codebases and familiar toolchains for building verifiable applications, such as Layer 2 (L2) rollup protocols.

This paper focuses on **Succinct Processor 1 (SP1)** [Roy, 2024] as a prime example of a modern, high-performance zkVM suitable for our architecture, particularly in scenarios where L2 operators batch-verify transactions or users generate complex proofs. SP1 specifically allows proving the execution of arbitrary Rust programs (or other LLVM-based languages) compiled to RISC-V. Its performance relies on the **Plonky3** proving system [Polygon Labs, 2023], which employs STARK-based techniques (like FRI and AIR) for fast proving times and transparency (no trusted setup for the core proofs).

A key feature of SP1, critical to our proposed design, is its built-in support for **native proof recursion** [Succinct Labs, a]. This native recursion capability allows an SP1 program, while executing within the zkVM, to efficiently verify a ZKP generated by a previous SP1 execution. This native capability enables SP1 to serve two essential functions in our hybrid attestation architecture:

1. For L2 rollups employing this architecture, SP1 can be used by network operators to prove the correct execution of the L2's state transition logic. This logic would include validating all user transactions in a batch, and for those transactions where users **opted into the lightweight attestation path**, the execution of this L2 state transition logic within the zkVM would also verify the correct execution of the 0-hop multi-list attestation query logic and the subsequent update of the `AttestationAccumulator` for the involved private notes. For L1 UTXO-based systems engaging the lightweight path, SP1 could be used by the user to generate an efficient, non-recursive ZKP for these same attestation-specific operations.

2. For users who have opted into generating an attestation history for their private commitments (notes/UTXOs) and subsequently need to produce a full historical verification, SP1 provides the *primary mechanism envisioned for implementing the recursive composition* needed in the user-generated **heavyweight proof path**. Here, each step involves an SP1 execution (run by the user's software) proving both the application logic for verifying a historical multi-list attestation (from an IADP) and the validity of the SP1 proof from the preceding step in the attestation history chain.

Consequently, the practical efficiency of SP1's native recursion (for user-generated heavyweight proofs) and its general proving capability (for L2 operator-generated batch proofs of opted-in lightweight updates, or user-generated lightweight proofs on L1s) are central factors in the feasibility of the overall architecture. These influence system performance for users choosing to generate and verify historical attestations.

2.3 ZKP-Friendly Accumulators

Cryptographic accumulators offer a method to represent a potentially large set of elements with a single, compact value. They often allow for efficient proofs about set membership (or non-membership) without revealing the entire set. For integration into ZKP-based systems like ours, it is crucial that these accumulators are **"ZKP-friendly"**: their update operations (adding elements) and the verification of membership proofs must be computationally inexpensive when performed *inside* a ZKP circuit. Common ZKP-friendly approaches include:

- **Pedersen commitments.** These are a common choice due to their ZKP-friendliness and useful homomorphic properties. A typical 2-generator additive Pedersen commitment allows committing to a message m (represented as a field element) using a blinding factor bf as:

  $$\text{Commit}(m, bf) = mG_0 + bfG_1$$

  where G_0 and G_1 are independent group generators. This scheme is binding (hard to find two different openings for the same commitment) and computationally hiding (the commitment reveals no information about m). For our `AttestationAccumulator`, multiple attestation components are first cryptographically hashed into a single field element, which then serves as the message m in such a 2-generator commitment. (The formal syntax and construction are detailed in Appendix A.)

- **Hash-Based Constructions:** Using standard cryptographic hash functions (especially ZKP-optimized ones like Poseidon [Grassi et al., 2021] or the prover-friendly Reinforced Concrete [Grassi et al., 2022], which reduces S-box depth) to build structures such as Merkle trees or hash chains. While lacking algebraic properties like homomorphism, hash computations themselves can be implemented very efficiently within ZKP circuits.

In our proposed architecture, such ZKP-friendly accumulators are the core primitive used to construct the `AttestationAccumulator`. This compact state commitment, updated during the efficient **lightweight path**, cryptographically summarizes the necessary historical attestation data (detailed in Appendix A) without requiring the full history to be processed or stored on-chain for every transaction.

2.4 Private Data Retrieval Techniques

Verifying information against external, frequently updated datasets—such as sanctions lists or other registries maintained by oracles—is often necessary for compliance or attestation purposes. However, performing these queries directly poses a significant privacy risk, as it would typically leak the identifier being checked (e.g., a user's DID or other sensitive handle) and potentially reveal their status across multiple lists to the oracle(s). Cryptographic techniques exist to mitigate this.

One relevant category is **Oblivious Message Retrieval (OMR)** [Liu and Tromer, 2022] and the related field of Private Information Retrieval (PIR) [Goldberg, 2007]. These protocols allow a client (e.g., a user's wallet) to retrieve specific information from a server's database without the server learning which piece of information was requested. **PerfOMR** [Liu et al., 2024], for instance, is a recent, efficient OMR construction using Ring-LWE and homomorphic encryption, designed to minimize communication and server computation for such private lookups.

While such techniques could hypothetically be used for querying attestation lists (e.g., checking a DID or other identifier against a list held by an OMR-enabled oracle), our proposed architecture takes a different approach for the primary 0-hop attestation check. As detailed in Section 2.6, we leverage the **Decentralized Identifier (DID) and Verifiable Credential (VC)** model. In this model, the user obtains credentials offline from trusted issuers attesting to their status (e.g., non-membership on a list). During the transaction, the user generates a Zero-Knowledge Proof (ZKP) demonstrating possession and validity of the relevant VC(s), rather than performing a real-time, privacy-preserving query to an external oracle.

This choice is motivated by several factors favoring the DID/VC approach for the core attestation logic integrated into transactions:

- **Avoiding Real-Time Oracle Dependency:** VC proofs rely on pre-issued credentials, eliminating the need for live interaction with an oracle during the transaction, thus improving latency and robustness against oracle downtime.
- **Potentially Simpler ZKP Verification:** While ZKPs are needed in both models, verifying a VC proof (often involving signature checks or specific ZKP schemes like BBS+) may be less complex to implement within a transaction's ZKP than verifying the correct execution of an OMR protocol like PerfOMR (which involves proving homomorphic encryption operations).
- **Standardization and Ecosystem Alignment:** The DID/VC model utilizes established W3C standards and benefits from a broader ecosystem of tools and implementations compared to bespoke OMR-based attestation systems.

The choice of mechanism for 0-hop multi-list attestation verification—whether a Decentralized Identifier (DID) and Verifiable Credential (VC) based model or an alternative system relying on direct private oracle queries (e.g., leveraging PerfOMR [Liu et al., 2024] combined with a hypothetical bespoke "Attestation Identifier Key")—involves complex trade-offs regarding on-chain and off-chain dependencies, and ZKP circuit complexity. Both paradigms, while offering different cryptographic pathways, present technical challenges for seamless and efficient integration of their respective methods within ZKP circuits.

More fundamentally, any model that anchors attestations to an identity layer risks introducing new gatekeepers, particularly if that identity is state-issued or otherwise legally privileged. Whether through VCs issued against such an ID, or a directly provisioned Attestation Identifier Key, the entity controlling that underlying identity infrastructure could potentially throttle access, revoke credentials or identifiers ex-post, or impose economic or political hurdles. Such control can recreate the

very chokepoints and structural inequities that "permissionless" systems aim to avoid. The implementation specifics of any such identity linkage, and critically, the governance mechanisms surrounding it, are therefore paramount to whether the permissionless spirit is cultivated or neglected, irrespective of the cryptographic sophistication employed. Our proposed governance model for VC issuers within OptAttest (detailed in Section 6) is a direct attempt to address these concerns for the VC-based path.

Given these broader challenges, the preference for the DID/VC model in the present work is influenced by pragmatic considerations such as its comparatively broader (though still developing) ecosystem, ongoing standardization efforts for identity credentials, the ability to avoid real-time oracle dependency during transactions (relying on pre-obtained VCs), and the potential for less complex ZKP verification for VC proofs compared to proving full OMR protocol execution.

However, this design choice on its own will not confer immunity to the gatekeeper risks. The DID/VC model still depends upon an infrastructure of trusted issuers (see Section 6), reliable DID resolution, and verifiable revocation mechanisms. Ensuring user wallets can access up-to-date information for generating $\pi_{\text{VC\_validity}}$ is an ongoing challenge.

2.5 Folding Schemes (e.g., Nova, HyperNova)

Proving a long chain of computations recursively (Section 2.1) can be computationally expensive. Folding schemes are a cryptographic technique to mitigate this. They allow two computational tasks—each defined by its mathematical rules (its 'constraint system')—to be efficiently merged along with their proofs. The result is a single, combined representation that is significantly cheaper to use in the next recursive step than verifying both original task proofs separately [Kothapalli et al., 2022], making it more practical to verify long sequences of computations.

The primary advantage is dramatically lower **recursive overhead**—the extra work needed at each step of the chain. Instead of requiring a complex and slow verification of the entire previous proof, the system only needs to perform a much simpler and faster check related to the folding process itself.

Two prominent examples include:

- **Nova** [Kothapalli et al., 2022]: A pioneering scheme designed for tasks expressed in a common ZKP format known as **R1CS** (Rank-1 Constraint System). It achieves very low recursive overhead (e.g., two scalar multiplications).
- **HyperNova** [Kothapalli and Setty, 2024]: A successor that works with a more versatile range of mathematical rule sets, called **Customizable Constraint Systems (CCS)** [Setty et al., 2023]. This versatility is crucial because CCS can support various ZKP systems, including those underpinning modern tools like the SP1 zkVM (Section 2.2), facilitating easier integration. HyperNova also maintains very low recursive overhead (e.g., potentially a single scalar multiplication) and supports advanced features for more complex scenarios.

Both schemes demonstrate the power of folding to dramatically reduce the cost of each recursive step. For the system proposed in this paper (Section 1.3), folding schemes like HyperNova are a promising way to make the user-generated 'heavyweight proof' more efficient for the user's device. Using such folding schemes could offer lower overhead per step compared to the native zkVM recursion described in Section 2.2. However, effectively integrating these schemes with the proof outputs of general-purpose zkVMs like SP1 (e.g., bridging SP1's STARK-based proofs to HyperNova's CCS input) remains a research and engineering challenge (discussed further in Section 10).

2.6 Decentralized Identifiers (DIDs) and Verifiable Credentials (VCs): Tools for User-Driven, Accountable Self-Assertion

Decentralized Identifiers (DIDs) and Verifiable Credentials (VCs) serve as foundational elements for enabling users to make privacy-preserving, verifiable claims about themselves in relation to external data, such as specified lists. Users establish control over one or more DIDs, which function as stable, self-sovereign pseudonymous identifiers. This control enables them to selectively obtain VCs from diverse authoritative entities (hereafter "issuers"). A VC is a digitally signed statement asserting specific claims about the subject (the DID holder). Users store these native VCs securely in a digital wallet. The framework for establishing trust and accountability for these issuers is a critical matter of ecosystem governance (as speculated in Section 6), aiming to foster a plurality of trustworthy sources.

For the optional 0-hop attestation check in our proposed lightweight path, the user utilizes relevant, pre-obtained VCs, making a conscious choice about which claims to present in a given context. A key challenge with directly using diverse VCs within a per-transaction ZKP is the potentially high and variable cost of verifying different native signature schemes. To address this challenge while avoiding the imposition of a SNARK-generation burden on all VC issuers, OptAttest proposes a user-centric approach that enhances both efficiency and user agency:

User-Side VC Validity Pre-Computation. The user's wallet performs a periodic background task to generate a comprehensive ZKP, termed a "cached VC validity proof" ($\pi_{\text{VC\_validity}}$). For each relevant VC (or set thereof), this $\pi_{\text{VC\_validity}}$ attests to its full status. This user-managed pre-computation shifts the intensive cryptographic work off the critical transaction path, allowing users to prepare their attestable claims efficiently.

Efficient Per-Transaction 0-Hop Attestation. When the user initiates an OptAttest transaction and opts for the lightweight path, the main 0-hop ZKP for that transaction achieves efficiency by verifying the pre-computed $\pi_{\text{VC\_validity}}$. Upon successful verification, it trusts the claims attested to by $\pi_{\text{VC\_validity}}$ and uses them to construct the transaction's attestation data. This two-step process allows OptAttest to support diverse VC signature suites (and thus a broader ecosystem of potential issuers) with a uniformly low marginal cost within each transaction's ZKP.

This user-centric model for handling VC proofs allows individuals to prove statements like "I possess a valid, non-revoked VC from trusted issuer Y (whose trustworthiness is ideally assessed by the ecosystem's governance) asserting I am not on list X" *without revealing the VC itself or their full DID to the public blockchain or network operators during the transaction*. This capability for selective disclosure, grounded in zero-knowledge principles, is fundamental to achieving accountable privacy on the user's terms.

Admissibility rules for VCs (input to user's daily proof generation). While the per-transaction ZKP sees a standardized $\pi_{\text{VC\_validity}}$, the underlying VCs used to generate $\pi_{\text{VC\_validity}}$ must still adhere to certain standards for interoperability to ensure the user's wallet can process them effectively.

The core requirement is that the transaction's ZKP must cryptographically link the attestation to the party initiating the transaction, proving their control over the relevant identifier (DID) without exposing main addresses or compromising the zero-knowledge properties of the attestation itself. The existence and integration of such a privacy-preserving identity and attestation credentialing mechanism within the wallet infrastructure, including the capability for background pre-computation of VC validity proofs, is therefore a prerequisite for users wishing to exercise their option for 0-hop attestation checks.

3 Goal: Optional, Practical, and Private Hop 0 & Hop 1 Attestation

Our goal is to integrate optional yet meaningful regulatory attestation capabilities directly into a privacy-preserving digital asset architecture, without fundamentally compromising default user privacy or practical usability for those who choose not to engage this layer. We focus specifically on enabling users who opt-in to achieve verifiable historical attestation for their specific private commitments.

3.1 Verifying Multi-List Attestations Two Hops Back for Opted-In Commitments

Verifiable Hop 0 & Hop 1 attestation history is the ability for the holder of an attestation-enabled commitment to cryptographically verify the status of entities involved in the last two preceding transaction hops (which also must have been attestation-enabled) relative to a *user- or verifier-specified set of external lists*. That is, if Alice (who opted-in for attestations) sends funds to Bob (who also opts-in), who then sends them to Carol (who receives an attestation-enabled commitment), the system should allow Carol (or a party verifying her transaction, such as a VASP, if Carol provides the proof) to obtain cryptographic assurance regarding the attestations for Bob (Hop 0 relative to Carol's commitment) and Alice (Hop 1 relative to Carol's commitment) concerning lists {L1, L2,...} at the time of their respective transactions. The system itself does not enforce policy based on these attestations but provides the verifiable data for downstream decision-making. Standard private transactions that do not opt into this system remain unaffected and carry no such attestation history.

This optional hop 0 & hop 1 attestation would be a significant enhancement over simple 0-hop checks that typically bind to a single list or compliance definition. For users who choose to use it, it addresses concerns about opaque transaction histories and layering techniques by providing a limited, but verifiable, look-back into an opted-in commitment's recent associational provenance relative to diverse criteria. While deeper historical analysis (N>1) might be desirable in some contexts, targeting hop 0 & hop 1 for these attestation-enabled commitments provides a concrete, bounded goal that balances increased transparency with the escalating complexity of proving deeper histories. This approach enables various actors to apply their own interpretations and policies to verified historical attestations derived from opted-in funds.

3.2 Balancing Core Tenets for an Empowering and Resilient Optional Attestation Layer

Achieving the Hop 0 & Hop 1 attestation history goal for opted-in commitments must be balanced against four foundational system requirements for the attestation layer itself. These tenets are not merely technical trade-offs but reflect a commitment to user agency, trustworthiness, and ecosystem vitality:

1. **Privacy Preservation and User Sovereignty:** For transactions where users choose to engage the attestation mechanism, the system must uphold the default privacy characteristics of the underlying digital asset system. The attestation mechanism itself must not leak sensitive user information beyond what is proven in the user-generated attestation; historical data (stored in IADPs) must be handled privately, with historical properties proven in zero-knowledge by the user generating a heavyweight proof. Users who choose not to opt-in remain entirely unaffected, their autonomy respected.

2. **Cryptographic Verifiability and Integrity:** For attestation-enabled commitments, the Hop 0 & Hop 1 attestation history must be cryptographically verifiable by the user or any party they authorize by sharing the proof. Cryptographic verifiability assures data integrity, enabling trusted interactions based on user-provided evidence.

3. **Practical On-Device Performance and Accessibility:** The system must remain practical for end-users operating standard consumer devices when they choose to interact with the attestation layer. The computational cost for a user associated with the **lightweight path** must be minimal. The user-generated **heavyweight proof**, while intensive, is infrequent and only applies to attestation-enabled funds, ensuring the core system remains accessible.

4. **Pluralism and a Permissionless Ethos for Community Self-Determination:** Engagement with the attestation system is entirely optional, reflecting a user's sovereign choice. If engaged, the system itself does not embed logic that blocks transactions based on attestation outcomes. Its role is to provide verifiable information for those who choose to generate and use it. This design allows diverse users, applications, communities, and potentially even jurisdictions to apply their own contextually relevant policies independently, fostering a **pluralistic ecosystem** for attestation. This permissionless spirit is vital for resisting capture by singular, potentially fallible or unjust, interpretive authorities and for enabling community self-determination in defining standards of trust and interaction.

4 The OptAttest Hybrid System Architecture

4.1 Overview: Optional Lightweight Attestation Updates vs. User-Generated Heavyweight Historical Proofs

The OptAttest hybrid attestation architecture, introduced conceptually in Section 1.3, provides users with an optional layer for generating verifiable Hop 0 & Hop 1 attestation history. This system distinguishes between two primary operational paths for opted-in transactions, balancing ongoing efficiency with the capability for comprehensive historical verification, all while standard private transactions remain unaffected.

The first, the **lightweight path**, is engaged when a user chooses to create an attestation-enabled commitment. As outlined in Section 1.3, this path leverages user-provided Verifiable Credentials (VCs) for immediate (0-hop) multi-list attestations. The core architectural elements involved are the generation of an Intermediate Attestation Data Packet (IADP) to store detailed evidence of these VC-based attestations, and the update of a compact `AttestationAccumulator` associated with the new commitment. This accumulator efficiently summarizes the recent attestation history. The cryptographic integrity of these optional updates (including VC proof verification and accumulator logic) is ensured through ZKPs—either batch-verified by L2 operators using tools like SP1 (Section 2.2) or generated directly by users in L1 UTXO systems. This design prioritizes minimal overhead for users opting to create or receive attestation-enabled funds.

The second, the **heavyweight path**, is invoked less frequently by a user holding an attestation-enabled commitment, specifically when a full, verifiable report of its Hop 0 & Hop 1 attestation history is needed. This user-initiated process, detailed conceptually in Section 1.3, involves generating a recursive ZKP. Using locally stored IADPs as witness data, this proof validates the entire attestation chain summarized in the commitment's `AttestationAccumulator`. The primary

implementation strategy for this recursive proof relies on the native recursive capabilities of general-purpose zkVMs (like SP1, as discussed in Section 2.2), while folding schemes (Section 2.5) like HyperNova point toward a potential, albeit more complex, optimization strategy. Though computationally demanding for the user, its infrequent and targeted application preserves the performance of routine transactions.

This dual-path approach allows the system to offer verifiable historical data for those who choose to participate, without excluding those who choose not to. The following sections elaborate on the specific components (Section 4.2) integral to this architecture and detail the transaction flow (Section 4.3) for both standard private transactions and those that engage this optional attestation system.

4.2 Key Components of the Optional Attestation Architecture: Building Blocks for User-Driven Accountability

When users choose to engage the optional hybrid attestation architecture, it relies on the interplay of the following components:

- **`AttestationAccumulator`:** A compact cryptographic commitment associated with a note or UTXO **for which a user has proactively chosen to generate attestation history**. It represents the Hop 0 & Hop 1 attestation history summary for that specific attestation-enabled commitment. It utilizes ZKP-friendly constructions allowing efficient updates.
- **Intermediate Attestation Data Packets (IADPs):** User-generated, encrypted data packets containing the detailed attestation information for a specific 0-hop event (including the unique attestation nonce, VC proof summaries, and VC issuer references) summarized by an `AttestationAccumulator`. IADPs are created only if the user engages the lightweight path, are committed to within that transaction's attestation logic, transmitted privately, and are required as input for subsequent user-generated heavyweight proofs, ensuring the user retains control over the detailed evidence of their attestations.
- **Decentralized Identifiers (DIDs) and Verifiable Credentials (VCs):** The primary mechanisms enabling users to manage their digital persona and make self-directed attestations. DIDs serve as user-controlled identifiers. VCs, issued by trusted entities (whose trustworthiness is ideally subject to the ecosystem's governance as speculated in Section 6), contain the claims proven in zero-knowledge during the opted-in 0-hop check. The unique nonce for the attestation event, alongside VC issuer identifiers, are recorded within IADPs, allowing for traceability of claim provenance without exposing the user's subject DID.
- **VC Issuers, Revocation Infrastructure, and Ecosystem Governance:** External trusted entities responsible for issuing VCs and maintaining revocation mechanisms. We envision the framework for establishing and maintaining trust in these issuers as a matter of ongoing, pluralistic ecosystem governance (Section 6), aiming to ensure issuers are accountable and contribute to a vital, rights-respecting attestation environment. The ZKP in the lightweight path verifies VC validity against this governed infrastructure.
- **Secure On-Device Storage:** Local, encrypted wallet storage for received and decrypted IADPs and the user's own VCs. Such user-controlled storage enables asynchronous data management and provides the necessary historical witness data for users to generate heavyweight proofs of the attestation history for their opted-in commitments, reinforcing their data sovereignty.

4.3 High-Level Transaction Flow: User Agency in Optional Attestation

The transaction process within OptAttest respects user choice: it can either be a standard private transaction, or, if the user proactively decides, can engage the optional attestation system. It is crucial that all transaction types are processed by the network based on standard protocol validity, regardless of attestation choices or outcomes, thereby upholding the system's permissionless ethos.

1. **User Decision: Engage Attestation System for this Interaction?** The sender first exercises their agency by deciding whether to enable attestations for the output private commitment(s) of the current transaction. This choice is paramount.
    - **If No (Default Path):** The transaction proceeds as a standard private transaction according to the underlying protocol. The user's choice not to generate attestation data is fully respected, and the remaining steps in this flow do not apply.
    - **If Yes (Opt-In Path):** The sender intends to generate a verifiable attestation history for the output commitment(s), taking active control of the claims associated with this interaction. The following steps apply.
2. **Initiation (for an Attestation-Enabled Transaction):** The sender selects input private commitment(s). The sender's wallet also determines which external lists or criteria (e.g., {L1, L2,...}) are relevant for the 0-hop attestation for this specific transaction. This determination is ideally based on user preference, context, or the articulated needs of their counterparty, rather than a centrally imposed mandate, reflecting a user-directed approach to claim-making.
3. **Attestation Path Determination (Contextual User Choice):** The wallet determines if the transaction requires only the **lightweight path** (generating/updating attestation summaries for the new output) or if a full **heavyweight path** proof (verifying historical attestations for an *input* commitment) is also needed for this outgoing transaction. This decision is driven by the user's context and intent.
4. **Lightweight Attestation Path (Common for Opted-In Transactions):** This path is always followed if the user opted to engage the attestation system (Step 1) and aims to create/update an `AttestationAccumulator` for the output commitment(s).
    - Perform a 0-hop multi-list attestation based on the sender's pre-obtained Verifiable Credentials (VCs) for the contextually specified lists.
    - Prepare attestation data for the transaction, including witness data for the ZKP that will demonstrate the validity of the VCs and the correctness of the derived claims, and specify the correct update of the `AttestationAccumulator`.
    - Generate corresponding IADP(s) containing the necessary details (VC proof summaries, issuer information, specified lists, and verified claims, all linked to the unique attestation nonce) and commit to the IADP(s) hash.
    - Encrypt necessary IADP(s) for the recipient.

- Submit the transaction. The network (L2 operators or L1 ZKP verification) validates the correct execution of the attestation logic for these opted-in components.
- The transaction is processed based on standard validity rules, irrespective of the attestation content. The system facilitates verification but does not enforce policy on claims.
- The recipient of an attestation-enabled commitment verifies its validity, decrypts the IADP(s) (gaining insight into the sender's 0-hop verified claims), and manages storage. The recipient or their application can then apply their own policies based on these user-provided, verifiable claims.

5. **Heavyweight Attestation Path (Less Frequent, User-Initiated):** This path is taken if Step 3 determined a full historical proof is needed for an *input* attestation-enabled commitment. This proof is generated by the sender, providing a robust, verifiable account of that input's opted-in attestation history.
    - Retrieve required historical IADP(s) from local storage.
    - Generate the full, recursive ZKP (e.g., using SP1 locally). This proof verifies the input commitment's historical chain of multi-list attestations and its `AttestationAccumulator` integrity.
    - The transaction is constructed to include or reference this heavyweight ZKP.
    - Submit/Broadcast the transaction.
    - The recipient (or other authorized verifying party) verifies this heavyweight ZKP, obtaining verifiable assurance of the historical attestation data for that specific input commitment.

This flow allows standard private transactions to coexist with an optional, layered system for generating and verifying attestation histories, emphasizing user choice and control at each stage of engagement with the attestation features. The following sections detail the mechanisms underlying this attestation layer.

5 Core Mechanisms

5.1 `AttestationAccumulator`: User-Controlled State Representation & Updates for Opted-In Commitments

The `AttestationAccumulator` is a cornerstone of the **optional lightweight path**, utilized only when a user exercises their choice to generate attestation history for a note or UTXO. If opted into, it provides a compact, fixed-size commitment to the relevant attestation history associated with that specific attestation-enabled commitment, reflecting the user's decision to make certain claims verifiable. Its design uses ZKP-friendly accumulator principles. Standard private commitments, where users have chosen not to engage this layer, do not possess an `AttestationAccumulator`, underscoring the system's respect for default privacy.

5.1.1 Structure: Committing to User-Asserted Attestation History

To maintain the Hop 0 & Hop 1 history window for an attestation-enabled commitment, its `AttestationAccumulator` commits to cryptographic hashes. These hashes represent the multi-list attestations (specifically, commitments to the *verified*

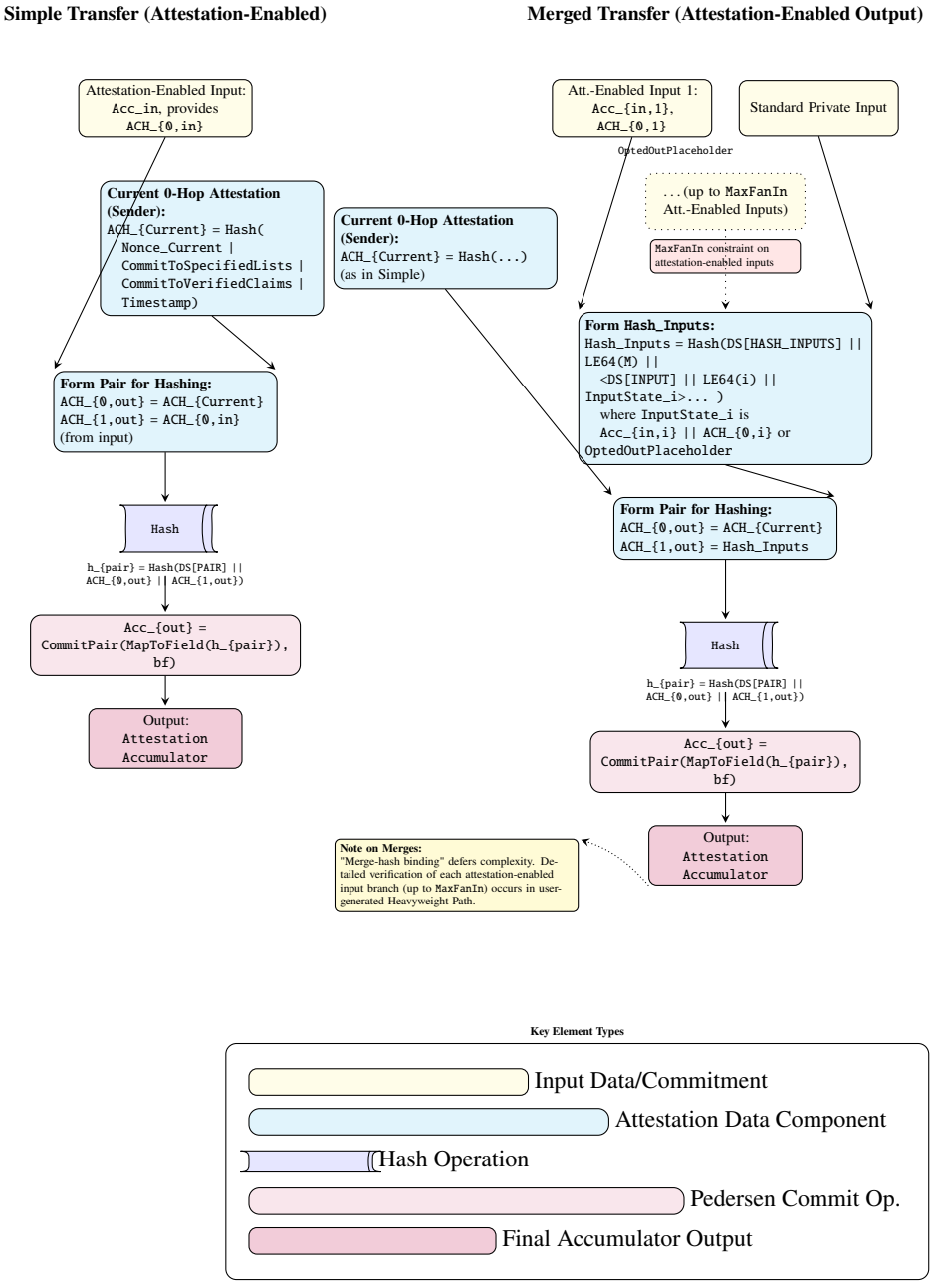

Figure 2: `AttestationAccumulator` Mechanics: Illustrating the formation of the accumulator for simple transfers (left) versus multi-input merges (right). Both paths culminate in a Pedersen commitment over a hashed pair representing current (`ACH_0`, using a `Nonce_Current`) and historical/merged (`ACH_1`) attestations. Merges utilize `Hash_Inputs` and respect the `MaxFanIn` constraint, deferring verification costs of attestation-enabled branches.

claims derived from the user's Verifiable Credentials (VCs)) for the transaction creating it (Hop 0) and the prior transaction (Hop 1, which must also have been attestation-enabled to contribute its history, reflecting a chain of user choices). The specific structure varies for simple versus merged transfers involving attestation-enabled commitments:

- For a simple (non-merge) transfer creating an attestation-enabled output commitment, the structure commits to the sender's current (Hop 0) attestation event (identified by their generated `Nonce_Alice_0`) and the prior (Hop 1) event from the input (identified by `Nonce_Charlie_1`).
  - `AttestationClaimsHash_0 = Hash(Nonce_Alice_0 | CommitmentToSpecifiedLists_0 | CommitmentToVerifiedClaims_0 | Timestamp_0)`
  - `AttestationClaimsHash_1 = Hash(Nonce_Charlie_1 | CommitmentToSpecifiedLists_1 | CommitmentToVerifiedClaims_1 | Timestamp_1)`
  - The combined pair hash:

    $$h_{\text{pair}} = \text{Hash}\big(\text{DS}[\texttt{PAIR}] \,\|\, \text{AttestationClaimsHash}_0 \| \text{AttestationClaimsHash}_1\big)$$

    (where $\text{AttestationClaimsHash}_1$ is a fixed zero-hash if no prior opted-in history exists).
  - The final `AttestationAccumulator` (`Acc`) is a Pedersen commitment:

    $$\text{Acc} = \text{Commit}(\text{MapToField}(h_{\text{pair}}), bf)$$

    The blinding factor 'bf' ensures privacy regarding the depth of the immediate prior attestation history.
- For a merged transfer (M inputs) creating an attestation-enabled output commitment, where inputs might have different attestation statuses (reflecting diverse prior user choices), the accumulator structure incorporates all their histories:
  - The current attestation-hash (sender's Hop 0, identified by `Nonce_Alice_Current`):

$$\text{AttestationClaimsHash}_{\text{Current}} = \text{Hash}\big(\text{Nonce}_{\text{Alice, Current}} \,\|\, \text{CommitToLists}_{\text{Current}} \,\|\, \text{CommitToClaims}_{\text{Current}} \,\|\, \text{Timestamp}_{\text{Current}}\big).$$

  - `Hash_Inputs = Hash(InputState_1 | ... | InputState_M)`, where each `InputState_i` is derived from an attestation-enabled input (carrying forward its nonce-identified attestation summary) or is the `OptedOutPlaceholder` if input i was a standard private commitment, explicitly acknowledging and respecting the choice not to attest for that input.
  - The combined pair hash using $\text{AttestationClaimsHash}_{0,\text{out}} = \text{AttestationClaimsHash}_{\text{Current}}$ and $\text{AttestationClaimsHash}_{1,\text{out}} = \text{Hash\_Inputs}$.
  - The final `AttestationAccumulator` (`Acc`) is formed as a Pedersen commitment to this pair hash.

The `Nonce` values are crucial for identifying specific user-initiated attestation events within the IADPs. The accumulator itself stores only compact commitments, preserving the privacy of the detailed claims and the identities of the VC issuers involved in each nonce-identified event.

5.1.2 Update Logic (Verifying User-Chosen Attestation Path)

When a user opts into the lightweight path, the correct computation of the output `AttestationAccumulator`(s) based on the input(s) and the current 0-hop multi-list attestation (verified via the user's VCs for their chosen lists and current nonce) must be ensured. This verification confirms that the party constructing the transaction correctly:

1. Opens input accumulator(s) for any attestation-enabled inputs.
2. Computes the current 0-hop `AttestationClaimsHash_Current` (incorporating the user's current nonce and verified VC claims).
3. Computes the historical component (e.g., `Hash_Inputs`), respecting the opted-out status of any standard private inputs.
4. Computes the new output accumulator commitment `Acc_out`.

This verification ensures the integrity of the summarized attestation state for opted-in commitments, reflecting the user's choices without enforcing any policy on the attestation content itself, thereby maintaining the system's role as a facilitator of verifiable claims rather than an enforcer of specific norms.

5.1.3 Handling Merges & Splits: Respecting Diverse Attestation Histories

Fan-in bound. To maintain practical prover costs for users who later generate heavyweight proofs, any transaction producing an *attestation-enabled* output respects the `MaxFanIn` limit for attestation-enabled inputs.

Transaction graph complexities involving attestation-enabled commitments are managed as follows, always prioritizing the integrity of user-chosen attestation paths:

- **Splits (1 Attestation-Enabled Input -> M Attestation-Enabled Outputs):** If an attestation-enabled input is split, its attestation history (summarized in `Acc_out`) is faithfully propagated to all M output commitments, which also become attestation-enabled.
- **Merges (M Inputs -> 1 Attestation-Enabled Output):** When a user opts to create an attestation-enabled output from M inputs (respecting the $M_{\text{opt-in}} \leq 2$ limit for attestation-enabled inputs), the "merge-hash binding" strategy is used. The output accumulator commits to `AttestationClaimsHash_Current` (the sender's current, nonce-identified attestation) and `Hash_Inputs`. `Hash_Inputs` represents all M input states, where attestation-enabled inputs contribute their prior nonce-identified attestation summaries, and standard private inputs contribute the `OptedOutPlaceholder`, explicitly acknowledging their opted-out status. The merge strategy maintains a fixed accumulator size while faithfully representing the combined attestation lineage chosen by the users involved.

Deferred Verification Cost (User-Controlled Proof of Merged Histories): This strategy defers the complexity of verifying the individual attestation histories encapsulated within `Hash_Inputs` to the user-generated **heavyweight proof path**. The 2 constraint ensures this user-side proving remains tractable. Standard private inputs (represented by `OptedOutPlaceholder`) contribute no further historical attestation branches to verify, respecting the prior choices of non-engagement. This design enables users to manage the complexity of their attestation histories.

5.2 Lightweight Path for Optional, User-Driven Attestation Generation

When a user exercises their choice to engage the attestation system for a transaction by opting into the **lightweight path**, their wallet performs several preparatory steps. These steps leverage pre-obtained Verifiable Credentials (VCs) to enable the user to make specific, verifiable claims about their current interaction. The correct execution of these user-initiated steps and the integrity of the resulting attestation data (which identifies the specific attestation event by a unique nonce, thereby anchoring the user's agency in this act of claim-making) are then ensured through cryptographic verification. Standard private transactions, where the user has chosen not to engage this path, remain unaffected.

5.2.1 User-Side Preparation for 0-Hop Attestation: Leveraging DIDs and VCs for Contextual Claims

If a user opts into the lightweight path for their transaction, they utilize their existing Decentralized Identifier (DID) and previously issued native VCs. The user's DID serves as a self-sovereign anchor for authorizing the use of their VCs, but it is not directly included in the attestation data propagated to recipients, preserving a layer of privacy. The process, managed by the sender's wallet, empowers the user through two main stages: periodic background pre-computation and context-driven per-transaction preparation.

1. Periodic Background Pre-computation (e.g., Daily or per VC Validity Period): The user's wallet, as a background task, generates and locally caches a comprehensive ZKP of full VC validity ($\pi_{\text{VC\_validity}}$). This pre-computation handles potentially expensive cryptographic operations off the critical path, ensuring that the user's choice to attest does not unduly burden transaction speed.

2. Per-Transaction Preparation (when opting into Lightweight Path): Before constructing the final transaction data for an OptAttest transaction, the sender's wallet, acting on the user's intent, performs the following:

1. Identifies the relevant cached VC validity proof(s) ($\pi_{\text{VC\_validity}}$) corresponding to the VCs needed to attest to their status against the **set of specified lists or criteria {L1, L2,...}** pertinent to *this specific transaction and interaction context*. This selection process reflects the user's agency in choosing which claims are relevant.

2. Generates a unique, cryptographically random nonce (`Nonce_Current`) for this specific 0-hop attestation event, providing a distinct identifier for this particular act of user-driven claim-making.

3. Prepares the necessary inputs for the main 0-hop OptAttest ZKP. This ZKP efficiently verifies the pre-computed $\pi_{\text{VC\_validity}}$ and its attested claims, then uses these claims along with the `Nonce_Current` to correctly form the `AttestationClaimsHash_Current` and update the `AttestationAccumulator`.

The main 0-hop OptAttest ZKP generated from these per-transaction steps becomes part of the witness data used to construct the transaction. This ZKP enables the OptAttest system to efficiently and privately verify the 0-hop attestation status for the specific nonce-identified event, reflecting the user's chosen claims for that interaction. Figure 4 illustrates this user-centric process, where the "ZKP Generation Module" focuses on verifying the cached $\pi_{\text{VC\_validity}}$ and incorporating the unique nonce.

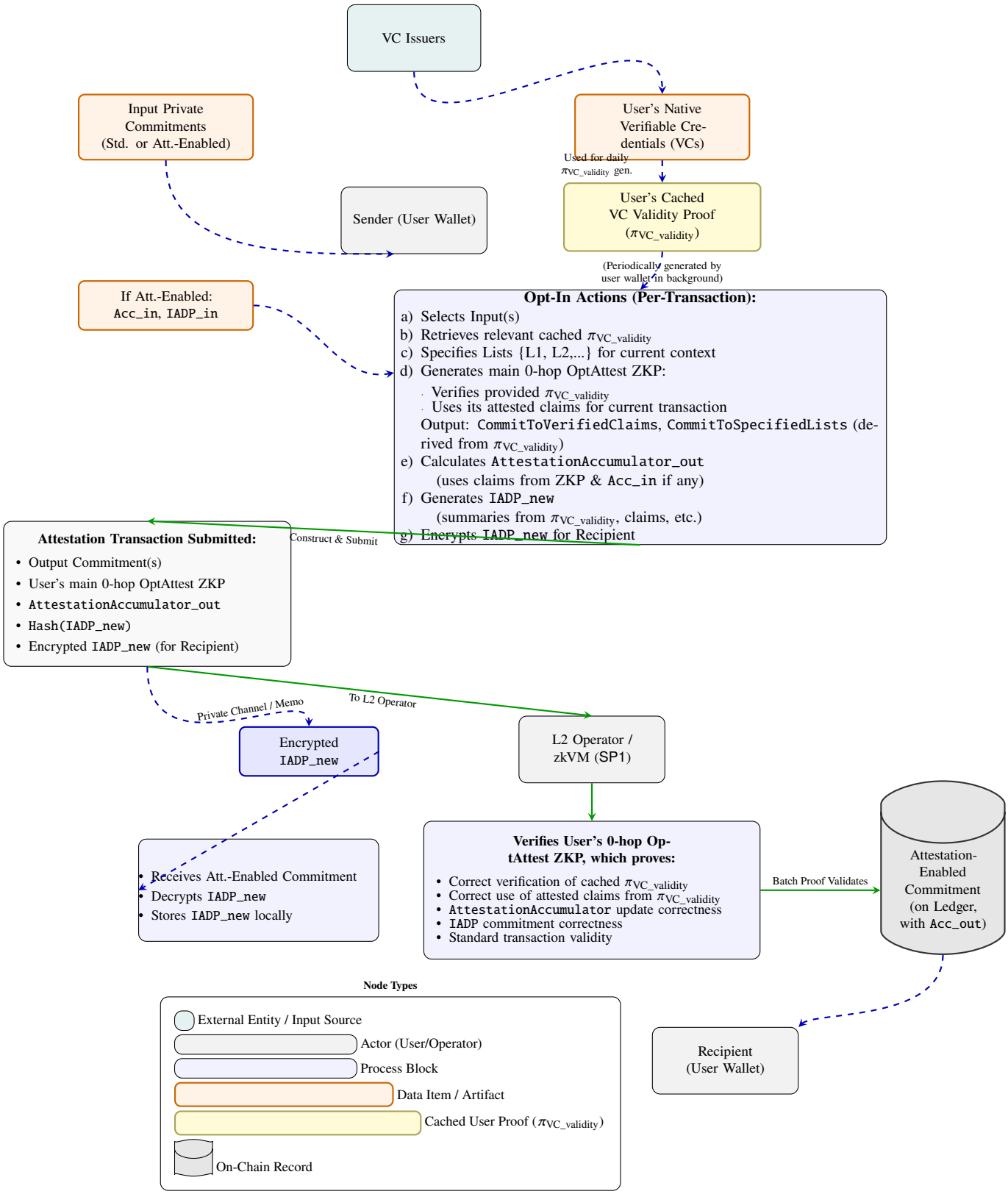

Figure 3: Lightweight Attestation Path: User's wallet periodically pre-computes and caches a VC validity proof ($\pi_{VC\_validity}$). For an opted-in transaction, the Sender generates a main 0-hop OptAttest ZKP that efficiently verifies $\pi_{VC\_validity}$ and its claims, calculates the `AttestationAccumulator` update, and generates an encrypted `IADP`. The L2 Operator verifies the user's ZKP and commits the attestation-enabled output. The Recipient decrypts and stores the `IADP`.

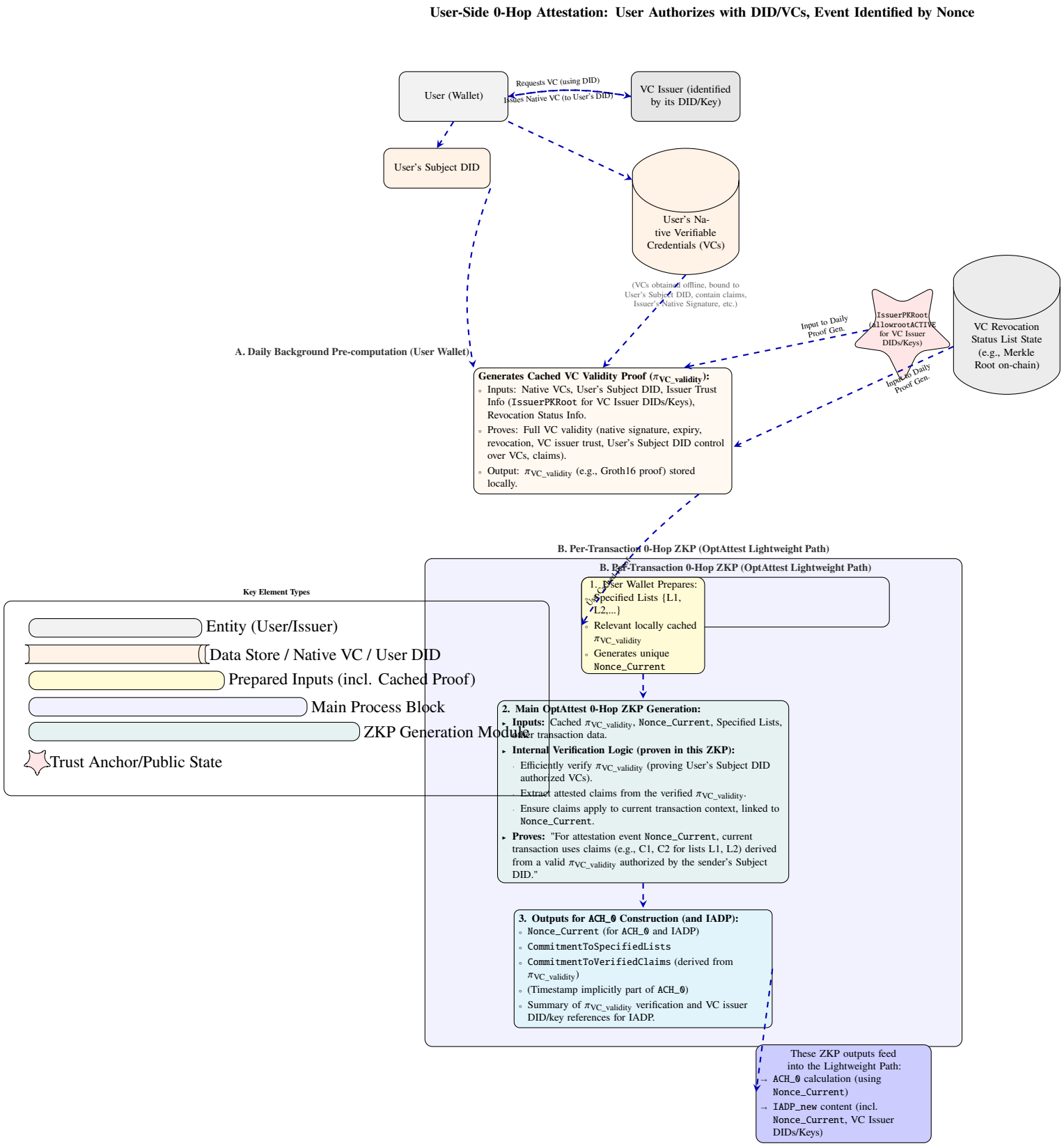

Figure 4: User-Side 0-Hop Attestation: User Authorizes with DID/VCs, Event Identified by Nonce. (A) User's wallet periodically generates a "Cached VC Validity Proof" ($\pi_{VC\_validity}$) for their native VCs, proving VC validity and authorization via their Subject DID. (B) For an opted-in transaction, the main 0-hop ZKP uses a new `Nonce_Current` and efficiently verifies $\pi_{VC\_validity}$ and its attested claims. This produces outputs for the Lightweight Path's `ACH_0` (using the nonce) and `IADP` (containing the nonce and VC issuer DIDs/keys, but not the user's Subject DID).

5.2.2 Ensuring Correctness of VC Proof, Accumulator Update & IADP Commitment

For a transaction where the user has opted into the lightweight attestation path using DIDs/VCs, the cryptographic verification process (either a user-generated ZKP on L1, or the L2 operator's batch proof using a zkVM like SP1) must establish the following regarding the attestation-specific components:

1. **Standard Validity (for the overall transaction)** is always checked, ensuring correct ownership of input private commitments (notes/UTXOs), value conservation, nullifier checks, etc., according to the base protocol rules.

2. **DID Control (for Sender's Authorization of VCs):** The system must ensure that the VCs, whose claims are being used for the current 0-hop attestation, were legitimately presented and authorized by an entity in control of the DID to which those VCs were issued. This assurance is primarily part of the generation of the user's cached VC validity proof ($\pi_{\text{VC\_validity}}$), which is then verified in step 3. The sender's subject DID itself is not included in the `AttestationClaimsHash` (which uses a nonce) or propagated in the IADP.

3. **0-Hop Attestation via Cached VC Validity Proof Verification:** This step verifies the integrity and applicability of the user's pre-computed cached VC validity proof(s) ($\pi_{\text{VC\_validity}}$) for the current transaction. This efficient check within the main transaction's ZKP ensures:

   - The provided $\pi_{\text{VC\_validity}}$ (e.g., a Groth16 proof) is itself a valid ZKP, typically verified using a constant-cost in-circuit verifier.
   - The statement proven by $\pi_{\text{VC\_validity}}$ is correctly utilized. This statement would encapsulate that, at the time $\pi_{\text{VC\_validity}}$ was generated (e.g., daily):
     - The underlying digital credential(s) (VCs) had their native signatures verified correctly.
     - The VCs originated from a trusted issuer (identifiable by their DID/key, e.g., not under $\mathsf{IssuerPKRoot}_t^{\mathsf{deny}}$ and potentially under $\mathsf{IssuerPKRoot}_t^{\mathsf{active}}$ at the time of $\pi_{\text{VC\_validity}}$ generation).
     - The VCs were current (not expired) and not revoked according to the status records checked during $\pi_{\text{VC\_validity}}$ generation.
     - The user demonstrated control of the DID to which the VCs are bound (authorizing their use for this attestation event).
     - Specific claims (e.g., confirming non-membership on specified lists like `SpecifiedLists`) were correctly derived from the VCs.
   - The main transaction ZKP ensures these attested claims from the verified $\pi_{\text{VC\_validity}}$, along with the unique `Nonce_Current` for this event, are correctly incorporated into the current transaction's attestation data (e.g., for the `AttestationClaimsHash_Current`).

   The complexity of deep VC verification (native signatures, comprehensive revocation checks against large lists) is thus handled within the user's periodic, background generation of $\pi_{\text{VC\_validity}}$, not within the per-transaction ZKP. The per-transaction ZKP only performs the efficient verification of $\pi_{\text{VC\_validity}}$.[4]

[4]The engineering challenges for VC interactions now shift to the user's wallet software for reliably generating $\pi_{\text{VC\_validity}}$. Reliably generating $\pi_{\text{VC\_validity}}$ includes managing diverse native VC cryptographic schemes for the initial proof generation, accessing up-to-date VC revocation status and trusted issuer information (including issuer DIDs/keys), handling credential updates, and securely managing

4. **`AttestationAccumulator` Update Correctness:** Verification that the output `AttestationAccumulator`(s) for the new attestation-enabled private commitment(s) were computed correctly following the specified logic (Section 5.1.2, Appendix A). Correct computation according to this logic includes handling splits/merges (incorporating `OptedOutPlaceholder` for any non-attestation-enabled inputs as per Section 5.1.3) based on the inputs and the set of *verified claims derived from the successfully verified* $\pi_{VC\_validity}$ *(associated with the current 0-hop nonce)*.

5. **IADP Commitment:** Verification that the corresponding Intermediate Attestation Data Packet (IADP) has been correctly formed by the sender (containing the `Nonce_Current` for this attestation event, along with summaries of the claims derived from $\pi_{\text{VC\_validity}}$, VC issuer DID/key references, revocation context from $\pi_{\text{VC\_validity}}$ generation, etc.; see Appendix D) and that its cryptographic commitment (e.g., hash) is consistent with the transaction outputs and the derived `AttestationAccumulator`.

This verification (whether user-generated ZKP or L2 operator batch proof) ensures the integrity of the immediate attestation step (now based on verifying pre-computed user proofs from VCs and identified by a unique nonce) for opted-in transactions and the correct evolution of the summarized attestation state for the newly created attestation-enabled commitment(s), linking it cryptographically to the detailed data (IADP) needed for potential future user-generated heavyweight verification. Standard private transactions not opting into this path do not undergo these attestation-specific checks.

5.3 Intermediate Attestation Data Management

Intermediate Attestation Data Packets (IADPs) are generated only when a user opts into the lightweight attestation path. For these attestation-enabled commitments, IADPs bridge the gap between the compact `AttestationAccumulator` state (which commits to an attestation event identified by a nonce) and the detailed attestation history required for the user to later generate a full hop 0 & hop 1 heavyweight proof. Standard private commitments have no associated IADPs.

5.3.1 IADP Content and Transmission

If the sender engages the lightweight path, they generate an IADP containing the raw data corresponding to the 0-hop multi-list attestation proven via Verifiable Credentials (VCs) for that transaction and its specific attestation event. These data are necessary as a witness for any future heavyweight proof verifying the history of the resulting attestation-enabled commitment(s). The specific content, detailed in Appendix D, includes:

- The unique random nonce (`Nonce_Sender_0`) generated by the sender for their 0-hop attestation event. This nonce is used in the calculation of the `AttestationClaimsHash_0` for this hop.
- The **set of lists for which attestations were proven** (`SpecifiedLists_Sender`).
- The **summary of the ZKP of VC validity** (`VCProofSummary_Sender`), which includes commitments to or essential features of the VCs used, references to their *issuers' DIDs/keys* and revocation status context, and the set of verified claims {R1, R2,...} for all specified lists.

the user's own DID for authorization. While the per-transaction ZKP becomes simpler, the overall ecosystem still relies on these components functioning correctly for the user's pre-computation.

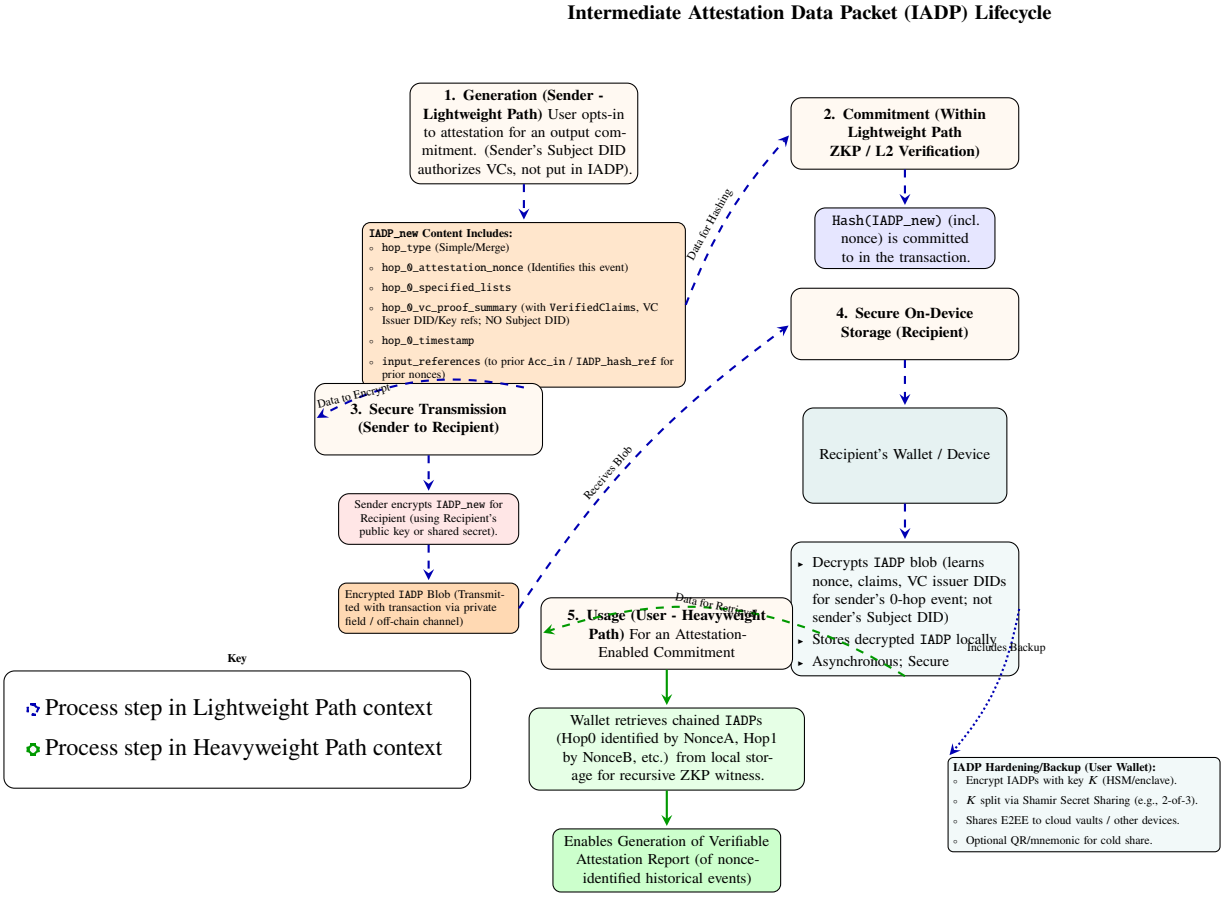

Figure 5: IADP Lifecycle (Nonce-Based Events): (1) Sender generates `IADP_new` for a nonce-identified 0-hop event (Subject DID authorizes VCs but is not in IADP; IADP includes nonce, claims, VC issuer DIDs). (2) Its hash (incl. nonce) is committed. (3) Encrypted and sent to Recipient. (4) Recipient decrypts (learns nonce, claims, VC issuer DIDs) and stores. (5) User retrieves IADPs (via nonce chain) for Heavyweight Proofs of historical nonce-identified events.

- Timestamp or block height.
- Cryptographic references (e.g., commitments or hashes) linking back to the IADP(s) / `AttestationAccumulator`(s) of any **attestation-enabled input commitments** used in this transaction. For merges involving mixed inputs (attestation-enabled and standard private), references are included only for the attestation-enabled inputs.

The sender encrypts the newly generated IADP (and potentially re-encrypts historical IADPs received with any attestation-enabled input funds being spent) using the recipient's public key or a derived shared secret. This encrypted IADP blob is included in a private field of the transaction data only when the lightweight attestation path is engaged. Standard private transactions carry no such encrypted blob.

5.3.2 Asynchronous On-Device Storage & Retrieval

The wallet of a recipient receiving an **attestation-enabled private commitment** (i.e., one created via the lightweight path and thus accompanied by an encrypted IADP blob) is responsible for managing the associated IADPs:

1. **Decryption:** Upon receiving such a transaction and verifying its validity (including the correctness of the lightweight attestation logic via the L2 operator proof or L1 ZKP), the wallet decrypts the associated IADP blob. The recipient learns the details of the sender's 0-hop attestation event, including the nonce, the claims proven, the lists attested against, and the pertinent VC issuer DIDs.
2. **Asynchronous Storage:** The wallet schedules a background task to store the decrypted IADP(s) securely in its local database. This task runs opportunistically, handling offline periods and retrying as needed ("near-chain" storage). Data are indexed according to the specific note or UTXO to which they pertain.
3. **Data Association:** The wallet maintains the chain of IADPs for its attestation-enabled commitments, linking each one back up to two hops through the stored data retrieved from predecessor IADPs (e.g., AttestationEnabledCommitment → IADP_for_EventNonceAlice → IADP_for_EventNonceCharlie → IADP_...). Standard private commitments are not part of this indexing.
4. **Retrieval:** When a user wishes to generate a heavyweight proof for spending a specific attestation-enabled commitment, their wallet retrieves the necessary chain of hop 0 & hop 1 historical IADPs (identified by their nonces and linked via references) from its local storage to use as private inputs for the prover generating the recursive ZKP. This retrieval is only possible for commitments with an attestation history.

This local, asynchronous management ensures data availability for the prover when generating heavyweight proofs for opted-in commitments, without impacting standard transaction speed. However, it necessitates robust backup, synchronization, and security strategies for the local storage (discussed in Future Work, Section 10). Loss or compromise of locally-stored IADPs would prevent the generation or compromise the integrity of heavyweight proofs of the attestation history for the associated attestation-enabled funds. Standard private funds are unaffected by IADP management issues as they do not rely on them.

Redundant backup and recovery. Relying on a single handset is brittle: if the device is lost or destroyed, the owner loses the ability to generate heavyweight

proofs for any unspent attestation-enabled funds tied to its IADPs. The wallet therefore *automatically hardens* IADP storage as follows:

1. IADPs are encrypted with an AEAD key K generated inside the wallet's HSM / secure enclave.
2. K is split into n shares using threshold Shamir secret sharing [Shamir, 1979]. Default parameters are (k=2, n=3): any two shares reconstruct K.
3. One share stays on-device; the others are *end-to-end-encrypted* and uploaded to user-selected cloud vaults (e.g. iCloud Keychain and Google Drive) or transferred to additional devices that run the same wallet. Each share is labelled with a BLAKE3 fingerprint so the wallet can detect tampering.
4. Optionally the user may export a share as a QR-code or mnemonic for cold offline storage.
5. The Merkle root of the share-fingerprint set is stored in the wallet and can be pinned on-chain in a future upgrade; having this Merkle root stored (and potentially pinned) lets an auditor prove that restored shares match the original root.

With two-of-three redundancy, any *single* loss event (phone theft, cloud-account lock-out, or misplaced paper backup) is survivable. The extra cryptographic material never leaves user custody, so privacy remains unchanged; only IADP availability is improved.

5.4 User-Generated Heavyweight Proof Path

This path is relevant only for notes or UTXOs that possess an attestation history because the user previously opted into the lightweight path during their creation or reception. It allows the current holder of such an attestation-enabled commitment to generate a proof verifying its Hop 0 & Hop 1 historical attestations (i.e., the sequence of nonce-identified attestation events and their outcomes). Standard private commitments without attestation history cannot utilize this path.

5.4.1 Triggers and Local IADP Use

The heavyweight proof generation path is triggered on-demand by the user holding an attestation-enabled commitment, typically under conditions such as: interaction with entities requiring historical attestations for such funds (e.g., regulated VASPs applying specific policies to attestation-enabled inputs), high transaction value involving an attestation-enabled commitment, periodic recertification requirements for certain applications using these commitments, or user-initiated requests for provenance reports on their own attestation-enabled funds. Upon triggering, the user's wallet retrieves the locally stored chain of IADPs corresponding to the hop 0 & hop 1 attestation history of the specific input attestation-enabled commitment(s) requiring proof (see Section 5.3.2). These IADPs will contain the attestation nonces, VC proof summaries (including VC issuer DID/key references and verified claims), and other metadata for each historical hop.

5.4.2 User-Side Recursive Proof Generation (Unrolling Attestation Accumulator)

The core of the heavyweight path involves the user's software generating a complete, recursive ZKP (Section 2.1) using the locally stored IADPs (Appendix D) as witness data. This proof verifies the integrity of the specific input commitment's `AttestationAccumulator` by computationally "unrolling" its summarized history. It uses the detailed IADP data (including nonces, VC issuer DIDs/keys, specified lists, and verified claims for each hop) to explicitly validate the sequence

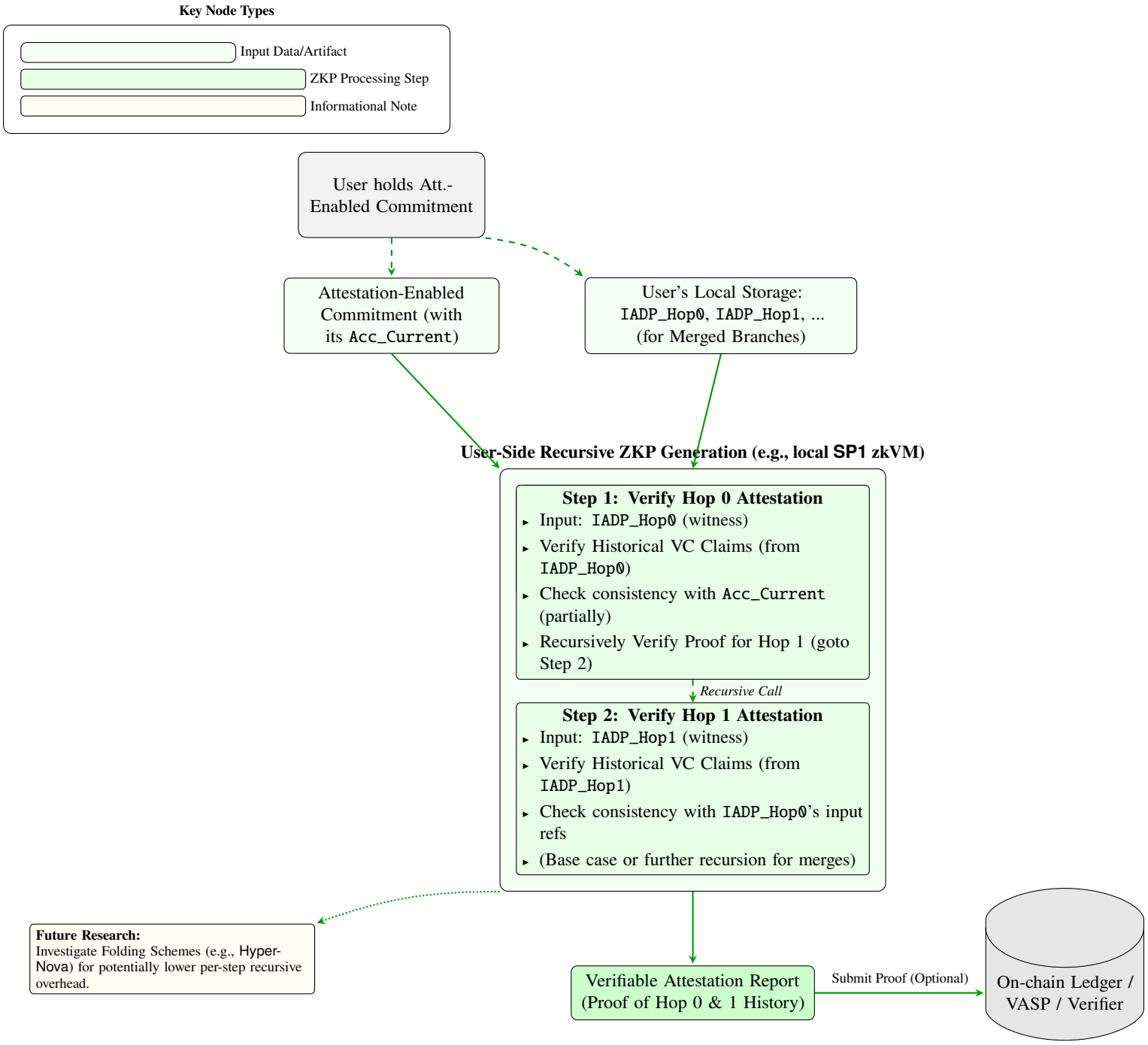

Figure 6: Heavyweight Attestation Path: User initiates with an attestation-enabled commitment, retrieves local IADPs, and generates a recursive ZKP (e.g., via local SP1 zkVM) "unrolling" the `AttestationAccumulator` to verify historical VC-based claims for Hop 0 & Hop 1. The output is a Verifiable Attestation Report.

of 0-hop multi-list attestations (derived from VC proofs for specific nonce-identified events) and their outcomes back two hops through the attestation-enabled lineage. Two main implementation approaches for this user-generated proof are considered:

First, the primary envisioned implementation uses a general-purpose zkVM (e.g., SP1, Section 2.2)—a software tool capable of creating proofs in sequential stages—executed locally by the user's software. In each stage (validating the recent Hop 0 attestation event and then the prior Hop 1 attestation event of the input commitment's history), the SP1 execution proves:

1. Correctness of the original multi-list attestation check from that historical point (identified by its nonce in the IADP), using data from the IADP corresponding to that historical point's check (the stored evidence packet, including VC proof summaries and VC issuer DID/key references). This step verifies that the claims were legitimately derived from VCs presented by an authorized subject DID (though the subject DID itself is not revealed by this proof to the final verifier).

2. Consistency between these IADP data (including the nonce used to compute the historical `AttestationClaimsHash` for that hop) and the compact summary (the `AttestationAccumulator`) for that historical link for opted-in funds.

3. Successful verification of the cryptographic proof (ZKP) from the *prior stage* using the zkVM's internal checking mechanism [Succinct Labs, a].

This recursive SP1 execution process builds a verifiable chain of evidence for the two preceding nonce-identified attestation events of opted-in funds. Handling merged inputs (as described in Section 5.1.3) requires recursively verifying each branch that originated from an attestation-enabled input (identified via the `Hash_Inputs` structure), leading to the prover cost scaling ("merge blowup") discussed previously. Branches corresponding to opted-out inputs do not require recursive verification. The practicality hinges on the concrete performance of SP1's native recursion for this user-side task, requiring benchmarking (Section 10).

Because $M_{\text{opt-in}} \leq 2$, the recursive prover executes at most 2 branch verifications per step, guaranteeing $O(2 \cdot N)$ witness size with $N = 2$ recursion depth.

Second, as an alternative optimization path for the user-side proof, highly efficient **folding schemes** (Section 2.5) like HyperNova (due to its CCS compatibility, see Section 2.5) could potentially be employed for the recursive aspect. In this model, the user's software would use SP1 to execute the 0-hop multi-list attestation verification logic for a historical nonce-identified step (validating claims, VC issuer trust, etc., from IADPs), and this computation would then be incrementally "folded" with the result from the previous step using the folding scheme's efficient verifier logic. While this folding-scheme-based model offers the potential benefit of lower per-step cryptographic overhead compared to native zkVM recursion, it introduces the significant R&D challenge of efficiently bridging the proof systems (e.g., SP1's STARK-based output to HyperNova's CCS input) within the user's proving environment, as discussed in Section 2.5 and Section 10. HyperNova's multi-folding capability (Section 2.5) might also offer cryptographic advantages for handling the merge blowup by combining multiple attestation-enabled branches, though witness processing costs likely remain.

Regardless of the specific recursive technique employed by the user, the final output of the heavyweight path is a single ZKP attesting to the validity of the entire Hop 0 & Hop 1 attestation history (composed of nonce-identified attestation events and their properties like claims, VC issuer DIDs/keys, and lists checked) derived from the IADPs for the specific attestation-enabled commitment. The final

ZKP provides a verifiable report of its historical attestations, without revealing the subject DIDs of the historical attesters to the verifier of this report.

5.4.3 Handling Performance for User-Generated Proofs (Optimized Schemes, Potential Offloading)

Generating the computationally intensive heavyweight recursive proof locally requires performance management strategies for the user. Options include:

- Utilizing the most efficient implementation of the chosen zkVM's native recursion (e.g., SP1, Section 2.2) on the user's hardware.
- Investigating advanced folding schemes (e.g., HyperNova, Section 2.5) if the integration challenges (Section 10) can be overcome to yield net performance benefits for local proving.
- Performing proof generation as a background task on the user's device, accepting potentially significant latency (minutes or longer).
- Optionally allowing the user to offload proof generation to external trusted servers or decentralized prover networks [Roy et al., 2024], which involves user-side trade-offs regarding trust (potentially revealing IADP data – including nonces, claims, and VC issuer DIDs/keys – to the prover), privacy, and cost versus speed.

The appropriate strategy depends on the specific application's requirements for speed, trust, and privacy, and the user's resources. The key architectural advantage remains that this high computational cost is incurred infrequently by the user, and only for attestation-enabled funds when proof is needed, thus preserving the performance of standard private transactions and the efficiency of the lightweight path operations.

6 Governance of Verifiable Credential Issuers

The core cryptographic mechanisms of OptAttest are largely *governance-adjacent*: they provide a technical framework for users to generate and verify attestations based on Verifiable Credentials (VCs). However, the actual trustworthiness of these VCs—and thus the utility and ethical integrity of OptAttest itself—hinges on the probity and accountability of their issuers, typically recognized through a mechanism like an `IssuerPKRoot`. The questions of *how* a set of trusted VC issuers is curated, and *by whom* such curation is stewarded, are governance challenges that extend far beyond mere technical correctness. The ethics, legitimacy, and *vitality* of attestations within the OptAttest ecosystem depend critically on establishing robust, transparent, and resilient governance for VC issuers. This section outlines foundational design goals for such a governance layer and proposes an illustrative model for its realization. Drawing inspiration from emerging theories on democratic material control, pluralistic collaboration, and the imperative for vitalist ethics in socio-technical systems [Dávila, 2023, Weyl et al., 2024, Marsh, 2024], our objective is to translate these principles into a practical framework for VC issuer curation, one capable of stewarding a new form of digital public infrastructure for trust.

6.1 Design Goals for a Pluralistic, Vital, and Capture-Resistant Attestation Ecosystem

A governance layer for VC issuers within an OptAttest ecosystem must aspire beyond functional resilience and legibility. It should cultivate a healthy, pluralistic, and non-predatory environment for verifiable claims. Cultivating such an environment involves embedding the following core principles:

1. **Upholding Universal Human Rights and Fundamental Freedoms:** The UDHR and similar foundational rights frameworks must serve as a non-negotiable ethical floor. Any issuer whose practices or attested claims demonstrably undermine these rights requires remediation or mitigation by the governance body.

2. **Legibility, Accessibility, and Transparency:** Governance processes for VC issuers, including evaluation criteria and decision rationales, must be transparent and understandable to a broad audience, encompassing non-technical users and the diverse communities relying on OptAttest attestations.

3. **Distributed Stewardship and Resilience Against Capture:** The issuer trust infrastructure is a common resource. Its governance must embody mechanisms for decentralized, democratic control by its users and affected communities [Dávila, 2023]. The nature of the issuer trust infrastructure as a common resource necessitates proactive design for resistance to capture by narrow interests, ideological rigidity, or ossification into a centralized controlling belief structure (an "endo-ideal"[5] [Marsh, 2024]), thereby ensuring its ongoing service to the entire ecosystem.

4. **Contextual Integrity and Support for Plural Publics:** Recognizing that attestations and their issuers operate within diverse social contexts, issuer governance must respect emergent social boundaries, diverse cultural norms, and the specific needs of different communities ("plural publics" [Weyl et al., 2024]). The aim is to support a rich ecosystem of varied and contextually appropriate issuers, rather than imposing a monolithic, one-size-fits-all standard that may be ill-suited or unjust in certain contexts.

5. **Promotion of Vital Reciprocity and Prevention of Systemic Harm:** The OptAttest governance framework must evaluate VC issuers based on their contribution to a balanced, beneficial, and humane attestation ecosystem. Such an evaluation includes assessing whether an issuer's data sourcing, operational practices, or the types of VCs they offer could inadvertently cause or facilitate relational or systemic harm. Such evaluation must consider if VCs might enable predatory data collection (akin to "anima extraction," referring to unbalanced, life-energy depleting interactions[6]) or if the attestations risk reinforcing harmful societal narratives or power imbalances (i.e., propagating damaging closed belief systems, or "endorealities"[7]), even if an issuer appears technically compliant with minimal standards [Marsh, 2024]. The objective

[5] In Marsh's framework [Marsh, 2024], an "endo-ideal" refers to a central belief, figure, or goal within a closed social system (an "endogroup") that dictates behavior and often serves to maintain the group's power structure, potentially at the expense of individual well-being or broader societal health. In the context of OptAttest, this could be a dominant faction controlling issuer approval or a rigid ideology about what constitutes a "good" attestation.

[6] "Anima extraction" [Marsh, 2024] describes processes or interactions where one entity draws vital life energy ("anima") from another in an unbalanced, often predatory, manner, leading to the depletion of the source. In the context of VC issuers, anima extraction might relate to exploitative data practices for obtaining VCs, or issuing credentials that structurally lead to user disempowerment or dependence.

[7] An "endoreality" [Marsh, 2024] is a closed system of belief, perception, and power maintained by

is to curate and cultivate issuers whose credentials contribute to a vital, just, and empowering digital environment.

6. **Ethical Dynamism and Reflexive Governance:** No system is perfect; new challenges and potential misuses of attestations will emerge. The governance framework for issuers must therefore be inherently adaptive and self-correcting. It requires built-in mechanisms for critical re-evaluation, reflexive learning from both successes and failures, and ongoing transformation through broad-based democratic engagement by the communities it serves [Weyl et al., 2024, Dávila, 2023]. Such adaptability and reflexivity ensure the ecosystem remains aligned with its core values and responsive to evolving needs.

Achieving these goals involves navigating tensions between efficiency, decentralization, the quality and trustworthiness of issuers, and the ethical responsibilities inherent in enabling verifiable claims about identity, status, and associations. The subsequent sections propose an illustrative model designed to begin addressing these vital tensions.

6.2 The Challenge of Trust for VC Issuers: Beyond Formal Compliance Towards Relational Ethics and Vital Ecosystem Health

The "OFAC paradox" discussed previously illustrates a challenge inherent in establishing trustworthy Verifiable Credential (VC) issuers: mere adherence by an issuer to published rules for credentialing, or their technical capacity to issue signed VCs, offers no guarantee that the issuer is trustworthy or that their VCs contribute positively to the health and justice of the OptAttest ecosystem. An issuer's operations can be simultaneously *compliant* with formal standards yet *complicit* in perpetuating harms if the underlying data sources are flawed, the rules themselves are unjust, or the application contexts of their VCs lead to negative social consequences (e.g., by issuing VCs that inadvertently legitimize or support a harmful "endoreality" [Marsh, 2024]). This recognition compels any robust governance framework for VC issuers within OptAttest to aspire beyond abstract rule-following and technical validation.

Instead, such a framework must aim to cultivate and assess what might be termed *relational ethics* and *vital considerations* in how issuers are evaluated and how they operate. For an OptAttest VC issuer, adhering to these relational ethics and vital considerations means the governance body (envisioned in §6.3) must consider:

- **The Provenance, Integrity, and Systemic Impact of Information Underlying their VCs:** Considering these factors involves inquiry into the data sources for the claims an issuer attests to. Are these sources auditable, transparently maintained, current, and gathered with demonstrable integrity and respect for rights? What are the foreseeable consequences—including potential systemic ripple effects and impacts on diverse communities—of users presenting VCs from this issuer? Does the credential genuinely empower users and enhance their agency, or does it risk endangering them or others, creating new forms of dependency or discrimination, or reinforcing existing inequities?

an "endogroup." For VC issuers, this could involve issuing credentials based on flawed or biased data sources that reinforce discriminatory systems, or offering attestations that obscure underlying inequities, even if the attestations are narrowly/technically accurate.

- **Assessing The Issuer's Relational Stance, Accountability, and Practices of Reciprocity** means evaluating the issuer's operational transparency regarding its data sources, claim verification methodologies, VC lifecycle policies (issuance, revocation, data retention), and its own internal governance. Is the issuer demonstrably accountable to the communities affected by the VCs they provide—both the subjects of VCs and the relying parties within OptAttest? Do their operational practices and terms of service for VC acquisition promote trust, informed consent, and *vital reciprocity* (i.e., balanced, mutually beneficial, and humane exchange, as per Design Goal 5 in §6.1)? Or do they risk becoming opaque, unaccountable authorities, creating exploitative dependencies, or engaging in practices akin to "anima extraction" (predatory depletion of users' vital energy, agency, or resources[8])?

- **The Potential for Misuse and Propagation of Systemic Harm through their VCs:** Could the types of VCs an issuer provides, even if facially neutral or technically accurate, be instrumentalized for widespread surveillance, undue discrimination, the creation of harmful social classifications, or the systemic disempowerment of certain groups? For example, could their VCs, due to their data sourcing, claim structure, or typical application contexts, inadvertently contribute to societal "anima depletion" or reinforce predatory dynamics within the digital ecosystem [Marsh, 2024]? The governance framework must prioritize issuers whose VCs are demonstrably designed and deployed to prevent such systemic harm and instead promote positive relational outcomes.

This perspective underscores that an OptAttest VC issuer's trustworthiness is not a static, check-box property determined at initial approval. Rather, it is an ongoing, relational quality that must be continuously assessed, nurtured, and held accountable by a dynamic and ethically attuned governance framework. It highlights the imperative for governance mechanisms that are sensitive to these deeper ethical, relational, and vital dimensions, even if these are more challenging to quantify than an issuer's mere compliance with technical standards. Operationalizing such assessments, perhaps through evolving, community-informed methodologies like "relational impact statements" or "ecosystem vitality reviews" (as further explored in §6.3 and §6.5), becomes a central challenge and aspiration for OptAttest.

6.3 An Illustrative Model: Towards Pluralistic, Vital, and Constitutively Legitimate Stewardship of VC Issuer Trust

This subsection outlines an illustrative model for governing the Verifiable Credential (VC) issuer trust roots within OptAttest. This model aims for robust pluralistic participation and vital stewardship of the attestation commons. The legitimacy of this governance structure is envisioned as *constitutively earned* through its transparent, accountable, and effective functioning.

6.3.1 Core Curation via a Plural DAO/Council: An Evolving Locus of Distributed Authority

The primary stewardship of the issuer trust roots ($\mathsf{IssuerPKRoot}_t^{\mathsf{candidate}}$, $\mathsf{IssuerPKRoot}_t^{\mathsf{active}}$, $\mathsf{IssuerPKRoot}_t^{\mathsf{deny}}$) could ultimately be vested in a **Plural Issuer Governance**

[8] "Anima depletion" [Marsh, 2024] refers to the loss or exhaustion of vital life energy ("anima") in an individual or system, often resulting from unbalanced or predatory interactions, leading to reduced well-being and agency. For VC issuers, anima depletion could manifest if their processes for VC issuance are exploitative, or if the VCs they issue lead to users feeling disempowered, overly surveilled, or locked into detrimental systems.

DAO (or Council). This body would be a novel form of distributed, digital public infrastructure governance.

- **Composition, Bootstrapping, and Evolution towards Broad Representation:** Initially, this body might be bootstrapped by a council of known, reputable entities or experts. A critical, explicitly defined goal would be to transition this initial council towards a broadly representative and directly accountable multi-stakeholder DAO. This transition would prioritize mechanisms ensuring that the DAO's authority remains derived from and responsive to the diverse OptAttest user communities ("plural publics" [Weyl et al., 2024]) it serves, mitigating risks of early capture or entrenchment of initial power structures.

- **Decision-Making Mechanisms and Anti-Capture Strategies for Sustained Legitimacy:** To promote collaboration across difference, ensure procedural fairness, and mitigate capture by dominant factions or vested interests as the DAO matures [Weyl et al., 2024], this body must progressively adopt and combine sophisticated governance tools. These tools are not mere technical features but are essential for its ongoing legitimacy and its ability to resist co-option:

  - **Augmented Deliberation Platforms:** Utilizing tools like Polis for nuanced, transparent, and broadly accessible discussion and consensus-finding on VC issuer applications, the evolution of ethical guidelines, and the refinement of policy criteria (e.g., for operationalizing "relational impact statements" or "vitality audits" as discussed in §6.2).
  - **Weighted, Representative, and Influence-Resistant Voting:** Employing mechanisms such as Quadratic Voting (QV) for key decisions to better reflect preference intensity and breadth of support [Weyl et al., 2024]. To further bolster resistance to undue external influence on DAO/Council members—a critical concern, particularly when evaluating issuers entangled with contentious geopolitical mandates or lists—the incorporation of advanced cryptographic voting protocols is paramount. Such protocols would aim to provide verifiable proof of correct decision execution while preserving the confidentiality of individual members' votes or contributions, thereby shielding them from targeted pressure and enabling principled decision-making even in adversarial environments.
  - **Resource Allocation for Ecosystem Health and Vitality:** Using tools like Quadratic Funding (QF) or participatory budgeting to allocate DAO-stewarded resources towards independent audits of issuers, research into vital metrics for ecosystem health, or initiatives that enhance the OptAttest ecosystem's relational integrity and the trustworthiness of its attestations [Weyl et al., 2024].
  - **Transparency, Forkability, and Exit Rights as Foundational Anti-Capture Measures:** Ensuring all governance processes, decisions (at an aggregate level, respecting shielded contributions where necessary), and financial flows are highly transparent. The underlying rule-sets and potentially even the DAO's operational framework must be "forkable," allowing subgroups to exit and form alternative governance structures if they fundamentally disagree with the main DAO's direction [Dávila, 2023]. This right of exit serves as a powerful deterrent against ossification, ideological rigidity, or capture, ensuring the ecosystem can adapt and retain the trust of its diverse constituents.

6.3.2 Roles for Specialized Entities: Enhancing Discernment and Accountability in Issuer Evaluation

Established NGOs, academic bodies, or other expert groups could play vital, though explicitly non-monopolistic, roles within this ecosystem, particularly in providing critical, independent input to the governance body:

- **Independent Auditors & Assessors of Relational and Vital Impact:** Providing in-depth reviews of prospective or active VC issuers. These reviews would extend beyond technical compliance to focus on operational integrity, accountability to affected communities, the provenance and reliability of data sources, and the potential systemic impacts and ethical risks of the VCs they issue, including assessments of "relational impact" or "anima audits" (see §6.2).
- **Nominators and Endorsers from Diverse Contexts:** Such groups could nominate or publicly endorse candidate VC issuers, leveraging their specific contextual knowledge and expertise to enrich the DAO/Council's decision-making.
- **Facilitators of Ecosystem Repair:** In cases of VC issuer misconduct or when VCs lead to demonstrable harm, these entities might help facilitate community-led restorative processes, focusing on repairing harm and ensuring issuer accountability.

6.3.3 The Three Merkle Trees: Transparently Reflecting Governed Issuer Status

The Merkle tree structure ($\mathsf{IssuerPKRoot}_t^{\mathsf{candidate}}$, $\mathsf{IssuerPKRoot}_t^{\mathsf{active}}$, $\mathsf{IssuerPKRoot}_t^{\mathsf{deny}}$) remains a useful on-chain mechanism for transparently representing the evolving, DAO-governed status of VC issuers' public keys at a given epoch t. Transitions between these states are direct outcomes of the Plural DAO/Council's deliberative and accountable processes, based on reviews against transparently established criteria informed by the Design Goals (§6.1) and Relational Ethics principles (§6.2).

6.3.4 Forkability, Contextual Lists, and Plural Sovereignty in Attestation: The Cornerstone of Resilience

A non-negotiable element for ensuring the resilience, pluralism, and long-term ethical viability of OptAttest is the principle of **forkability and contextual self-determination** regarding VC issuer trust. This principle directly addresses the inherent risk of any single governance body, even one as carefully designed as the Plural DAO/Council, becoming a single point of control, capture, or failure to meet the evolving needs of diverse stakeholders.

- While the Plural DAO/Council would steward the "default" or broadly endorsed global trust roots, the OptAttest architecture must technically and conceptually guarantee support for the use of alternative or supplementary issuer trust lists. Users and applications must be able to select issuer lists curated by different, independent governance bodies or communities [Dávila, 2023].
- This architectural feature would enable different "plural publics" [Weyl et al., 2024] using OptAttest to establish and enforce their own context-specific criteria for trusting VC issuers, reflecting their unique values, risk appetites, relational needs, and specific use cases for attestations. Such empowerment

Illustrative Model: Governance of VC Issuer Trust for OptAttest

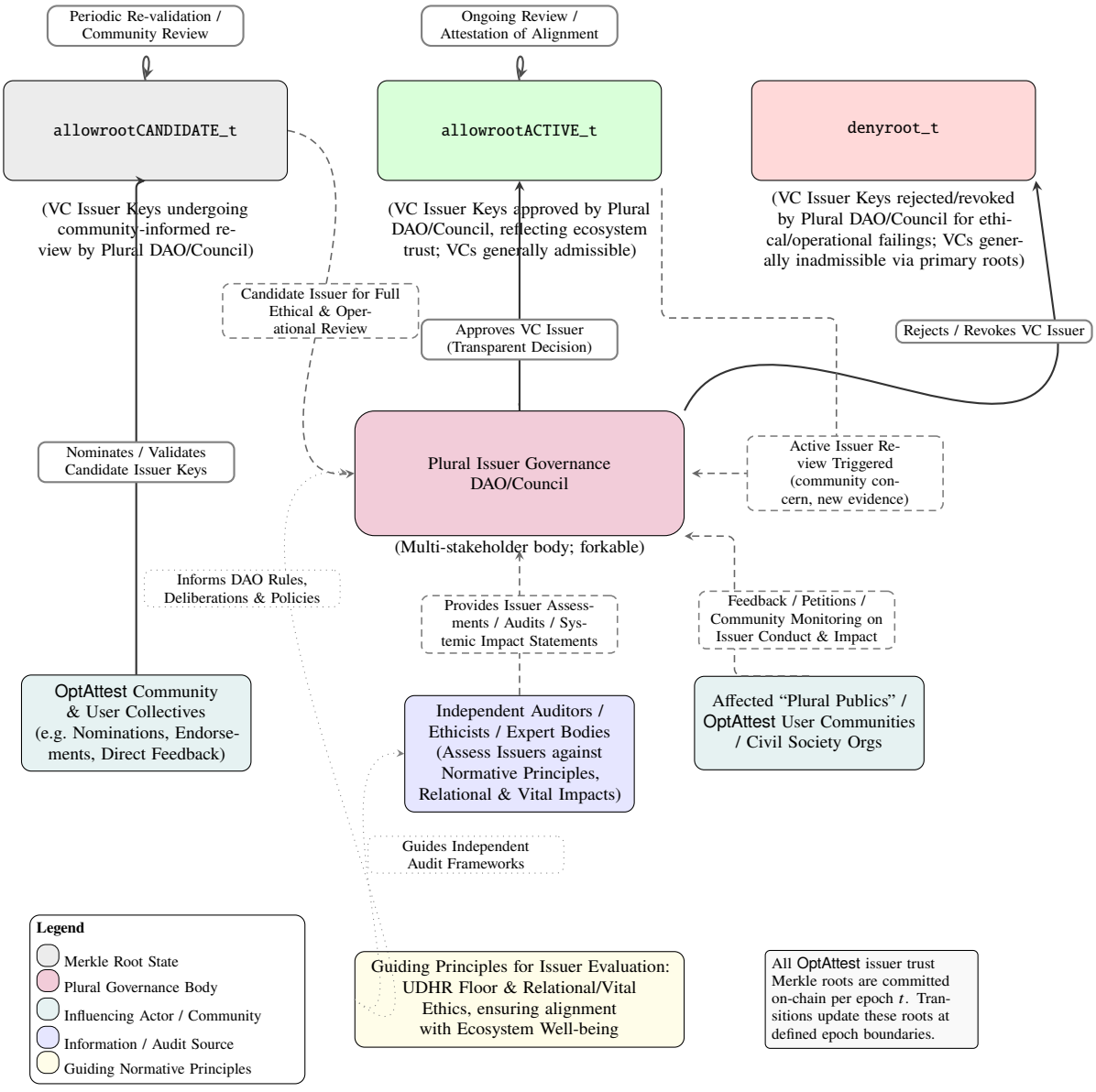

Figure 7: Illustrative Model: Governance of VC Issuer Trust for OptAttest. Transitions between VC-issuer key states are stewarded by a DAO/Council whose legitimacy derives from transparent, ethical processes, independent audits, and affected-public feedback. Forkability promotes contextual trust lists and polycentric sovereignty.

is an expression of polycentric sovereignty within the attestation ecosystem, obviating the need for permission from a central global body for every niche or specialized context.

- This "forkability" of trust serves as a check on the power of the main Plural DAO/Council, providing a mechanism to prevent the primary trust roots from becoming a chokepoint or an instrument of coercion.

6.4 Normative Baselines for VC Issuer Governance: Upholding Rights, Cultivating Vitality, and Navigating Sovereign Contradictions

The Universal Declaration of Human Rights (UDHR) serves as an essential, *non-negotiable foundational ethical floor* for the conduct and evaluation of Verifiable Credential (VC) issuers within the OptAttest ecosystem. Any issuer whose operational practices, data sourcing for VCs, or typical use-cases of provided VCs demonstrably and systematically violate UDHR principles should be inadmissible for inclusion in OptAttest's primary trusted issuer roots, as determined by its pluralistic governance body.

However, a governance framework aspiring to a vital, resilient, and plural attestation ecosystem, as outlined in §6.1, must engage with frameworks emphasizing the *quality of interactions* and their impact on individual and collective well-being (e.g., [Marsh, 2024]). Engaging with such frameworks requires the Plural Issuer Governance DAO/Council to consider factors beyond minimal compliance or non-violation of rights, thereby deliberately shaping an ecosystem conducive to human flourishing:

- **Prioritizing Relational Well-being and Vital Reciprocity in Attestation Practices:** Candidate or active VC issuers must be assessed not merely on their adherence to formal standards, but on whether their overall practices demonstrably contribute positively to the relational health and *vital reciprocity* of the OptAttest ecosystem. Such an assessment involves a holistic evaluation of their data acquisition (consent, transparency, minimization), interaction with VC subjects (clarity, fairness, redress mechanisms), and the intrinsic nature of the claims within their VCs. The overarching aim is to promote balanced and fair exchanges in the credentialing process, actively working to prevent the creation of predatory dependencies or exploitative relationships for any party involved.

- **Scrutinizing Systemic Impacts and Proactively Mitigating Potential for Relational Harm:** The Plural DAO/Council, aided by independent auditors and ethicists from diverse backgrounds, must consider the potential for even well-intentioned or technically accurate VCs from a particular issuer to generate unintended negative systemic consequences. Such consideration includes evaluating whether VC issuers might inadvertently create or exacerbate social stratification, enable disproportionate surveillance of vulnerable groups, reinforce harmful biases, or foster exploitative dynamics (e.g., VC issuers becoming unavoidable gatekeepers that disproportionately burden certain individuals, potentially causing widespread "anima extraction" or depletion [Marsh, 2024]). The goal is to proactively identify, mitigate, or exclude VC issuers or issuer practices that could systemically undermine relational well-being, justice, or the vital energy of the communities OptAttest aims to serve.

The "OFAC paradox" and similar situations—where state mandates for lists (which an issuer might attest to) directly conflict with these deeper relational ethics,

UDHR principles, or the imperative to prevent systemic harm—present critical and complex cases for the Plural DAO/Council's evaluation and discernment. This evaluation must encompass both the issuer's practices in handling such attestations and the ethical standing of the specific lists themselves. The DAO/Council's decisions regarding such issuers or list attestations must be reached through transparent, robust, and broadly accessible community deliberation (potentially utilizing augmented deliberation tools [Weyl et al., 2024]), and all rationales must be rigorously documented. The primary aim in navigating such sovereign contradictions is to minimize harm, uphold the core ethical principles and vitality of the OptAttest ecosystem, and maintain the earned trust of its diverse users. Strategies for navigating these inherent tensions include:

- Permitting an otherwise ethically sound VC issuer to issue VCs attesting to certain state-mandated lists, but mandating (e.g., through technical standards for VC presentation or wallet display guidelines within the OptAttest ecosystem) that wallets or relying party applications surface clear, contextual warnings about the potential ethical ambiguities, known harms, or disputed legitimacy associated with relying on such specific attestations. Surfacing such warnings provides end-users with critical context.
- The Plural DAO/Council actively promoting, incentivizing (e.g., through QF or grants), and prioritizing VC issuers who offer alternative, ethically robust VCs. These alternatives should be based on independently verifiable, community-endorsed, or humanitarian criteria, thereby promoting a principled, diverse, and resilient attestation landscape within OptAttest.
- In cases of severe and irreconcilable conflict with UDHR principles or a high, demonstrable risk of systemic harm, the Plural DAO/Council may, through its legitimate governance process, decide that VCs attesting to certain specific lists are inadmissible through the primary OptAttest trust roots. This decision would stand irrespective of an issuer's compliance with external mandates, while always respecting the fundamental forkability principle (§6.3.4) which allows distinct communities to recognize them differently if they so choose.

This approach seeks to balance pragmatism with ethics.

6.5 Open Questions and the Evolutionary Path of VC Issuer Governance: Stewarding a Living Attestation Commons

In aspiring to be robust, ethical, and empowering, this governance model for Verifiable Credential (VC) issuers within OptAttest naturally raises questions that underscore the ongoing, evolutionary nature of the endeavor. These are not merely technical puzzles but inquiries into the nature of constituting and sustaining legitimate, decentralized authority for a vital digital commons:

- **DAO/Council Viability, Constitutive Legitimacy, and Enduring Capture Resistance:** How can the Plural Issuer Governance DAO/Council itself remain vital, participatory, and exhaustively resistant to capture by special interests, dominant factions, or resource-rich entities seeking to control issuer approval, especially in the face of potential external pressures from established sovereigns? What specific mechanisms, beyond those outlined in §6.3, are needed to ensure its own internal operational integrity, its accountability to the diverse OptAttest user base, and its capacity for effective "collaboration across difference" [Weyl et al., 2024] rather than succumbing to internal power dynamics or becoming an ossified "endo-ideal" [Marsh, 2024]? How

will the crucial transition from an initial bootstrapping council to a fully plural and representative DAO be managed to build and maintain legitimacy while actively preventing early capture?

- **Operationalizing Advanced Issuer Evaluation Metrics for Relational and Vital Health:** What are actionable, sufficiently objective, yet qualitatively rich "proxies for relational and vital health" (inspired by [Marsh, 2024]) that the DAO/Council could use to evaluate VC issuers beyond formal compliance checks and self-attestations? How can concepts like "relational impact statements" or pilot "anima audits" of issuers be developed and implemented to generate meaningful, non-bureaucratic, fair, and contestable assessments of their impact on the OptAttest ecosystem, its users, and broader societal dynamics? What interdisciplinary expertise and community-governed resources are essential for this?

- **Ensuring Pluralism and Self-Determination via Forkable and Contextual Issuer Trust:** How can the OptAttest system technically and socially ensure effective, practical support for "forkable governance charters" for VC issuer trust [Dávila, 2023, Weyl et al., 2024] as outlined in §6.3.4? What specific technical standards and social infrastructures are needed to empower diverse "plural publics" to meaningfully curate and govern their own context-specific VC issuer lists for use with OptAttest, thereby exercising a form of distributed sovereignty over their own trust frameworks, while maintaining desired levels of interoperability or baseline security?

- **Balancing Anonymity, Accountability, and Trust for Diverse VC Issuers in Contested Spaces:** How can the governance system accommodate the potential, legitimate need for pseudonymous or community-attested VC issuers (e.g., for activist groups or those operating under repressive regimes needing to issue VCs discreetly to protect their members [Dávila, 2023]) with the benefits of richer, verifiable issuer identities that enhance accountability? What nuanced gradations of issuer verification or context-specific trust might be appropriate and how can these be justly administered?

- **Fostering Ethical Dynamism and Reflexive Governance Processes:** What specific, embedded processes and open forums will ensure that the entire governance framework for VC issuers—its rules, ethical principles, dispute resolution mechanisms, and the Plural DAO/Council's mandate—remains subject to ongoing critical review, robust contestability, and community-driven evolution? How can the system learn from failures, harms, or near-misses without ossifying or resorting to overly centralized control, thereby truly embodying "reflexive governance" [Weyl et al., 2024]?

- **Resource Allocation and Sustainability for Independent Governance Functions:** How would the Plural DAO/Council and its essential functions (such as funding independent audits, maintaining deliberation platforms, supporting broad community participation, and developing vital metrics) be sustainably and transparently resourced in a manner that preserves its independence, ethical orientation, and unwavering resistance to capture when making critical judgments about VC issuers?

- **Interplay with Protocol-Level Governance and Navigating the "Duality of Law Enforcement":** How does the governance of VC issuers interact with the governance of the core OptAttest protocol itself? More broadly, how does this entire ecosystem maintain its ethical integrity and protect its users when faced with demands or pressures from existing law enforcement or state

authorities, acknowledging their dual role as both potential protectors against harm and potential agents of unjust persecution? What principles guide this engagement to prevent co-option while enabling responsible interaction where aligned with fundamental rights and ecosystem values?

The governance of VC issuers for OptAttest is thus envisioned not as a static solution or a fixed constitution, but as an ongoing, dynamic process of socio-technical co-design, experimentation, reflection, and adaptation. It must be driven by the diverse communities it serves, and committed to promoting an empowering and vital digital commons. The technical framework of OptAttest should serve as a resilient and flexible foundation upon which such evolving, vital, and pluralistic governance can be built, tested, refined, and defended.

7 Security & Privacy Analysis

7.1 Privacy Guarantees

The system architecture aims to protect user privacy against several potential adversaries. A core principle is that users not opting into the attestation layer retain the full privacy guarantees of the underlying digital asset protocol (e.g., private L2 rollup or L1 privacy coin) without generating any additional attestation-related metadata. For users who choose to engage the optional attestation mechanism by presenting Verifiable Credentials (VCs) via Zero-Knowledge Proofs (ZKPs) to authorize their 0-hop attestations (which are then identified by nonces), the following privacy considerations apply:

- **Versus the Public (On-Chain/L1 Data Privacy):** Standard transaction details (sender, receiver, amount, specific notes/UTXOs involved) are assumed confidential via the underlying privacy protocol. For transactions where the user opts into the lightweight attestation path, the added components rely on cryptographic opacity: the `AttestationAccumulator` utilizes hiding commitments (Section 2.3), transmitted IADPs are encrypted (and typically sent via a private channel or memo field), and the ZKPs verifying the attestation logic (whether user-generated on L1 or operator-generated for L2 batches) uphold the zero-knowledge property (Section 2.1). These proofs reveal only the validity of attestation claim verification (derived from VCs for a specific nonce-identified event) and historical linkage for opted-in commitments. They do not reveal the underlying data (such as the subject DIDs used by historical attesters to authorize their VCs, the specific VCs presented, the individual attestation results beyond the committed claims, or the full history contained in IADPs) to passive observers of public L1 data. The attestation nonces themselves, while part of the committed data within the accumulator, are also effectively hidden from public view by the accumulator's commitment scheme.
- **Versus Attestation Issuers/Oracles (During Transaction):** An advantage of the DID/VC model (Section 2.6) for the 0-hop check is that there is no direct interaction with external oracles or VC issuers at the time of the transaction. Users present ZKPs based on previously obtained VCs (authorized by their DIDs). This setup eliminates metadata leakage (e.g., query timing, IP address) to these entities at the time of the transaction. The privacy of obtaining VCs initially depends on the issuer's policies and processes, which occur offline relative to the transaction.
- **Versus the Recipient:** If a transaction involves the lightweight path, the recipient receives an attestation-enabled commitment and an associated

encrypted IADP. The decrypted IADP contains only the cryptographic information necessary for potential future heavyweight proof generation for that specific lineage of attestation-enabled funds. This necessary information includes the unique nonce identifying the sender's 0-hop attestation event, VC proof summaries (including references to the VC *issuer's* DID/key), the verified claims, specified lists, timestamp, and references from Appendix D. It does not inherently reveal extraneous sender information beyond what is intrinsic to this attestation event data itself (which pertains only to the opted-in history of verified VC claims for nonce-identified events). Cryptographic commitments within the attestation logic verification prevent fabrication. Standard private commitments received without attestation do not convey any IADPs.

- **Potential Leakage Points:** Despite these protections, metadata leakage remains possible, especially related to the choice to opt-in:
  - **Opt-In Signal:** The very act of engaging the lightweight path (creating an `AttestationAccumulator` and sending an IADP) could potentially be a distinguishable characteristic if most transactions remain standard private ones. The significance of this potential distinguishability depends on the anonymity set of users opting in.
  - **Timing/Frequency of Heavyweight Proofs:** As before, the generation (and potential submission within a transaction) of a heavyweight proof for an attestation-enabled commitment could signal specific user activities (e.g., VASP interactions requiring historical attestation reports for such funds).
  - **Merge Structure Visibility:** The distinct structure of the `AttestationAccumulator` after a merge involving attestation-enabled commitments (Section 5.1.3, Appendix A) might reveal that a merge occurred and potentially hint at whether inputs were attestation-enabled or opted-out, even if input specifics are hidden.
  - **VC Characteristics Leakage:** While ZKPs can prove claims from VCs without revealing all attributes, the type of VC presented (inferred from the VC issuer DID/key or claims structure within the IADP if a recipient sees it, or more broadly from the ZKP structure if known) might itself leak some information (e.g., "user presented a proof related to a KYC credential issued by Issuer X" or "user presented a proof related to three distinct list attestations"). The granularity of selective disclosure and unlinkability depends on the specific VC scheme (e.g., BBS+, ZK-SNARK based VCs) employed [Hyperledger Foundation, 2025].

  Mitigations for these leakages may be necessary depending on the target privacy level and the expected usage patterns of the optional attestation feature.

7.2 Security and Integrity Properties

The security and integrity of the attestation mechanism rely on the properties of the underlying cryptographic primitives and the logic enforced by the ZKPs:

- **ZKP Soundness:** The cryptographic soundness property of the chosen ZKP system (e.g., SP1, Section 2.2, including its native recursive mechanism, Section 2.1) ensures that a malicious prover cannot generate a valid proof

for a transaction with falsified attestation data (e.g., incorrect claims for a nonce-identified event) or a forged attestation history, except with negligible probability. This soundness property underpins the core *verifiability* of the attestation reports generated by the system.

- **Attestation State Integrity:** The lightweight ZKP rigorously enforces the correct update logic for the `AttestationAccumulator` as specified in Section 5.1.2 and Appendix A. By proving the output accumulator state is correctly derived from validated inputs and the complete set of current 0-hop multi-list claims (verified from VCs for a specific nonce-identified attestation event), it prevents manipulation of the summarized attestation history. Preventing such manipulation relies on the ZKP soundness and the binding property of the accumulator commitment (Section 2.3), which commits to hashes incorporating these nonces and claims.

- **DID Control (for Sender's Own 0-Hop Authorization) & VC Unforgeability:** The lightweight ZKP, by verifying the sender's cached $\pi_{\text{VC\_validity}}$ (Appendix B.1, Assertions 2 and 3), ensures that the VCs used to derive claims for the current 0-hop nonce-identified event were legitimately authorized by an entity in control of the subject DID to which those VCs were issued. The verification of $\pi_{\text{VC\_validity}}$ prevents a user from falsely claiming attestations based on another user's VCs for their own attestation event. The verification of the VC's cryptographic integrity (e.g., issuer's signature, non-revocation, trusted issuer DID/key) within $\pi_{\text{VC\_validity}}$ (and thus by the lightweight ZKP) prevents the use of forged or invalid VCs. Such prevention relies on the knowledge soundness of the ZKPs and the security of the underlying digital signature schemes used for subject DIDs and VCs. The sender's subject DID is used for this authorization but is not itself included in the `AttestationClaimsHash` or propagated in the IADP.

- **Attestation History Integrity:** The commitment to the IADP within the lightweight ZKP (Appendix B.1, Assertion 5) cryptographically binds the detailed historical attestation data (including the nonce for the current event, claims, and VC issuer DID/key references) to the transaction and its resulting accumulator state, preventing retroactive tampering or dissociation. The subsequent heavyweight recursive proof (Section 5.4.2) validates this entire chain of nonce-identified attestation events based on the linked IADPs. It relies on the soundness of the recursive composition (Section 2.1) and the integrity enforced by the lightweight ZKPs at each historical step (which verified the correct formation of the `AttestationClaimsHash` using the nonce and claims for that event). The validation by the heavyweight recursive proof ensures the final attestation report is a faithful representation of the recorded historical verified claims and their associated metadata (like VC issuer DIDs/keys for each nonce-identified event).

7.3 Systemic Resilience Against Malign Use (e.g., Ransomware) and Principled Engagement with External Authorities

A primary design goal of OptAttest is to deter predatory use, such as the laundering of ransomware proceeds, without resorting to a panopticon that compromises fundamental privacy rights or creates tools ripe for abuse by those same authorities it might seek to inform. The system aims to achieve this delicate balance by shifting the focus from direct tracking of historical user identities within IADPs to scrutinizing the verifiable quality, depth, and provenance of attestation *events* themselves (identified by nonces and characterized by their claims, the Verifiable

Credential (VC) issuer DIDs/keys, and the lists attested against).

Several interconnected mechanisms contribute to this deterrence:

- **Focus on VC Issuer Integrity and Ethically-Governed Accountability:** The trustworthiness of the entire attestation chain heavily relies on the diligence, integrity, and robust ethical governance of VC issuers (identifiable by their own DIDs/keys). A resilient, pluralistic governance framework (as speculated in Section 6) for vetting, monitoring, and holding issuers accountable is paramount. If predatory actors cannot easily obtain "clean" VCs from reputable issuers operating under such principled oversight (especially if DIDs associated with predatory activity are flagged or if issuers perform meaningful due diligence appropriate to the claims they attest), their ability to generate legitimate-looking 0-hop attestations is severely hampered. The difficulty for predatory actors to obtain 'clean' VCs places significant pressure on the quality, reliability, and ethical conduct of the entire VC issuer ecosystem, making it a first line of defense.
- **Empowering VASP Policies through Rich Attestation Data (Not Identity Disclosure):** When evaluating deposits of attestation-enabled funds accompanied by a Heavyweight Proof, Virtual Asset Service Providers (VASPs) and other relying parties can enforce stringent criteria based on the rich data revealed about historical nonce-identified attestation events, *without needing the subject DIDs of historical attesters*. Such criteria, defined by the VASP according to their own risk assessment and regulatory context, may include:
  - *Minimum Attestation Depth and Quality:* Requiring a verifiable history of N "clean" attestation events.
  - *Trustworthiness of Involved VC Issuer DIDs/Keys:* Prioritizing attestations involving issuers recognized by robust governance or independent audits.
  - *Relevance and Legitimacy of Lists Attested Against:* Evaluating not just that a list was checked, but whether the list itself is considered credible and non-prejudicial.
  - *Attestation Freshness and Contextual Relevance.*

  These VASP-level policies, applied to the properties of the attestation chain itself, create a high bar for the quality of attestation history a predatory actor must present, increasing their operational friction without compelling universal identity disclosure.
- **Increased Operational Complexity and Friction for Predatory Actors:** To create a convincingly deep and high-quality attestation history using "clean" DIDs, predatory actors face significant operational hurdles. The need to manage multiple DIDs, obtain VCs from reputable and ethically governed issuers for each, and correctly perform OptAttest steps for each hop introduces considerable friction, cost, and risk of detection through off-chain means or operational errors.

OptAttest thus endeavors to make it significantly harder for predatory actors to convincingly fake a robust history of compliant attestation events. The system relies on the integrity of the governed VC issuer ecosystem and the diligence of relying parties in interpreting the rich historical attestation data that OptAttest makes verifiable.

Regarding engagement with external authorities, including law enforcement, OptAttest is guided by the following principles to navigate the "duality" of their role (as both potential protectors and potential persecutors):

- **No Direct LE Tooling or Privileged Access:** OptAttest is not designed as a direct surveillance or enforcement tool for existing LE agencies. There are no backdoors, special decryption keys, or privileged APIs that would allow LE to bypass user consent or the ZKP-based privacy protections for attestation data. This stance is critical to prevent the importation of existing biases or the potential for abuse inherent in current LE structures.
- **User-Controlled Disclosure as the Sole Mechanism for Sharing:** If a lawful and rights-respecting request is made by an appropriate authority, a user *can choose* to generate a Heavyweight Proof from their locally stored IADPs and provide the resulting verifiable attestation report. OptAttest provides the technical means for such voluntary, bounded disclosure, but the decision and control reside with the user.
- **The Plural Issuer Governance DAO as an Ethical Buffer:** The governance body for VC issuers (Section 6) plays a vital role in upholding the ethical standards of the OptAttest ecosystem. If external authorities pressure a VC issuer to issue VCs based on unjust criteria, to share information improperly, or to otherwise act against the ecosystem's principles, the DAO's responsibility is to act as an ethical buffer. Acting as an ethical buffer might involve refusing to recognize such an issuer or flagging VCs derived from compromised processes, thereby asserting the ecosystem's autonomy in defining trust.
- **Focus on Systemic Resilience and Deterrence:** The primary aim is to make the *system* unattractive and risky for broad categories of problematic fund flows by requiring high-quality, multi-hop attestations from reputable, ethically-governed VC issuers for those choosing to engage the attestation layer. This systemic approach raises the bar for all who opt-in, indirectly hindering those who cannot meet these standards legitimately, rather than focusing on individual tracking.
- **Forkability as an Ultimate Safeguard:** The principle of forkability in VC issuer trust governance (§6.3.4) serves as a final safeguard. If the primary OptAttest governance structures were ever to be captured or forced into unjust complicity, communities retain the ability to curate and utilize alternative issuer trust roots that align with their ethical commitments and need for protection.

This overall approach seeks to curtail predatory use by making legitimate-looking compliance difficult and costly to forge within an ecosystem that prioritizes user agency, robust ethical governance, and principled resistance to becoming an instrument of unwarranted surveillance or oppression.

7.4 Trust Assumptions and the Importance of Ecosystem Governance

While designed to minimize reliance on singular trusted third parties for its core attestation logic and to empower users, the OptAttest architecture, especially when integrated into broader systems like L2 rollups, necessarily makes several assumptions, and understanding them reveals the pivotal role of governance.

- **Cryptographic Assumptions:** The security of OptAttest fundamentally relies on the standard computational hardness assumptions underpinning the chosen cryptographic primitives.
- **ZKP System and Protocol Logic Implementation Integrity:** The correctness of the ZKP system implementations (e.g., SP1 zkVM) and the

application logic for both lightweight and user-generated heavyweight paths is assumed. For L2 rollups, this assumption of correctness extends to the correct execution of attestation-related logic by L2 operators.

- **User Device Security (for IADP, VC, and Key Storage):** For users who opt into receiving attestation-enabled commitments, the security and availability of locally stored IADPs and the private keys for their DIDs/VCs depend on the user's device and wallet software security. This dependency highlights the importance of user education and robust wallet solutions.
- **External Provers (Optional Heavyweight Proof Offloading):** If a user chooses to offload the generation of an optional heavyweight proof, trust assumptions regarding data privacy and proof integrity shift to the external prover service.
- **Verifiable Credential Issuer Integrity, Availability, and Ecosystem Accountability** form a nexus of trustworthiness. Users choosing to engage the lightweight path rely on VC issuers to:
  - Issue VCs based on accurate, ethically sourced, and up-to-date list data.
  - Implement secure and transparent issuance processes.
  - Maintain and correctly operate reliable, accessible revocation mechanisms.
  - Be available to issue or refresh credentials as needed.

  The integrity of any attestation derived from a VC is directly dependent on the trustworthiness of its issuer. However, in OptAttest, this reliance is not blind trust. It is *governed trust*, actively shaped and maintained by the pluralistic mechanisms outlined in Section 6. The strength of the ecosystem relies on this governance layer effectively vetting, monitoring, and holding issuers accountable to ethical and operational standards.
- **DID System Integrity and Governed Revocation Infrastructure:** The system relies on the secure operation of the underlying DID method(s) and resolvers. Furthermore, the ZKP verification of VCs must have access to trustworthy and up-to-date revocation status information. The integrity of this revocation infrastructure itself is ideally also subject to or informed by the ecosystem's governance to ensure its reliability and resistance to manipulation.

Thus, while cryptographic soundness provides the technical foundation, the overall trustworthiness and resilience of the OptAttest ecosystem, particularly concerning the VCs that fuel its attestations, are intertwined with the successful implementation and sustained vitality of its governance.

8 Performance Analysis

While concrete benchmarks require full implementation, we can analyze the contributing factors based on the underlying cryptographic primitives. Standard private transactions that do not engage this optional layer are unaffected by these performance considerations.

8.1 Performance for Optional Lightweight Attestation Path

The performance characteristics of the optional lightweight attestation path depend on the user's choice to opt-in and the division of cryptographic labor. Our model emphasizes user-side pre-computation to ensure per-transaction efficiency, whether integrated into an L1 UTXO system or an L2 rollup.

- **User-Side Workload (Two-Part Process when Opting In):** When a user opts into the lightweight path, their wallet manages two types of cryptographic workloads:
  - **Periodic Background Pre-computation (e.g., Daily):** The user's device (e.g., smartphone, desktop) generates a comprehensive ZKP of full VC validity ($\pi_{\text{VC\_validity}}$) for their relevant VCs (Section 5.2.1). This proof covers native VC signature verification (e.g., for BBS+, EdDSA), expiry checks, simplified revocation status, issuer trust, DID control, and attested claims.
    - **Prover Time (Periodic):** Generating $\pi_{\text{VC\_validity}}$ can be computationally intensive, especially if VCs use complex signatures like BBS+ (which might involve proving logic equivalent to over a million constraints). However, this intensive proof generation is performed infrequently (e.g., once per day or VC validity period) as a background task, potentially leveraging hardware acceleration (GPUs, NPUs) on modern devices to complete within a reasonable timeframe (e.g., minutes) without disrupting foreground activities.
    - **Output:** A locally cached $\pi_{\text{VC\_validity}}$ (e.g., a Groth16 proof).
  - **Per-Transaction 0-Hop ZKP Generation (OptAttest Lightweight Path):** For each transaction where the user opts into attestation, their device generates the main 0-hop OptAttest ZKP. This ZKP primarily:
    - Verifies the relevant pre-computed $\pi_{\text{VC\_validity}}$ (e.g., a constant-cost Groth16 verifier, requiring minimal pairings and constraints, leading to sub-second proving time for this part on consumer hardware).
    - Incorporates the attested claims from the verified $\pi_{\text{VC\_validity}}$ into the current transaction's attestation logic (e.g., for `AttestationClaimsHash_Current`).
    - Proves the correctness of the `AttestationAccumulator` update and IADP commitment.

    The overall proving cost for this per-transaction ZKP is therefore significantly reduced and made consistent, as the complexity of native VC signature verification is handled by the background pre-computation.
  - **Attestation Data Preparation (Per-Transaction):** Constructing inputs for the main 0-hop OptAttest ZKP (including retrieving $\pi_{\text{VC\_validity}}$), inputs for the `AttestationAccumulator` update, and forming/encrypting the IADP. These are generally very fast computations.

  In the L2 rollup model, the user prepares the witness for their main 0-hop OptAttest ZKP (which includes $\pi_{\text{VC\_validity}}$) and submits this ZKP with their L2 transaction. The L2 operator then verifies this user-generated ZKP as part of the batch. For L1, the user also generates this entire main 0-hop OptAttest ZKP.
- **Main ZKP Prover Time (Responsibility Varies for the *overall system ZKP*):** The user always generates their main 0-hop OptAttest ZKP per transaction. The distinction below refers to how this proof is processed by the broader network:

- **L2 Rollup Context (Operator Responsibility for Batching):** L2 network operators take user-submitted transactions (which include the user's main 0-hop OptAttest ZKP if opted-in) and include them in a batch. The operators then generate a single large batch proof (e.g., using SP1) covering the validity of all included L2 transactions and their associated user-generated ZKPs. The cost of verifying each user's efficient main 0-hop OptAttest ZKP within this batch is amortized. The overall L2 batch proving system benefits from the lower and consistent complexity of these user proofs.
- **L1 UTXO Context (User Responsibility for Full Tx Proof):** The user's main 0-hop OptAttest ZKP (which is efficient due to the pre-computation strategy) is submitted directly with the L1 transaction. This combined proof is what L1 nodes verify.

- **Verification Time:**
  - **L2 Rollup Context:** Verification happens on L1 by checking the single batch proof submitted by operators. Checking the L2 batch proof is efficient (e.g., milliseconds for a SNARK wrapper).
  - **L1 UTXO Context:** Verification of the user-generated main 0-hop OptAttest ZKP (which verified the cached $\pi_{\text{VC\_validity}}$) is expected to be very fast (milliseconds).

- **Proof Size / On-Chain Overhead:**
  - **L2 Rollup Context:** The user's main 0-hop OptAttest ZKP is part of the L2 transaction data submitted to the operator. Its size is manageable (e.g., if Groth16, a few hundred bytes; if PLONK/STARK, potentially larger but subject to L2 data policies). There's no additional L1 proof overhead per opted-in transaction beyond the L2 batch proof.
  - **L1 UTXO Context:** The user's main 0-hop OptAttest ZKP would add a constant size overhead (kilobytes, depending on the scheme) to the transaction on L1.

By shifting the intensive computation of verifying native VC signatures (like BBS+, which could involve over a million constraints) to an infrequent, user-side background pre-computation task, the per-transaction 0-hop OptAttest ZKP becomes highly efficient. Its primary task concerning VC validity is to verify the pre-computed $\pi_{\text{VC\_validity}}$ (e.g., a fixed low-cost Groth16 verifier), yielding fast and consistent performance for the user when opting into attestations.

In summary, the optional lightweight path, through user-managed pre-computation, allows for efficient per-transaction ZKP generation for 0-hop attestations. For L2 users, the main 0-hop OptAttest ZKP is generated by the user and then verified by operators. For L1 users, they also generate this efficient main 0-hop OptAttest ZKP.

8.2 Heavyweight Path Performance for User-Generated Historical Attestation Verification

The performance characteristics of the heavyweight path—which is invoked only by a user holding an attestation-enabled private commitment (note/UTXO) who needs to verify its historical attestations—differ significantly from the lightweight path. This process is computationally intensive for the user.

- **User Prover Time** is the main bottleneck for the user employing the heavyweight path. The cost is dominated by the user-side recursive proof generation (Section 5.4.2, using locally stored IADPs) and depends on several factors:

- **Recursive Step Cost:** The cost added by the user's software verifying the previous step's proof within the current step's ZKP.
  - Using *native zkVM recursion* (e.g., the user's software using SP1 to verify a previous SP1 proof, Section 2.2), the efficiency depends heavily on the specific implementation (e.g., Plonky3 techniques) and requires benchmarking for typical user hardware.
  - Using *folding schemes* (e.g., HyperNova, Section 2.5) offers potentially lower cryptographic overhead per step *in theory* for the user's prover. However, the practical cost and complexity of bridging these schemes with general zkVM outputs within the user's proving environment (the integration challenge discussed in Section 10) currently make their net benefit unclear for our specific N=2 depth.
- **Per-Step Application Logic Cost:** The cost of proving the historical 0-hop multi-list attestation *VC proof verification logic* (using IADP witness data retrieved from local storage, including verifying historical VC claims, issuer trust, and revocation status for all queried lists) and accumulator consistency checks within each recursive step performed by the user's software. This per-step application logic cost depends on the complexity of these VC-related checks, potentially increasing with the number of lists attested against per hop in the commitment's history.
- **Merge Complexity (Scaling with Attestation-Enabled Inputs):** As detailed in Section 5.1.3, if the attestation-enabled commitment being proven was created from a merge, verifying its history requires processing only those input branches that were themselves attestation-enabled. If M_{opt-in} inputs out of M total inputs were attestation-enabled, the user's prover work (both recursive verification and application logic cost) scales roughly with M_{opt-in}. Such scaling significantly impacts proving time for commitments with complex attestation histories originating from merges involving many opted-in predecessors.
- **Recursion Depth:** Fixed at N=2 (Hop 0 and Hop 1) for the attestation history in our design, bounding the number of sequential recursive steps but not the parallel complexity from merges of attestation-enabled inputs.

Due to these factors, particularly the recursive step cost and merge complexity involving opted-in branches, real-time on-device generation of the heavyweight proof by the user is likely infeasible with current technology. Such infeasibility necessitates strategies like background processing or user-controlled offloading (Section 5.4.3). Standard private transactions (not opted-in) incur none of this cost.

- **Verification Time (of the User-Generated Proof):** Despite the prover complexity for the user, verification of the final heavyweight ZKP (whether a native recursive proof, potentially wrapped, or a folding-scheme-derived proof) by a third party (e.g., a VASP receiving the proof from the user) is expected to remain fast (typically milliseconds range), similar to standard ZKP verification.
- **Proof Size (of the User-Generated Proof):** Using state-of-the-art recursive constructions (like recursive STARKs generated via SP1, or folding schemes, potentially combined with a final SNARK wrapper), the final proof size (for the attestation report generated by the user) is expected to remain relatively small and constant (kilobytes), suitable for efficient transmission and verification (e.g., peer-to-peer or submitted to an L2 contract), regardless of computation complexity or merge width (M_{opt-in}).

8.3 Communication & Storage Costs

Beyond ZKP computational costs, the following overheads must be considered:

- **VC Issuance/Management Communication:** While not part of the transaction flow itself, users need to interact with DID infrastructure and VC issuers offline to obtain and manage their credentials. Such interaction involves standard internet communication. Revocation checks during VC proof generation might require fetching updated revocation lists or proofs of non-revocation from external sources or on-chain registries, incurring data retrieval costs.
- **IADP Transmission:** The encrypted Intermediate Attestation Data Packet (IADP) blob (containing VC-related data; structure defined in Appendix D) must be transmitted privately from sender to recipient if the lightweight path is engaged. Its size depends on the included data per hop (DID references, VC commitments/summaries, issuer info, revocation context, verified claims, and references) and transaction complexity. For Hop 0 & 1 verification, performing such verification typically involves data for two prior hops. While an added overhead compared to standard transactions, this overhead from IADP transmission occurs off-chain and the size is expected to be manageable (likely kilobytes, though scaling with the number of lists attested against per hop and the richness of VC data).
- **On-Device Storage:** The recipient's wallet locally stores decrypted IADPs (Appendix D) for their attestation-enabled commitments. The total storage requirement scales with the number of unspent attestation-enabled commitments and the data per IADP (which includes summaries of multi-list VC attestations). Effective local database management, including indexing and potential pruning strategies for outdated attestations or lists, will be crucial for managing storage growth.

9 Related Work

Our proposed architecture for optional, hybrid N-hop attestations builds upon and differentiates itself from several areas:

- **Existing Compliance Features in Privacy-Preserving Systems:** Current approaches in privacy-focused systems (both L1 coins and L2 protocols) often involve:
    - **Optional Full Disclosure via Viewing Keys:** Mechanisms like Zcash Full Viewing Keys [Hopwood et al., 2024] and Railgun viewing keys [RAILGUN Project, e] allow users to grant broad, decrypt-all access to their transaction history (senders, receivers, amounts, memos). As discussed in Section 1.1, while intended for auditability, such comprehensive keys can inadvertently set a precedent where surrendering wholesale historical access becomes the expected norm for 'compliance.' OptAttest offers a distinct model: even when users opt-in to attestations, the detailed historical data (IADPs) containing VC proof summaries and context remains encrypted under user control and is not designed for wholesale access via a single master key. Instead, users generate bounded, recursive ZKPs from their IADPs to prove specific historical claims, thereby providing targeted verifiability without compelling complete data handover.

- **Optional Transaction-Specific Disclosure:** Features like Zcash payment disclosure allow revealing details of individual transactions.
- **Optional 0-Hop Attestations & Attestation Propagation:** Systems like Railgun, operating within account-based environments like Ethereum, offer a 'Private Proofs of Innocence' (PPOI) mechanism [Railgun Project DAO, 2023, RAILGUN Project, a]. PPOI enables users, when shielding funds, to generate ZK proofs attesting that these funds do not originate from addresses on lists provided by multiple external data sources (e.g., Chainalysis, Elliptic) [RAILGUN Project, b]. This attestation of non-interaction at the shielding event (0-hop relative to entry) is then cryptographically carried forward with the funds through subsequent internal private transactions, leveraging recursive SNARKs [RAILGUN Project, d]. This mechanism of carrying the attestation forward offers a form of historical provenance tracing back to the initial compliance check.
- **Obfuscation without Verifiable Checks:** Protocols like Monero rely on strong obfuscation techniques (ring signatures, RingCT, stealth addresses) for their UTXOs but lack mechanisms for users to generate verifiable attestations about historical provenance [Noether et al., 2016].

Our work is distinct in offering users the option to generate verifiable, bounded N-hop (specifically Hop 0 & 1) attestation history concerning list interactions for their notes or UTXOs. The OptAttest approach involves new, independent attestations based on user-provided Verifiable Credentials at each opted-in hop (whose complex validation is handled efficiently per-transaction via user-side pre-computed and cached VC validity proofs), rather than solely propagating an initial source-of-funds check. The nature of the optional disclosure (multi-hop attestation history based on VC claims against user-chosen lists per hop vs. raw transaction details or a propagated 0-hop status) is also different.

As the comparison in Table 1 illustrates, while Railgun's PPOI offers a robust solution for attesting to the source of funds at shielding and propagating this status, our proposal differs significantly in its mechanism, scope, and philosophical approach to disclosure. Key distinctions include the use of user-centric DIDs/VCs for per-hop attestations against user-defined list criteria (with per-transaction efficiency achieved via user-pre-computed cached VC validity proofs), the bounded Hop 0 & Hop 1 historical verification of these distinct attestations, and the explicit avoidance of a global viewing key for the attestation data itself. Railgun's PPOI has made notable strides in on-chain compliance for its users. However, the broader impact of such systems on the universal exchange listing of all privacy-centric assets or the complete alleviation of off-ramp restrictions by VASPs is still an evolving area. The regulatory landscape for privacy-enhancing technologies remains complex, suggesting that a diversity of approaches to demonstrating accountability may be beneficial. Our system aims to provide a granular, user-controlled attestation layer, which, even if compelled, would offer a more constrained disclosure (limited to specific historical data in IADPs for opted-in hops) compared to the wholesale transparency of a full viewing key.

- **Privacy-Preserving Data Retrieval and Attestation Mechanisms:**
  - **Decentralized Identifiers (DIDs) and Verifiable Credentials (VCs):** Our architecture leverages DIDs [Sporny et al., 2022b] and VCs [Sporny et al., 2022a] for the 0-hop multi-list attestation component (Section 2.6).

Users obtain native VCs from trusted issuers offline. Their wallets then periodically generate and cache ZKPs of these VCs' validity ($\pi_{\text{VC\_validity}}$) in the background. During an optional lightweight transaction path, a new ZKP efficiently verifies the relevant $\pi_{\text{VC\_validity}}$ to prove claims. This approach benefits from standardization and an emerging ecosystem [Polygon Labs, 2022, Hyperledger Foundation, 2025] while addressing the challenge of integrating efficient VC proof verification into per-transaction ZKPs by shifting the main validation complexity to the user's background task.

- **Private Oracles & PIR/OMR:** While not the mechanism for our proposed 0-hop check, Oblivious Message Retrieval (OMR) protocols like PerfOMR [Liu et al., 2024] (Section 2.4) or general Private Information Retrieval (PIR) [Goldberg, 2007] are alternative techniques for private external data access.

- **Zero-Knowledge Proof Techniques:** Our architecture specifically combines:
  - **General-Purpose zkVMs:** We utilize the flexibility of zkVMs like SP1 (Section 2.2). In L2 contexts, operators use them to batch-verify user-generated 0-hop ZKPs (which themselves verify cached VC validity proofs) for opted-in lightweight path transactions. For L1 contexts or for user-generated heavyweight proofs, the zkVM and its native recursion capabilities provide the core proving engine. User wallets also employ zkVM capabilities or ZKP libraries for the background generation of cached VC validity proofs.
  - **Folding Schemes:** Folding schemes like HyperNova (Section 2.5) are a key related advancement offering potential recursive optimization for the user-generated heavyweight proofs, although we would expect integration challenges (Section 10) in bridging them with general-purpose zkVM outputs.

10 Future Work: Cultivating a Resilient and Empowering Attestation Ecosystem

The realization of OptAttest's full potential involves ongoing research and development across technical, social, and governance dimensions. Key areas include:

- **Benchmarking User-Side Heavyweight Proof Generation:** Empirically evaluating the performance of native zkVM recursion on typical user hardware for generating the N=2 heavyweight proof.
- **Benchmarking User-Side Daily VC Validity Proof Generation:** Empirically evaluating the performance of generating the "cached VC validity proof" ($\pi_{\text{VC\_validity}}$) on typical user hardware for various native VC types.
- **L2 Integration & Operator Performance:** For L2 rollup integrations, benchmarking the marginal cost added to batch proof generation by L2 operators when verifying opted-in lightweight attestation updates.
- **Full System Implementation & Integration:** Building an end-to-end prototype integrating all components, with a strong focus on user agency and intuitive control over attestation choices.

Table 1: Comparison of OptAttest with Railgun PPOI

Feature / Aspect	Railgun	OptAttest
Primary Goal of Attestation Feature	Provide cryptographic assurance that shielded funds are not from known "bad actor" lists; user compliance tool. [RAILGUN Project, a]	Enable users to *optionally* generate verifiable, bounded historical attestations about interactions/non-interactions with *user-specified lists* for their assets.
0-Hop Attestation Primitive	ZK proof against datasets from List Providers (e.g., Chainalysis, Elliptic) after an initial shield and standby period. [RAILGUN Project, a]	ZK proof of Verifiable Credentials (VCs) obtained by the user from various issuers, attesting to claims about specified lists. Per-transaction ZKP efficiently verifies a user-pre-computed cached proof of VC validity.
Multi-List Capability	Yes, supports multiple List Providers; wallets/users can select which lists/providers to use for PPOI. [RAILGUN Project, b]	Yes, users select which VCs (potentially referencing different lists from different issuers) to present for their 0-hop attestation via their cached VC validity proofs.
Historical Attestation / Provenance	An initial PPOI status (attesting to non-interaction with specified lists at the time of shielding) is "carried forward" using recursive SNARKs, linking subsequent internal transfers back to this original attestation. Carrying the status forward in this manner proves the flow from a compliant shield but does not involve new, independent attestations at each internal hop. [RAILGUN Project, d]	Explicit, user-generated recursive ZKP verifying Hop 0 & Hop 1 attestations (based on historical VC proofs, utilizing the cached validity proof model for each hop's 0-hop component) for an opted-in commitment.
Depth of Verifiable History	Continuous "innocence" status from a valid shield/PPOI onwards within Railgun.	Bounded: specifically Hop 0 & Hop 1 of the opted-in lineage.
Data Storage for Attestation Details	PPOI proof data (blinded) potentially stored/synced via PPOI Nodes (IPFS-like). [RAILGUN Project, c]	Intermediate Attestation Data Packets (IADPs) containing VC proof summaries (linked to cached validity proofs), list details, etc., are encrypted and stored on the user's device.
Viewing Key for Transaction Data	Yes, supports Viewing Keys to decrypt and view full transaction history and balances of a 0zk address. [RAILGUN Project, e]	Does not propose viewing keys for the *attestation history data itself*. The underlying privacy coin/L2 *could* have its own viewing key for base transactions, but if it did, it would simply be perpetuating the "mandatory volunteering" issues identified earlier for Zcash and Railgun in Section 1.1.
User Control over Lists/Content	Users/wallets select from available List Providers or custom PPOI lists (datasets of addresses). List content is provider-defined.	Users select which VCs to obtain and present. The *content and claims* (including which list is referenced) are defined by the VC issuer and chosen by the user.
Disclosure Philosophy	Offers viewing keys for full transparency and PPOI for "innocence" assurance. Aims for "compliant privacy."	Aims to shift expectation from full viewing key disclosure to user-controlled, bounded, specific historical attestations. Seeks minimal, context-specific disclosure for attestation data.
Core Technology for Attestation	Groth16 zk-SNARKs for PPOI; direct checks against list data.	General-purpose zkVMs (e.g., SP1) for ZKP of VCs (0-hop, via verification of cached user proofs) and recursive proofs (heavyweight path); DIDs/VCs as primary primitive. User-side background ZKP generation for caching VC validity.
Extensibility of Attestation Types	Primarily focused on non-association with "bad actor" lists for PPOI.	Highly extensible via VCs; could attest to a wide range of claims (e.g., ESG compliance, KYC status, etc.) depending on available VC issuers/types.
Permissioning of Attestation	PPOI check is largely automatic for shielded funds in wallets that integrate it.	Purely optional, per-transaction decision by the user to engage the lightweight attestation path and generate/store IADPs.
Underlying Asset Privacy	Provides a strong privacy system for shielded transactions on EVM chains.	Designed as an optional layer on top of existing or new privacy-preserving digital asset systems (L1s or L2s). Relies on base layer for asset privacy.

- **Efficient ZKP Circuits for User-Side VC Validity Pre-Computation:** Developing and optimizing ZKP circuits or zkVM routines for the user's periodic generation of $\pi_{\text{VC\_validity}}$.
- **Handling Merges with Mixed Attestation Statuses:** Developing and rigorously testing the logic for handling merges, ensuring the system correctly respects and represents users' choices to opt-in or opt-out for different inputs.
- **Analysis and Mitigation of Merge Blowup Costs for Attestation Histories:** Further analysis of the practical impact of the "merge-hash binding" strategy

on user-side heavyweight proof costs.

- Wallet UI must surface clear errors and guidance when a user attempts a merge that would exceed the MaxFanIn = 2 attestation-enabled-input limit.
- **On-Device Attestation Data (IADP), VC, and Cached Proof Management:** Developing robust, efficient, and secure mechanisms for managing user-controlled data stores.
- **Formal Security Modeling and Proofs:** Formal modeling for the optional attestation layer, including analysis of privacy leakage vectors and the robustness of user-centric controls.
- **Folding Scheme Integration (for User-Side Heavyweight and Daily Proofs):** Investigating bridging zkVM outputs with folding schemes for potential performance gains in user-controlled proof generation.
- **Design and Optimization of Attestation Accumulator for Optionality:** Refining the `AttestationAccumulator`'s cryptographic design to efficiently and clearly represent user choices regarding attestation.
- **User-Side Prover Performance and Offloading Models:** Continued research into optimizing ZKP prover performance on user devices and investigating secure, user-controlled offloading models.
- **DID/VC Ecosystem Infrastructure, Trust Bootstrapping, and Governance Evolution:**
  - Designing robust mechanisms for discovering trusted VC issuers, managing issuer public keys, and accessing revocation status, all within the framework of the pluralistic governance model envisioned in Section 6.
  - **Ongoing Development, Empirical Stress-Testing, and Community Ratification of the Plural Issuer Governance Model:** Such development and ratification include defining clear processes for DAO evolution, capture resistance, ethical deliberation, and accountability to the user community. A full governance charter would be specified in an independent, community-driven RFC process.
  - **Developing and Piloting "Relational Impact" and "Vitality" Metrics for Issuers:** Further research into actionable methodologies for assessing VC issuers beyond technical compliance, focusing on their contribution to a healthy ecosystem, as discussed in §6.2.
  - **Exploring integrations with novel key management and DID generation platforms,** such as Human Network [Holonym Foundation, 2024], which focuses on deriving cryptographic keys from human attributes via privacy-preserving methods. Such integrations could enhance the usability and accessibility of the DIDs required for users to participate in the OptAttest ecosystem.
- **User Experience, Accessibility, and Adoption Dynamics:** Designing intuitive interfaces that clearly communicate user choices and control over optional attestations. Studying incentives for users to obtain VCs from governed issuers and opt-in, particularly for use cases that enhance community self-governance or provide alternatives to coercive compliance.
- **Recipient-Side Policy Engines and Legibility for Diverse Communities:** Developing standardized yet flexible ways for recipients (individuals, DAOs, or other entities) to define and apply policies based on verifiable attestation reports, supporting pluralistic interpretation and contextual appropriateness.

- **Exploring Legal and Institutional Frameworks for OptAttest Governance:** Investigating how the Plural Issuer Governance DAO/Council and the broader OptAttest ecosystem could achieve broader social and potentially legal recognition as a legitimate, self-regulating steward of attestation infrastructure, particularly in environments where traditional regulatory bodies are failing or are themselves sources of mistrust. Exploring such frameworks includes exploring pathways for asserting the ecosystem's autonomy and its right to define its own standards for trustworthy interaction.

11 Conclusion: Towards User-Sovereign, Ethically Governed Attestation

The escalating tension between financial privacy and demands for auditability hinders the adoption and flourishing of privacy-preserving digital asset systems. Our proposed OptAttest architecture offers a pathway forward by enabling users to *choose* to generate verifiable Hop 0 & Hop 1 attestation history for their notes or UTXOs, relative to multiple, user-specified external lists defined via Verifiable Credentials. This foundational principle of opt-in engagement ensures that default user privacy and standard transaction performance are maintained for users not engaging the attestation layer. For those who proactively opt-in, OptAttest would facilitate user-controlled, targeted data sharing for diverse policy applications.

OptAttest provides an **optional lightweight path** for everyday transactions involving private commitments, featuring user-initiated 0-hop attestations grounded in VC proofs. The correctness of these opted-in updates is efficiently verified. The optional lightweight path is complemented by a less frequent, user-initiated **heavyweight path** for attestation-enabled commitments, employing user-generated recursive ZKPs to verify historical attestation chains using locally stored IADPs. User-controlled presentation of VCs, proven in zero-knowledge, underpins the system's ability to support diverse, context-specific claims.

Significant engineering and research challenges remain, particularly in optimizing user-side proof generation and ensuring the robustness of user-managed data. Nonetheless, by enabling users to optionally generate precise, verifiable attestations about limited historical interactions—without resorting to comprehensive viewing keys—this architecture endeavors to shift the paradigm towards greater user sovereignty over their informational footprint.

The aspiration is a digital ecosystem where robust user privacy and accessible interaction coexist with transparent, pluralistic, and user-driven accountability. Achieving this aspiration will depend not only on the cryptographic architecture's soundness and usability, but also on the organic development and sustained vitality of resilient, decentralized, and ethical governance mechanisms for the attestation ecosystem itself. OptAttest endeavors to equip individuals and communities with the means to forge healthy relationships on their own terms, promoting a more just and vital digital future, at a time when traditional authorities are intentionally undermining the rights they are supposed to protect.

A Attestation Accumulator Specification (for Opted-In Commitments)

This appendix provides a more detailed cryptographic specification for the `AttestationAccumulator`. This component exists only for notes or UTXOs for which a user has explicitly opted into generating attestation history via the lightweight path (Section 5.2). Standard private commitments do not have an

associated `AttestationAccumulator`. When present, it outlines the structure and update logic designed to be compatible with Zero-Knowledge Proof (ZKP) systems used for verification (either user-generated lightweight ZKPs on L1 or L2 operator batch proofs).

Domain–separation convention. For any literal label $\ell \in \{\texttt{PAIR}, \texttt{ACC}, \texttt{HASH\_INPUTS}, \texttt{INPUT}\}$ we write

$$\mathsf{DS}[\ell] := \text{BLAKE3}(\texttt{"OptAttest|"} \,\|\, \ell)\ [0..15]$$

—the first 128 bits of BLAKE3 of the concatenation—so that *all hash invocations in this appendix are deterministically domain separated*. The tag is always prepended to the byte-encoded message being hashed.

Accumulator commitment. Define the pair-compression hash

$$h_{\text{pair}} := \text{Hash}\big(\mathsf{DS}[\texttt{PAIR}] \,\|\mathsf{AttestationClaimsHash}_0 \,\|\mathsf{AttestationClaimsHash}_1\big) \in \{0,1\}^{\kappa},$$

where $\mathsf{AttestationClaimsHash}_1 := \mathbf{0}^{\kappa}$ if no Hop-1 data exist (e.g. fresh note or width–1 merge). The **AttestationAccumulator commitment** is a two-generator Pedersen commitment using generators $G_0, G_1 \in \mathbb{G}$:

$$Acc := \text{MapToField}(h_{\text{pair}})\, G_0 \;+\; bf\, G_1 \;\in\; \mathbb{G}.$$

The commitment Acc can also be expressed as $\mathsf{Acc} = \text{Commit}(\text{MapToField}(h_{\text{pair}}), bf)$. Only a single group element leaks, and the blinding factor bf hides the presence or absence of Hop 1 (as encoded in h_{pair}), eliminating the "hop-order width" side-channel.

Collision-resistant input hashing. For any merge with M ordered inputs we set

$$\mathsf{Hash\_Inputs} := \text{Hash}\Big(\mathsf{DS}[\texttt{HASH\_INPUTS}] \,\|\, \text{LE64}(M) \,\|\, \underbrace{\big\langle \mathsf{DS}[\texttt{INPUT}] \,\|\, \text{LE64}(i) \,\|\mathsf{InputState}_i \big\rangle_{i=1}^{M}}_{\text{fixed-len encoding}}\Big).$$

Explicit length prefixes and the per-input index $\text{LE64}(i)$ make the map injective under the assumption that Hash is collision resistant.

A.1 Purpose

When created as part of the optional lightweight path, the `AttestationAccumulator` serves as a compact, fixed-size cryptographic commitment attached to an attestation-enabled private commitment. It summarizes the attestation status (specifically, the 0-hop check results, identified by a nonce) of the transaction that created this commitment (Hop 0) and the prior transaction in its attestation-enabled lineage (Hop 1). This summarization by the `AttestationAccumulator` allows the verification logic (lightweight ZKP or L2 batch proof) to efficiently prove the correct propagation of attestation state for opted-in commitments without needing to process the full historical data on every transaction.

A.2 Underlying Primitives

The construction relies on standard cryptographic primitives assumed to be efficiently implementable within the chosen ZKP system:

- **Collision-Resistant Hash Function `Hash()`:** A function mapping arbitrary

length inputs to fixed-length outputs, computationally infeasible to find distinct inputs mapping to the same output. ZKP-friendly hashes like Poseidon [Grassi et al., 2021] are preferred.

- **Cryptographic Commitment Scheme `Commit()`:** For the `AttestationAccumulator`, we use an additive Pedersen-style commitment on an elliptic-curve group $\mathbb{G}$ of prime order p. Publish two *independently sampled* generators $G_0, G_1 \in \mathbb{G}$ in the system's Common Reference String (CRS). For a message $m \in \mathbb{Z}_p$ (which will be the mapped h_{pair}) and a blinding factor $bf \in \mathbb{Z}_p$, define

$$\text{Commit}(m, bf) = mG_0 \; + \; bfG_1 \; \in \; \mathbb{G}.$$

  This additive form (equivalent to $G_0^m G_1^{bf}$ in multiplicative notation) is chosen to avoid ambiguity in generator usage. The scheme is binding (hard to find $m \neq m'$ or $bf \neq bf'$ such that $\text{Commit}(m, bf) = \text{Commit}(m', bf')$ if G_0, G_1 are independent) and computationally hiding (given $\text{Commit}(m, bf)$, it's hard to find m) under the discrete logarithm hardness assumption in $\mathbb{G}$.

- **Mapping Function `MapToField()`:** An injective function mapping hash outputs (typically bitstrings) to elements of the field $\mathbb{Z}_p$ used by the commitment scheme's exponents.

- **Placeholder Constant `OptedOutPlaceholder`:** A predefined, fixed constant value used to represent an input to a merge operation that was a standard private commitment (i.e., did not have an `AttestationAccumulator`). This constant must be chosen carefully to avoid collisions with actual hash outputs or commitment representations.

A.3 State Structure and Commitment for Attestation-Enabled Commitments

The `AttestationAccumulator`, when present, commits to a representation of the attestation checks (based on verified Verifiable Credential claims) from the last two relevant hops in the attestation-enabled lineage. The exact structure depends on whether the attestation-enabled commitment was created by a simple transfer (from another attestation-enabled commitment) or a merge (potentially involving mixed inputs).

A.3.1 State for Simple Transfers (Creating an Attestation-Enabled Output)

For an attestation-enabled output commitment created by a non-merge transaction (sender Alice, who opted-in, using an attestation-enabled input commitment from Charlie), the committed state captures the verified claims for Alice's 0-hop attestation event and Charlie's prior 0-hop attestation event (which becomes Hop 1 for Alice's output).

- Define `AttestationClaimsHash_i = Hash(Nonce_i | CommitmentToSpecifiedLists_i | CommitmentToVerifiedClaims_i | Timestamp_i)` for a given hop's attestation event i. Here, `Nonce_i` is a unique random nonce generated by the sender of hop i for their 0-hop attestation event and stored in the IADP for that event. `CommitmentToSpecifiedLists_i` is a commitment (e.g., hash) to the set of lists for which attestations were proven. `CommitmentToVerifiedClaims_i` is a commitment (e.g., hash) to the set of claims (e.g., {Claim(NotOnList1, True), Claim(NotOnList2, False),...}) that were successfully verified from the user's VCs for those lists at hop i, as established by the ZKP in the lightweight path (see Section 5.2.2).

- The accumulator state is the tuple `(AttestationClaimsHash_0, AttestationClaimsHash_1)`.
- The two attestation claim hashes are combined into a single pair hash (where $\mathsf{AttestationClaimsHash}_1$ is a fixed zero-hash if no Hop-1 data exists):

$$h_{\text{pair}} = \text{Hash}\big(\mathsf{DS}[\texttt{PAIR}] \| \mathsf{AttestationClaimsHash}_0 \| \mathsf{AttestationClaimsHash}_1\big).$$

- The accumulator commitment is:

$$\text{Acc} = \text{Commit}(\text{MapToField}(h_{\text{pair}}), bf).$$

A.3.2 State for Merged Transfers (Creating an Attestation-Enabled Output)

For an attestation-enabled output commitment created by merging M inputs (sender Alice, who opted-in; inputs $Input_1, ..., Input_M$, which may be a mix of attestation-enabled and standard private commitments), we use the "merge-hash binding" strategy incorporating the placeholder for opted-out inputs.

- Define the current attestation-claims hash (uses the 0-hop VC claims for Alice's current attestation event):

$$\mathsf{AttestationClaimsHash}_{\text{Current}} = \text{Hash}\big(\text{Nonce}_{\text{Alice, Current}} \parallel \mathsf{CommitToLists}_{\text{Current}} \parallel \mathsf{CommitToClaims}_{\text{Current}} \parallel \text{Timestamp}_{\text{Current}}\big).$$

  Here, `Nonce_Alice_Current` is a unique random nonce generated by Alice for her current 0-hop attestation event and stored in the IADP for this event.

- Define `InputState_i` for each input $i \in [1, M]$:
  - If $Input_i$ was attestation-enabled (has `Acc_in_i` and associated historical `AttestationClaimsHash_0_i`): `InputState_i = Acc_in_i | AttestationClaimsHash_0_i` (where '|' denotes concatenation with appropriate encoding/domain separation, and `AttestationClaimsHash_0_i` was computed using a nonce from input i's creating transaction).
  - If $Input_i$ was a standard private commitment (no attestation data): `InputState_i = OptedOutPlaceholder`.

7. For each $i \in [1, M]$ set

$$\text{InputState}_i = \begin{cases} \mathsf{Acc}_{\text{in},i} \parallel \mathsf{AttestationClaimsHash}_{0,i} & \text{if input } i \text{ is attestation-enabled,} \\ \texttt{OPT\_OUT} & \text{otherwise.} \end{cases}$$

8. Compute the collision-resistant aggregate

$$\mathsf{Hash\_Inputs} := \text{Hash}\Big(\mathsf{DS}[\texttt{HASH\_INPUTS}] \parallel \text{LE64}(M) \parallel \big\langle \mathsf{DS}[\texttt{INPUT}] \parallel \text{LE64}(i) \parallel \text{InputState}_i \big\rangle_{i=1}^{M}\Big).$$

   The explicit index i and length prefix bind both the *order* and the *multiplicity* of inputs; forging a second preimage thus requires breaking the collision resistance of Hash. (See Theorem **??** for the formal statement.)

9. Set $\mathsf{AttestationClaimsHash}_{0,\text{out}} := \mathsf{AttestationClaimsHash}_{\text{Current}}$ and $\mathsf{AttestationClaimsHash}_{1,\text{out}} := \mathsf{Hash\_Inputs}$.
10. Output accumulator:

$$\mathsf{Acc}_{\text{out}} := \text{Commit}(\text{MapToField}(\text{Hash}(\mathsf{DS}[\texttt{PAIR}] \| \mathsf{AttestationClaimsHash}_{0,\text{out}} \| \mathsf{AttestationClaimsHash}_{1,\text{out}})), bf_{\text{new}})$$

- The accumulator state is the tuple `(AttestationClaimsHash_Current, Hash_Inputs)`.
- Let $\mathsf{AttestationClaimsHash}_{0,\text{out}} = \mathsf{AttestationClaimsHash}_{\text{Current}}$ and $\mathsf{AttestationClaimsHash}_{1,\text{out}} = \mathsf{Hash\_Inputs}$. The combined pair hash is:

  $$h_{\text{pair}} = \text{Hash}(\mathsf{DS}[\texttt{PAIR}] \| \mathsf{AttestationClaimsHash}_{0,\text{out}} \| \mathsf{AttestationClaimsHash}_{1,\text{out}}).$$

- The accumulator commitment is:

  $$\text{Acc} = \text{Commit}(\text{MapToField}(h_{\text{pair}}), bf).$$

In both cases, the accumulator `Acc` maintains a fixed size. The merge strategy ensures this fixed accumulator size by compressing the state (or opted-out status) of M inputs into a single hash (`Hash_Inputs`), deferring the complexity of verifying the attestation-enabled branches to the user-generated heavyweight proof.

A.4 Accumulator Update Logic (Verification for Opted-In Lightweight Path)

When a user opts into the lightweight path, the verification logic (within the L1 ZKP or L2 operator proof) must prove the correct computation of the output accumulator(s) (`Acc_out`) for the new attestation-enabled output commitment(s) based on the input(s) and the current 0-hop *verified claims derived from VCs* (associated with the current 0-hop attestation event's nonce).

A.4.1 Handling Splits (1 Attestation-Enabled Input → M Attestation-Enabled Outputs)

1. The verification logic takes `Acc_in` (from the attestation-enabled input) and proves knowledge of its opening, yielding `AttestationClaimsHash_0_in` and `AttestationClaimsHash_1_in`.
2. Compute the current hop's attestation-claims hash (based on the 0-hop verified VC claims for the current attestation event, identified by `Nonce_Current`):

$$\mathsf{AttestationClaimsHash}_{\text{Current}} = \text{Hash}(\text{Nonce}_{\text{Current}} \parallel \mathsf{CommitToLists}_{\text{Current}} \parallel \mathsf{CommitToClaims}_{\text{Current}} \parallel \text{Timestamp}_{\text{Current}}).$$

   (Here $\mathsf{CommitToClaims}_{\text{Current}}$ is itself a cryptographic commitment—typically a hash—of the set of claims derived from the user's VCs that were successfully proven valid for the current transaction; see Section 5.2.2. The `Nonce_Current` is generated by the sender for this specific 0-hop attestation event and stored in the IADP for this event.)

3. Define the output state components that will form the new $h_{\text{pair,out}}$: `AttestationClaimsHash_0_out` = `AttestationClaimsHash_Current`, `AttestationClaimsHash_1_out` = `AttestationClaimsHash_0_in`. (The oldest attestation claims hash from the input, `AttestationClaimsHash_1_in`, is effectively dropped from the hop 0 & hop 1 window being committed to in the new output accumulator).

4. Compute the combined pair hash for the output:

$$h_{\text{pair,out}} = \text{Hash}\big(\mathsf{DS}[\texttt{PAIR}] \| \mathsf{AttestationClaimsHash}_{0,\text{out}} \| \mathsf{AttestationClaimsHash}_{1,\text{out}}\big).$$

5. Compute the output accumulator:

$$\texttt{Acc\_out} = \text{Commit}(\text{MapToField}(h_{\text{pair,out}}), bf_{\text{new}}),$$

using a fresh random blinding factor bf_{new}.

6. The verification asserts that `Acc_out` is correctly computed and assigned to all M output commitments, which are now also attestation-enabled.

A.4.2 Handling Merges (M Mixed Inputs → 1 Attestation-Enabled Output)

1. The verification logic takes inputs representing all M input commitments. For each input i that is attestation-enabled, it takes `Acc_in_i` and proves knowledge of its opening, yielding `AttestationClaimsHash_0_i`, `AttestationClaimsHash_1_i`. For inputs that are standard private commitments, this step is skipped (they contribute the placeholder).

2. Compute the current hop's attestation claims hash based on the 0-hop verified VC claims (for the current attestation event, identified by `Nonce_Current`):

```
AttestationClaimsHash_Current = Hash(Nonce_Current | CommitmentToSpecifiedLists_Current
| CommitmentToVerifiedClaims_Current | Timestamp_Current).
```

(The `Nonce_Current` is generated by the sender for this specific 0-hop attestation event and stored in the IADP for this event.)

3. Set

$\mathsf{AttestationClaimsHash}_{0,\text{out}} := \mathsf{AttestationClaimsHash}_{\text{Current}}$
and
$\mathsf{AttestationClaimsHash}_{1,\text{out}} := \mathsf{Hash\_Inputs}$.

4. Compute the combined pair hash for the output:

$$h_{\text{pair,out}} = \text{Hash}\big(\mathsf{DS}[\texttt{PAIR}] \| \mathsf{AttestationClaimsHash}_{0,\text{out}} \| \mathsf{AttestationClaimsHash}_{1,\text{out}}\big).$$

5. Output accumulator:

$$\mathsf{Acc}_{\text{out}} = \text{Commit}(\text{MapToField}(h_{\text{pair,out}}), bf_{\text{new}}).$$

6. The verification asserts that `Acc_out` is correctly computed based on all inputs (respecting their attestation status) and assigned to the single output commitment, which is now attestation-enabled.

Note that the logic described here defines the *state transitions* that must be proven correct for opted-in transactions. The verification logic (lightweight ZKP or L2 operator proof) validates the immediate update, while the user-generated heavyweight ZKP (Appendix B.2) verifies the correctness of the entire historical sequence for attestation-enabled branches implied by these updates, using the stored IADPs. The merge strategy's cost during heavyweight proof generation depends only on the number of attestation-enabled input branches encapsulated in `Hash_Inputs`.

A.5 ZKP Implementation Notes (for Verification Logic)

- The ZKP circuit (or L2 logic proven by operators) must implement the chosen hash function (`Hash`) and commitment scheme (`Commit`); ready-to-use circuits for MiMC-Pedersen and Reinforced Concrete are given in [Albrecht et al., 2016, Grassi et al., 2022].
- It must correctly handle the conditional logic for merges based on whether inputs are attestation-enabled or contribute the `OptedOutPlaceholder`.
- Proving knowledge of the opening of input Pedersen commitments (for attestation-enabled inputs) and the correctness of the output commitment involves reasoning about the committed values and blinding factors according to the scheme's algebraic properties.
- The mapping `MapToField()` must be handled consistently.
- The verification relies on private inputs provided by the transaction creator (or available to the L2 operator). These include the opening information for attestation-enabled input accumulators; the unique random nonce for the current 0-hop attestation event, the Verifiable Credentials (VCs) presented, and any necessary proofs of VC validity and non-revocation (as outlined in Appendix B.1, where the sender still proves control of their own DID to authorize the use of their VCs for this nonce-identified event); placeholder indicators for opted-out inputs; and the new blinding factors. The sender's DID itself is not included in the 'AttestationClaimsHash' nor directly propagated to the recipient via the IADP.

This specification provides a framework for the `AttestationAccumulator` within an optional system, balancing historical summary needs with ZKP efficiency and handling mixed input types. The exact choice of hash, commitment scheme, and `OptedOutPlaceholder` value would be determined during implementation based on the specific ZKP system used.

B ZKP Circuit Descriptions

This appendix provides high-level descriptions of the Zero-Knowledge Proof (ZKP) circuits for both the lightweight and heavyweight paths outlined in the main architecture (Section 4). These descriptions focus on the primary inputs, outputs, and the logical assertions enforced by each circuit, rather than low-level gate implementations. The specific instantiation would depend on the chosen ZKP system (e.g., systems like Groth16 or PLONK, or STARKs), potentially combined with a folding scheme (e.g., HyperNova).

B.1 Logic Verified for Optional Lightweight Attestation Path

This section describes the core logical assertions related to the optional lightweight attestation path that must be cryptographically verified when a user chooses to engage this path for a transaction. For L1 UTXO systems, these assertions would be enforced within a user-generated, efficient, non-recursive ZKP submitted with the transaction. For L2 rollups, this logic would be part of the L2's state transition function, and its correct execution for all opted-in transactions in a batch would be verified by the main L2 validity proof generated by operators (e.g., using SP1).

B.1.1 Relevant Public Inputs (for Verification Context)

The verification context needs access to relevant public data, which may include:

- Identifiers/Nullifiers of input private commitments (notes/UTXOs) being spent.
- Commitments to output private commitments being created.
- Commitment(s) to the output `AttestationAccumulator`(s) (`Acc_out`) associated with any new attestation-enabled output commitments.
- Commitment(s) to the hash(es) of any Intermediate Attestation Data Packet(s) (`Hash(IADP_new)`) generated if the lightweight path was engaged.
- **Issuer trust root & epoch pinning:** Let $\mathsf{IssuerPKRoot}_t$ be a Merkle (or Poseidon) root committed on L1 that authenticates the *current* allow-list of issuer verification keys (associated with issuer DIDs) at block height t. The proof takes the pair $(\mathsf{IssuerPKRoot}_t, t)$ as *public inputs*. For every VC verified inside the circuit (via a cached $\pi_{\text{VC\_validity}}$) the prover must also supply evidence (as part of $\pi_{\text{VC\_validity}}$'s statement) that:

  a) the issuer's verification key was included under a recognized $\mathsf{IssuerPKRoot}_{t'}$ (where t' is relevant to $\pi_{\text{VC\_validity}}$'s generation time), and

  b) the VC's signature under that key was valid.

  Supplying these proofs pins the statement to the exact issuer key set that was authoritative. Verifiers can enforce freshness by rejecting proofs whose underlying VCs (as attested by $\pi_{\text{VC\_validity}}$) relied on an epoch t' that precedes the latest acceptable rotation or revocation window.
- **Public identifiers (e.g., DIDs) or keys of trusted Verifiable Credential (VC) issuers.** These are referenced by the $\mathsf{IssuerPKRoot}$.
- **Public state of VC revocation mechanisms (e.g., Merkle roots of revocation lists or status accumulators).** This state is checked during the generation of $\pi_{\text{VC\_validity}}$.
- System constant MaxFanIn (attestation fan-in limit).
- (Potentially) Public identifier(s) for the set of external lists (`SpecifiedLists_Sender`) against which attestations are being proven, if this context is public.

B.1.2 Relevant Private Inputs (Witness Data Provided by User Choosing Lightweight Path)

To verify the assertions below for an opted-in transaction, the verification logic requires access to private witness data provided by the user:

- Secret spending key(s) or equivalent proof of ownership for the input commitment(s).
- For any attestation-enabled input commitments: Opening information for their input `AttestationAccumulator`(s) (`Acc_in`), including the committed attestation set values (which include nonces) or `Hash_Inputs`, and blinding factor(s).
- For any standard private input commitments (used in a merge): Indication that they contribute the `OptedOutPlaceholder`.

- **Sender's authorization for VC usage:** Evidence that the sender controls the DID to which the VCs (used in $\pi_{\text{VC\_validity}}$) were issued. This proof of control is primarily established during the generation of the cached $\pi_{\text{VC\_validity}}$.
- **Cached VC Validity Proof(s) ($\pi_{\text{VC\_validity}}$)** corresponding to the VCs used for the current attestation.
- **Unique random nonce (`Nonce_Current`)** for the current 0-hop attestation event. This nonce will be part of the `AttestationClaimsHash_Current`.
- **The specific claims derived from the successfully verified $\pi_{\text{VC\_validity}}$** (e.g., non-membership on lists L1, L2,...).
- (If not public) The set of list identifiers (`SpecifiedLists_Sender`) for which attestations are being demonstrated.
- Full contents of the newly generated Intermediate Attestation Data Packet(s) (`IADP_new`), which must include `Nonce_Current`.
- New blinding factor(s) (`bf_new`) for the output `AttestationAccumulator`(s).
- Standard private transaction details (e.g., output values, recipient information needed for output commitments).

B.1.3 Main Assertions / Constraints Verified (for Opted-In Lightweight Path Transactions)

The verification logic (within the user's L1 ZKP or the L2 operator's batch proof for the main 0-hop OptAttest transaction) enforces the following conditions for transactions engaging the lightweight path:

1. **Standard Transaction Validity:** Ensures the prover owns the input commitments, prevents double-spending (via nullifiers), and guarantees value conservation, consistent with the base protocol rules. (Standard transaction validity applies to all transactions).
2. **Linkage to User's Pre-Computed VC Validity Proof and Authorization:** Ensures that the claims used for the current attestation originate from a valid, user-authorized, pre-computed cached VC validity proof ($\pi_{\text{VC\_validity}}$). $\pi_{\text{VC\_validity}}$ itself proves that the VCs were legitimately presented by an entity in control of the DID to which those VCs were issued. The current main ZKP verifies $\pi_{\text{VC\_validity}}$. The sender's subject DID is used for this authorization but is not included in the `AttestationClaimsHash` (which uses the `Nonce_Current`) or propagated in the IADP.
3. **0-Hop Attestation via Cached VC Validity Proof Verification:** This step efficiently verifies the integrity and applicability of the user's pre-computed cached VC validity proof(s) ($\pi_{\text{VC\_validity}}$) to the current transaction. It ensures:
    - The provided $\pi_{\text{VC\_validity}}$ (e.g., a Groth16 proof, taken as a witness by the current ZKP) is itself a valid ZKP. Verifying that $\pi_{\text{VC\_validity}}$ is a valid ZKP is typically achieved by including a constant-cost verifier circuit (e.g., a Groth16 verifier) within the current ZKP. The cost for this verification per $\pi_{\text{VC\_validity}}$ is minimal (e.g., a small number of pairings if it's a Groth16 proof).
    - The statement proven by $\pi_{\text{VC\_validity}}$ is correctly utilized. This statement (whose contents were established during the generation of $\pi_{\text{VC\_validity}}$, see Section 5.2.1) would have encapsulated that:

- The underlying native VC's signature was valid.
- The VC was from a trusted issuer (whose DID/key is recognizable, e.g., key in $\mathsf{IssuerPKRoot}_t^{\mathsf{active}}$ and not $\mathsf{IssuerPKRoot}_t^{\mathsf{deny}}$ at the time of $\pi_{\text{VC\_validity}}$ generation).
- The VC was not expired and not revoked (per checks made during $\pi_{\text{VC\_validity}}$ generation).
- The user demonstrated control of the DID to which the VC is bound (authorizing its use for this attestation event).
- Specific claims (e.g., "not on list X") were correctly derived.

- The current ZKP ensures these attested claims, as output by the verified $\pi_{\text{VC\_validity}}$, along with the unique `Nonce_Current` for this event, are correctly incorporated into the current transaction's attestation data (e.g., for constructing `AttestationClaimsHash_Current` as per Appendix A).

4. **Attestation Accumulator Update Verification:** Enforces the correct computation of the output `AttestationAccumulator`(s) (`Acc_out`) for the new attestation-enabled output commitment(s), following the rules specified in Appendix A. This condition includes correctly handling splits/merges and incorporating the `OptedOutPlaceholder` for non-attestation-enabled inputs merged, based on the provided witness for input accumulators and the current set of *verified 0-hop claims derived from the successfully verified* $\pi_{VC\_validity}$ *(these claims are now associated with* `Nonce_Current` *in the* `AttestationClaimsHash_Current`*)*. Checks the underlying hash and/or commitment scheme arithmetic.

5. **IADP Commitment Verification:** Checks that the committed `Hash(IADP_new)` correctly corresponds to the hash of the provided `IADP_new` witness contents, and that the contents of `IADP_new` (which must contain the `Nonce_Current` for the attestation event, along with summaries of the claims derived from $\pi_{\text{VC\_validity}}$, VC issuer DID/key references, contextual information from $\pi_{\text{VC\_validity}}$ generation like revocation status references; see Appendix D) are consistent with the transaction's attestation data and the computed `Acc_out`.

Verification of these points ensures the integrity of the immediate attestation step (now based on efficiently verifying user-pre-computed proofs from VCs, with the event identified by a unique nonce) and the correct evolution of the summarized attestation state for commitments where the user chose to enable this feature.

B.2 Heavyweight ZKP Circuit for Historical Attestation (using SP1 Native Recursion)

Generated infrequently for full hop 0 & hop 1 attestation verification, this circuit is the computation proven by a general-purpose zkVM like SP1 [Roy, 2024] employing its native recursive capabilities [Succinct Labs, a]. It validates the historical attestation trace (composed of nonce-identified attestation events and their properties) derived from locally stored IADPs.

B.2.1 Public Inputs

- Identifier/Nullifier of the primary input UTXO being spent in the *new* transaction authorized by this proof.
- The `AttestationAccumulator` (`Acc_Current`) associated with this primary input UTXO.

- Commitments to the output UTXO(s) of the *new* transaction.
- Public parameters (e.g., *trusted VC issuer DIDs/keys referenced in historical IADPs, public state of VC revocation mechanisms relevant to historical VCs*, ZKP parameters for SP1 including recursive verifier key/config, genesis state identifier for attestation history).
- (Potentially) Information about the trigger condition (e.g., destination VASP identifier, list of lists for which attestations are being proven).
- Commitment or hash of the previous SP1 proof being verified (if required by the recursive scheme).

B.2.2 Private Inputs (Witness)

- Secret spending key for the primary input UTXO.
- The chain of relevant historical Intermediate Attestation Data Packets (`IADP_Hop0`, `IADP_Hop1`) retrieved from local storage, corresponding to the history summarized in `Acc_Current`. If `Acc_Current` resulted from a merge, this history includes IADP chains for all M merged branches back to the required depth. Each IADP contains the attestation nonce for its hop, multi-list *VC proof summaries (including VC issuer DID/key references and verified claims)*.
- The previous SP1 proof (`previous_sp1_proof`) being verified recursively (corresponding to the Hop 1 attestation verification).
- (Potentially) Openings/blinding factors for historical accumulators if needed by the verification logic, including the nonces used in historical `AttestationClaimsHash` calculations.
- Details of the *new* transaction being authorized (amounts, recipients, etc.).
- Intermediate witnesses required by the SP1 prover and its recursive mechanism.

B.2.3 Main Assertions / Constraints Enforced (Recursive Structure within SP1)

The SP1 execution trace being proven effectively validates the chain of historical multi-list *attestation events (identified by nonces) and their outcomes (derived from VC proofs, referencing VC issuer DIDs/keys)* backwards from the present UTXO:

1. **Outermost Layer (Current Transaction + Hop 0 Attestation Event Verification):** The SP1 program execution performs/enforces:
    - Standard validity checks for the *new* transaction being authorized (ownership of primary input via spending key, value balance, nullifier check, etc.).
    - Verification of the 0-hop multi-list attestation event's *claims derived from VCs*, as recorded in `IADP_Hop0` (which corresponds to the transaction that created the primary input UTXO). This verification involves using the attestation nonce, VC proof summaries (including VC issuer DID/key references), specified lists, and revocation context witness data from `IADP_Hop0` to execute the necessary VC proof verification logic (e.g., checking issuer trust, revocation status, claim validation) within SP1. This verification of VC-derived claims proves that the claims were

legitimately derived from VCs presented by an authorized subject DID at that historical point, without revealing that subject DID here.

- Verification of the consistency between the data (verified claims, nonce, lists, timestamp used to form `AttestationClaimsHash_Hop0`) in `IADP_Hop0` and the state represented by the public input `Acc_Current`.
- **Recursive Verification Call:** Invokes the native SP1 recursive verifier logic/gadget to check the validity of the provided `previous_sp1_proof` (which corresponds to the Hop 1 attestation event verification).

2. **Inner Layer (Hop 1 Attestation Event Verification - Proven by `previous_sp1_proof`):** The previous proof (verified in step 1d) would have enforced similar logic for the Hop 1 attestation event relative to its output (which is Hop 0's input):
   - Verification of the 0-hop multi-list attestation event's *claims derived from VCs*, corresponding to `IADP_Hop1`, using data (nonce, VC proof summaries including VC issuer DID/key references, etc.) from `IADP_Hop1` witness.
   - Verification of the consistency of data in `IADP_Hop1` (including its nonce) with the state referenced by `IADP_Hop0`.
   - (If Hop 1 resulted from a merge) Recursive verification calls for all branches involved in that prior merge, validating their attestation histories (sequences of nonce-identified events) down to the required depth.
   - Base case check if recursion hits genesis/trusted state for attestation history.
3. **Merge Handling:** If the outermost layer (or any inner layer) verifies an `Acc_Current` representing a merge of M inputs, the SP1 program execution must perform the attestation event verification steps (0-hop VC proof verification logic based on IADP data for that nonce-event, consistency check, recursive verification call) for *each* of the M branches, using the corresponding IADP chains retrieved from storage as witnesses. This constraint significantly increases the computation executed and proven by SP1.

The final output is a single SP1 proof (e.g., a compressed STARK or a wrapped SNARK) attesting that the entire Hop 0 & Hop 1 attestation history (sequence of nonce-identified attestation events, with their claims, VC issuer DIDs/keys, and lists checked), as reconstructed from the provided IADPs, is valid and consistent with the `AttestationAccumulator` of the UTXO being spent, and that the new transaction is also valid according to the base protocol rules. This proof serves as a verifiable report of historical attestations, without revealing the subject DIDs of the historical attesters to the verifier of this report.

B.2.4 Alternative using Folding Schemes (e.g., HyperNova)

An alternative architecture could use a folding scheme like HyperNova [Kothapalli and Setty, 2024] for the recursive composition. In this model:

1. At each step, SP1 executes only the 'AppLogic' (0-hop multi-list attestation *VC proof verification for a nonce-identified event* from IADP data including VC issuer DIDs/keys, state consistency checks), generating a proof or claim π_{SP1} for this execution.

2. A separate process (potentially within SP1, or as a dedicated circuit) executes the HyperNova folding verifier logic. This separate process takes the claim derived from π_{SP1} and folds it with the accumulated folded instance from the previous step (Z_{N-1}), producing the new folded instance Z_N.

This architecture aims to leverage the folding scheme's low cryptographic overhead per step. However, it introduces the significant challenge of efficiently proving/verifying the SP1 execution claim (π_{SP1}) within the algebraic structure required by the folding scheme (e.g., CCS for HyperNova). Bridging STARK/AIR-based systems like SP1 with CCS/R1CS-based folding schemes is an active area of research and may introduce non-trivial overheads that could negate the folding efficiency gains, especially for a low number of recursive steps like N=2. HyperNova's multi-folding capability might offer a more streamlined cryptographic approach to combining the M branches during merge verification compared to M separate recursive calls in the native SP1 approach, but the witness computation cost for processing all attestation data (nonces, claims, VC issuer DIDs/keys etc. for each historical event) likely remains similar.

C Security Model

This appendix outlines the security model, defines key privacy and security properties for the hybrid attestation architecture, and provides arguments for why these properties hold based on the underlying cryptographic primitives.

C.1 Adversarial Model

We consider a standard probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ whose goal is to violate either the privacy or the security/integrity properties of the system. The adversary's capabilities include:

- **Observation:** $\mathcal{A}$ can observe all public blockchain data, including transaction structures, lightweight and heavyweight ZKPs, encrypted Intermediate Attestation Data Packet (IADP) blobs, public `AttestationAccumulator` commitments, and any public state related to DID/VC infrastructure (e.g., DID registries, public keys and DIDs of VC issuers, roots of revocation lists).
- **Network Control:** $\mathcal{A}$ may control the network, delaying or reordering messages, but cannot break underlying cryptographic protocols (e.g., forge signatures, decrypt ciphertexts without keys, break ZKP soundness).
- **User Corruption:** $\mathcal{A}$ can corrupt an arbitrary polynomial number of users, learning their secret keys (spending keys, DID private keys), Verifiable Credentials (VCs), local IADP storage (including historical nonces and VC issuer DIDs from received IADPs), and randomness. However, we assume the specific target users whose privacy or transaction integrity is being analyzed remain honest.
- **VC Issuer Interaction/Corruption:** $\mathcal{A}$ can interact with VC issuers. We consider issuers to be honest-but-curious (follow issuance protocols but might try to correlate data) or potentially malicious (deviate from issuance or revocation protocols). The security of the system relies on the verifier only accepting VCs from trusted issuers (identified by their DIDs/keys) and checking for revocation (typically as part of $\pi_{\text{VC\_validity}}$ generation).
- **Active Attacks:** $\mathcal{A}$ can create and send arbitrary transactions, potentially using corrupted user keys or DIDs to authorize their own VCs.

C.2 Privacy Definitions and Arguments

C.2.1 On-Chain Privacy (vs. Public Observer)

Property: An observer of the public blockchain cannot distinguish transactions using the hybrid Attestation mechanism from standard privacy-preserving transactions (where applicable), nor learn the subject DIDs involved in historical attestations, the specific VCs presented, the claims proven from VCs beyond what's committed, the attestation nonces, or the contents of IADPs. *Argument:* Relies on:

1. The zero-knowledge property of the lightweight and heavyweight ZKPs, which reveal nothing about the private witness (subject DIDs authorizing VCs, VCs themselves, VC proof elements, IADP contents including nonces and VC issuer DIDs, historical data).
2. The hiding property of the `AttestationAccumulator` commitment scheme (e.g., Pedersen commitments are computationally hiding), which conceals the attestation nonces and other hashed data from public view.
3. The semantic security of the encryption scheme used for transmitting IADPs.
4. The privacy guarantees of the underlying base cryptocurrency layer (shielding sender, receiver, amount).

The public only observes opaque cryptographic elements (proofs, commitments, ciphertexts) and potentially references to public revocation states (like Merkle roots), which do not reveal individual user status or specific attestation event details.

C.2.2 Privacy from VC Issuers (During Transaction)

Property: During a transaction involving an opted-in 0-hop check, the VC issuers for the credentials being proven learn nothing about this specific transaction or the user's current activity. *Argument:* The DID/VC model for the 0-hop check relies on users presenting ZKPs of VCs that were *previously obtained* offline (using their subject DID to interact with the issuer). There is no direct interaction with VC issuers at the time of the transaction. The ZKP only proves claims based on existing VCs without contacting the issuer.

C.2.3 Recipient Privacy (vs. Sender)

Property: The sender learns no additional information about the recipient beyond what is necessary for the base transaction (e.g., recipient's public key for IADP encryption if the recipient also opts into receiving attestation-enabled funds). *Argument:* The attestation mechanism primarily involves sender-side computation (generating ZKP of their VCs and the attestation event). The sender only needs the recipient's public key or a derived shared secret to encrypt the IADP blob for attestation-enabled outputs.

C.2.4 Sender Privacy (vs. Recipient)

Property: The recipient of an attestation-enabled commitment (and its associated IADP) learns only the necessary historical attestation data for the specific lineage of funds. For each historical hop, this historical attestation data includes the unique nonce identifying the attestation event, summaries of the VCs used (including references to the VC *issuer's* DID/key), and the verified claims. Crucially, the sender's subject DID used to authorize their VCs for that 0-hop attestation event is *not* included in the IADP and thus not revealed to the recipient. The recipient cannot

link these nonce-identified attestation events to the sender's unrelated activities beyond what is revealed by the attestation event data itself. *Argument:* The IADP contains only the data relevant for future heavyweight proofs related to *this specific lineage of funds* (see Appendix D). The sender's subject DID for a given historical attestation event is not part of this propagated data. Privacy relies on the security of the encryption scheme protecting the IADP blob during transmission. The attestation nonces are specific to attestation events and are not directly linkable to a user's main transaction DIDs without further context.

C.3 Security/Integrity Definitions and Arguments

C.3.1 Attestation Soundness

Property: A PPT adversary $\mathcal{A}$ cannot produce a cryptographically valid ZKP that falsely asserts a "clean" or "unsanctioned" attestation (i.e., verified claims from VCs for a specific nonce-identified event) for a transaction when the relevant VCs, if honestly obtained from trusted issuers (identified by their DIDs/keys) by an authorized subject and checked against valid revocation status, would not support such claims for the specified lists at the relevant hop.

Argument:

- **Lightweight Proof:** Relies on the soundness of the chosen ZKP scheme (e.g., SP1/Plonky3). Specifically:
  - The 0-Hop Attestation VC Proof Verification constraint (Appendix B.1, Assertion 3) ensures that the ZKP only validates if the presented cached $\pi_{\text{VC\_validity}}$ is valid. $\pi_{\text{VC\_validity}}$ itself attests that the underlying VCs were cryptographically valid, issued by trusted entities (whose DIDs/keys are verifiable), not revoked, authorized by a controlling subject DID, and correctly imply the claimed attestations for the nonce-identified event. Forging such a validation requires breaking ZKP soundness, the integrity of $\pi_{\text{VC\_validity}}$, or the underlying VC cryptography.
  - The Accumulator Update Verification constraint (Assertion 4) ensures the output accumulator correctly reflects these verified claims (associated with the nonce) and the input state. Forging requires breaking ZKP soundness or the binding property of the commitment scheme.
- **Heavyweight Proof:** Relies on the soundness of the ZKP system used for recursion (e.g., SP1's native recursive mechanism). The recursive proof verifies the entire chain of 0-hop nonce-identified attestation events using the IADPs (which contain nonces, claims, and VC issuer DIDs/keys) as witnesses (Appendix B.2). Forging requires breaking soundness at one of the recursive steps, which involves either forging the VC proof verification for that nonce-identified step or forging the verification of the previous recursive proof instance.

Multi-suite soundness. Because all suites reduce their native signature verification to a Groth16 proof over the canonical message hash (as part of $\pi_{\text{VC\_validity}}$ generation), forging a credential that passes the dispatch gadget without possession of a valid signature implies forging *either* (a) the underlying signature itself *or* (b) the $\pi_{\text{VC\_validity}}$ or the ZKP verifying it. Both contradict the hardness of the suite's-specific curve discrete log or the knowledge-soundness of the ZKP schemes, respectively. Thus the soundness bound degrades by at most a negligible union-bound factor in the number of supported suites.

C.3.2 Accumulator Integrity

Property: An adversary cannot create a UTXO with a `AttestationAccumulator` commitment `Acc_out` that does not correctly represent the result of applying the update rules (Appendix A) to valid input accumulators and validly verified 0-hop VC claims (associated with a nonce for the current attestation event). *Argument:* This property is directly enforced by the Accumulator Update Verification constraint (Assertion 4) within the lightweight ZKP. Its security reduces to the soundness of the ZKP and the binding property of the commitment scheme used for the accumulator.

C.3.3 DID Control and VC Unforgeability (for Sender's Own 0-Hop)

Property: An adversary cannot successfully generate a 0-hop attestation ZKP for a specific nonce-identified event unless they control the subject DID's private key that authorized the VCs (whose claims are being used) and the VCs are legitimate and correctly bound to that authorizing subject DID. *Argument:* Enforced by the Linkage to User's Pre-Computed VC Validity Proof and Authorization constraint (Assertion 2) and the 0-Hop Attestation VC Proof Verification (Assertion 3) in the lightweight ZKP (Appendix B.1). These rely on verifying $\pi_{\text{VC\_validity}}$, which itself proves that the user demonstrated control of the subject DID to which the VCs are bound and that the VCs (including issuer signature) are valid. Security reduces to ZKP knowledge soundness and the unforgeability of the digital signatures used for subject DIDs and VCs. The subject DID itself is not propagated to recipients, but its control is essential for the sender to make a valid 0-hop attestation.

C.3.4 IADP Integrity

Property: An adversary cannot cause a valid heavyweight proof to be accepted based on forged or inconsistent IADP data. *Argument:* The lightweight ZKP commits to `Hash(IADP_new)` (Assertion 5). The heavyweight ZKP takes the sequence of IADPs as witnesses and its recursive verification logic implicitly checks the consistency between the data in the IADPs (e.g., nonces, VC proof summaries including VC issuer DIDs/keys, verified claims), the sequence of accumulator updates (which incorporate hashes derived from these nonces and claims), and the 0-hop VC proof verification results for each nonce-identified historical event. Any inconsistency would cause the heavyweight proof verification to fail. Relies on ZKP soundness and the collision resistance of the hash function used for the IADP commitment.

C.4 Trust Assumptions Recap

These security and privacy arguments rely on the trust assumptions outlined in Section 7.4, including the hardness of underlying cryptographic problems (e.g., for ZKPs, hashes, VC signatures), the integrity and availability of VC Issuers (identifiable by their DIDs/keys) and the DID/VC revocation infrastructure, the correctness of the ZKP system's implementation (including its recursive components), user device security for IADP and VC storage, and potentially the trustworthiness of external provers if offloading is used. A failure in any of these assumptions could compromise the system's guarantees.

D Intermediate Attestation Data Packet (IADP) Structure

This appendix defines the structure and content of the Intermediate Attestation Data Packet (IADP). IADPs are generated by the sender only when they engage the optional lightweight transaction path (Section 5.2). The sender uses their Decentralized Identifier (DID) and Verifiable Credentials (VCs) to authorize their 0-hop attestation, which is linked to a unique nonce for that specific attestation event. The hash of the IADP (`Hash(IADP)`) is committed to within the verification logic for that transaction (Appendix B.1). The IADP is then encrypted for the recipient of the resulting attestation-enabled private commitment (note/UTXO) and stored locally on the recipient's device. IADPs contain the necessary detailed information (including the attestation nonce, VC proof summaries, and VC issuer references) summarized by the `AttestationAccumulator`. This information is required for the user to later generate the full heavyweight hop 0 & hop 1 proof of attestation history for that specific commitment. Standard private commitments do not have associated IADPs.

D.1 Purpose

For attestation-enabled commitments, the IADP serves as the verifiable link between the compact `AttestationAccumulator` (which commits to a nonce-identified 0-hop attestation event) and the actual 0-hop multi-list attestation event (Section 5.2.1) performed during its creation. When a user needs to generate a heavyweight proof for an attestation-enabled commitment, their wallet retrieves the relevant chain of IADPs from local storage to provide the detailed witness data for the recursive ZKP.

D.2 Data Fields

Each IADP corresponds to the single transaction hop that created a specific attestation-enabled commitment. It must contain sufficient information to verify the 0-hop multi-list attestation (proven via VCs for a specific nonce-identified event) performed at that hop and link back to the previous hop(s) in the attestation-enabled lineage. A potential structure is as follows:

`hop_type: Enum{Simple, Merge}` Indicates whether the attestation-enabled commitment associated with this IADP was created by a simple transfer (one attestation-enabled input) or a merge of multiple inputs (potentially mixed types).

`hop_0_attestation_nonce: NonceValue` The unique random nonce generated by the sender for the 0-hop attestation event at this hop. This nonce is used in the `AttestationClaimsHash_0` for this hop.

`hop_0_specified_lists: List<ListIdentifier>` An identifier or commitment to the set of external lists for which attestations were proven via VCs for this attestation event.

`hop_0_vc_proof_summary: VerifiableCredentialProofData` Data summarizing the ZKP of VC validity presented by the sender for this attestation event. The sender proved control of their own DID to authorize the use of these VCs, but that subject DID is not stored here. Such summary data would include:

- Commitments to (or essential distinguishing features of) the VCs used.

- References to the *issuer DIDs/keys* for those VCs.
- References to the state of revocation mechanisms (e.g., Merkle roots, accumulator values) checked for those VCs at the time of transaction.
- The set of verified claims (e.g., {Claim(NotOnList1, True), Claim(NotOnList2, False),...}) derived from the VC proof.
- Sufficient cryptographic evidence (e.g., a ZKP snippet or its hash, if the main ZKP allows such extraction) to re-verify the core claims within the heavyweight proof for this nonce-identified event.

`hop_0_timestamp: TimestampValue` A timestamp or block height associated with the transaction of this hop.

`input_references: Union{SimpleInputRef, List<MergedInputRef>}` Contains references needed to link back to the state of the input commitments used in this transaction, dependent on `hop_type`:

- **If `hop_type == Simple`:** (Implies the single input was attestation-enabled). Contains a single `SimpleInputRef` structure:
  - `input_accumulator_ref: CommitmentValue | HashValue` A reference (e.g., the commitment value or its hash) to the `AttestationAccumulator` (`Acc_in`) of the single attestation-enabled input commitment used.
  - `input_IADP_hash_ref: HashValue` The hash commitment (`Hash(IADP_Previous)`) of the IADP associated with that attestation-enabled input. This hash commitment provides the cryptographic link needed to retrieve the IADP containing the Hop 1 attestation data (which will include its own 'hop_0_attestation_nonce').
- **If `hop_type == Merge`:** Contains a list (`List<MergedInputRef>`) of structures, one for each of the M merged inputs ($i \in [1, M]$). The structure for each input i depends on whether it was attestation-enabled or standard private:
  - **If input i was attestation-enabled:**
    - `input_i_type: Enum{AttestationEnabled}`
    - `input_i_accumulator_ref: CommitmentValue | HashValue` Reference to its `Acc_in_i`.
    - `input_i_attestation_claims_hash_ref: HashValue` The value of its `AttestationClaimsHash_0_i`. (Needed for heavyweight proof to verify `Hash_Inputs`; this hash was computed using a nonce from input i's IADP).
    - `input_i_IADP_hash_ref: HashValue` The hash (`Hash(IADP_Previous_i)`) of its associated IADP, linking to historical data for this branch.
  - **If input i was standard private (opted-out):**
    - `input_i_type: Enum{StandardPrivate}`

  This structure allows the heavyweight proof generator to know which branches require further IADP retrieval and historical verification.

D.3 Generation and Usage (Only for Optional Attestation Path)

- **Generation:** The sender's wallet constructs the IADP during the lightweight transaction process only if they opt-in. It uses the current 0-hop attestation data (derived from proving VC validity for a newly generated attestation nonce) and references derived from any attestation-enabled input commitment(s) and their associated (decrypted) IADPs.

- **Commitment:** The transaction verification logic (L1 ZKP or L2 operator proof) includes a commitment (e.g., `Hash(IADP)`) to the generated IADP structure for opted-in transactions, ensuring its integrity and binding it to the attestation-enabled output commitment(s) and their `AttestationAccumulator`.
- **Transmission:** For opted-in transactions, the sender encrypts the full IADP (and potentially necessary historical IADPs from attestation-enabled inputs) for the recipient and includes it in a private transaction field. Standard private transactions do not include IADPs.
- **Storage:** The recipient of an attestation-enabled commitment decrypts and stores the IADP locally, associated with that specific commitment. This local storage is critical for future proofs, although it is a point of failure if data are lost or compromised for that commitment's history. Standard private commitments require no IADP storage.
- **Retrieval:** When a user generates a heavyweight proof for spending an attestation-enabled commitment, their wallet retrieves this IADP and uses the `input_references` to recursively retrieve the IADP(s) for the previous hop(s) (only for branches indicated as 'AttestationEnabled' in the merge list) from its local store. This retrieval reconstructs the necessary hop 0 & hop 1 attestation history (based on nonce-identified VC proof events, detailing claims, VC issuer DIDs, and lists checked) needed for the proof witness.

This IADP structure aims to provide the necessary linkage and detailed data for user-generated heavyweight proofs of attestation history, while remaining relatively compact and explicitly handling mixed inputs in merges. The exact encoding would depend on implementation details. The reliance on local storage introduces practical challenges for users managing attestation-enabled funds. The attestation event itself remains verifiable and linked via its nonce and the details of the VC issuers involved.

Acknowledgments

Thank you to Alex Levy (Thunder Hill Cinema) for insightful discussions about human rights, and encouragement that helped maintain focus and momentum. The writing and creative process for this manuscript involved significant assistance from the Large Language Models Google Gemini (2.5 Pro Preview 03-25 and 05-06 with Grounding), ChatGPT 4o (Deep Research), ChatGPT o3, ChatGPT o4-mini-high, ChatGPT 4.1 (Deep Research), Claude 3.7 Sonnet, and Claude 3.7 Sonnet (Thinking), particularly during drafting, brainstorming, and literature synthesis. In my professional capacity as Research Engineer at iterabloom LLC, I take full responsibility for the entire content, claims, and correctness (or incorrectness) of this preprint.

References

Al Jazeera Staff. UN's food stocks in Gaza 'completely depleted' amid Israel blockade. https://perma.cc/F68G-Z6TL, April 2025. Accessed: 2025-05-05.

Martin Albrecht, Lorenzo Grassi, Christian Rechberger, Arnab Roy, and Tyge Tiessen. Mimc: Efficient encryption and cryptographic hashing with minimal multiplicative complexity. In Jung Hee Cheon and Tsuyoshi Takagi, editors,

Advances in Cryptology – ASIACRYPT 2016, pages 191–219, Berlin, Heidelberg, 2016. Springer Berlin Heidelberg. ISBN 978-3-662-53887-6.

Amnesty International. What do the trump administration's sanctions on the icc mean for justice and human rights? https://perma.cc/D7GM-3ZG9, March 2025. 2025-05-06.

Associated Press. Trump orders sanctions on international criminal court for investigating israel. https://perma.cc/NA45-S45P, February 2025.

Shlomit Azgad-Tromer, Joey Garcia, and Eran Tromer. The case for on-chain privacy and compliance. *Stanford Journal of Blockchain Law & Policy*, 6(2): 1–32, June 2023. Available: https://stanford-jblp.pubpub.org/pub/onchain-privacy-compliance.

Smadar Becker and Muhammad Dabsan. Khalet al-mafatih: Settlers from yinon levy's farm threaten a shepherd and his family. https://perma.cc/YW75-2DYN, March 2025. Original title in Hebrew. Translator: Natanya at MachsomWatch. Accessed: 2025-05-05.

Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. IACR Cryptology ePrint Archive, Report 2018/046, 2018. Available: https://eprint.iacr.org/2018/046.

Raúl Carrillo. Testimony before the u.s. house of representatives committee on financial services, subcommittee on digital assets, financial technology and inclusion regarding "digital dollar dilemma: The implications of a central bank digital currency and private sector alternatives". Technical report, Columbia Law School, September 2023. URL https://perma.cc/R5UK-LVDP. Accessed: 2025-05-11.

Tom Dannenbaum, Janina Dill, et al. Top experts on why aid to Gaza can't be conditioned on hostage release (in response to remarks by US official). https://perma.cc/9FAG-9KVG, November 2023. Accessed: 2025-05-05.

democracynow.org. Israeli human rights lawyer attacked while documenting settler raid on gaza aid convoy. https://perma.cc/S5PB-CJTG, May 2024. Accessed: 2025-05-05.

Joshua Dávila. *Blockchain Radicals: Taking the Capitalism out of Anarcho-Capitalist Technology*. Radical Press Example, Oakland, CA, 2023. Illustrative reference for concepts on decentralized community control and activism.

Kobi Ettinger. Settlers protest blocks jerusalem entrance after police shooting. https://perma.cc/3Z3A-Y493, January 2025. Original title in Hebrew. Accessed: 2025-05-05.

Executive Office of the President. Executive order 14115: Imposing certain sanctions on persons undermining peace, security, and stability in the west bank. Federal Register, February 2024. URL https://perma.cc/E5U6-DPVJ. President Joseph R. Biden, Jr. Signed February 1, 2024. FR Doc. 2024-02354. Official URL at https://www.federalregister.gov/documents/2024/02/05/2024-02354/imposing-certain-sanctions-on-persons-undermining-peace-security-and-stability-in-the-west-bank. Revoked by EO 14148, January 20, 2025.

Executive Office of the President. Executive order 14148: Initial rescissions of

harmful executive orders and actions. Federal Register, January 2025. URL https://perma.cc/3LTR-LN4G. President Donald J. Trump. Scheduled for publication January 28, 2025. FR Doc. 2025-01901. Archived PDF at https://federalregister.gov/d/2025-01901.

Ian Goldberg. Improving the robustness of private information retrieval. In *Proceedings of the 2007 IEEE Symposium on Security and Privacy (S&P '07)*, pages 131–148. IEEE Computer Society, 2007.

Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. In *Proceedings of the Seventeenth Annual ACM Symposium on Theory of Computing (STOC '85)*, pages 291–304. ACM, 1985.

Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schofnegger. Poseidon: A new hash function for zero-knowledge proof systems. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 519–535. USENIX Association, 2021.

Lorenzo Grassi, Dmitry Khovratovich, Reinhard Lüftenegger, Christian Rechberger, Markus Schofnegger, and Roman Walch. Reinforced concrete: A fast hash function for verifiable computation. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, CCS '22, page 1323–1335, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450394505. doi: 10.1145/3548606.3560686. URL https://doi.org/10.1145/3548606.3560686.

Andrew Gray. Eu's borrell: Israeli moves in gaza break international law. https://perma.cc/V4CS-3UWG, October 2023. Accessed: 2025-05-05.

Rohan Grey. Testimony before the u.s. house of representatives committee on financial services, task force on financial technology regarding "digitizing the dollar: Investigating the technological infrastructure, privacy, and financial inclusion implications of central bank digital currencies". Hearing testimony, U.S. House of Representatives Committee on Financial Services, Task Force on Financial Technology, June 2021. URL https://perma.cc/TX7R-HM24.

Richard Hall. First thing: No power, water or fuel for Gaza until hostages freed, Israel says. https://perma.cc/475W-GXWT, October 2023. Accessed: 2025-05-05.

Holonym Foundation. Human network documentation. https://docs.network.human.tech/, 2024. Accessed: May 27, 2025.

Daira Hopwood, Sean Bowe, Taylor Hornby, and Nathan Wilcox. Zcash protocol specification. Technical Report Version 2024.5.1-112-gcf7a5c [NU6], Electric Coin Company, November 2024. Available: https://zips.z.cash/protocol/protocol.pdf.

Human Rights Watch. Collective punishment: War crimes and crimes against humanity in the Ogaden area of Ethiopia's Somali Region. Technical report, Human Rights Watch, June 2008. Accessed: 2025-05-05.

Human Rights Watch. Us: Trump authorizes international criminal court sanctions. Website, Feb 2025. https://perma.cc/2889-8JF2.

Hyperledger Foundation. Anoncreds specification v1.0 draft. Hyperledger Technical Specification, 2025. Available: https://hyperledger.github.io/anoncreds-spec/.

International Criminal Court. Rome statute of the international criminal court. https://perma.cc/KTL4-2UDV, January 2002. Accessed: 2025-05-05.

Israel National News Staff. Usa imposes sanctions on anti-aid protest organization. https://perma.cc/7GQF-F79F, 2024. Accessed: 2025-05-05.

Ariel Kahana. Residents on whom the previous administration imposed sanctions to trump: "do not give a hand to aid for hamas". *Israel Hayom*, 1 2025. URL https://perma.cc/4ZL2-EJLK. Original title in Hebrew.

Shuky Katz. Israeli activists appeal to trump: Stop the aid to hamas. *Hamechadesh*, 1 2025. URL https://perma.cc/BNC3-JCQY. Website also referred to as HM News. Original title in Hebrew.

Abhiram Kothapalli and Srinath Setty. Hypernova: Recursive arguments for customizable constraint systems. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024*, pages 345–379, Cham, 2024. Springer Nature Switzerland.

Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022*, pages 359–388, Cham, 2022. Springer Nature Switzerland. ISBN 978-3-031-15985-5.

Simon Lewis. Us imposes sanctions on israeli group that attacked gaza aid. https://perma.cc/D2F8-3PLS, June 2024. Accessed: 2025-05-05.

Canaan Lidor. Protesters hold up aid to Gaza as government faces rising domestic pressure. https://perma.cc/M6TB-HMC8, January 2024. Accessed: 2025-05-05.

Sveta Listratov. 'stop empowering Hamas': Israelis appeal to Trump to halt aid, revoke Biden-era sanctions. https://perma.cc/K53T-3NET, January 2025. Accessed: 2025-05-05.

Zeyu Liu and Eran Tromer. Oblivious message retrieval. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022*, pages 753–783, Cham, 2022. Springer Nature Switzerland. ISBN 978-3-031-15802-5.

Zeyu Liu, Eran Tromer, and Yunhao Wang. PerfOMR: Oblivious message retrieval with reduced communication and computation. Cryptology ePrint Archive, Paper 2024/204, 2024. URL https://eprint.iacr.org/2024/204.

Jacob Magid. US sanctions far-right Israeli group behind attacks on aid convoys bound for Gaza. https://perma.cc/K7QR-YZA8, June 2024. Accessed: 2025-05-05.

Heather Marsh. *Abstracting Divinity*. Binding Chaos. MustRead Inc., 2024. ISBN 9781989783092. Book 3 of the Binding Chaos series. Canonical reference for concepts on vitalism, relational ethics, and systemic harm.

Shen Noether, Adam Mackenzie, and the Monero Research Lab. Ring confidential transactions. *Ledger*, 1:1–18, Dec. 2016. doi: 10.5195/ledger.2016.34. URL https://ledger.pitt.edu/ojs/ledger/article/view/34.

Sasha Polakow-Suransky. Analysis: International law and the Israeli government's planned destruction of Palestinian assailants' family homes. https://perma.cc/MK7W-QDX6, November 2015. Accessed: 2025-05-05.

Polygon Labs. Introducing polygon id, zero-knowledge identity for web3. Polygon Labs Blog, March 2022. Available: https://polygon.technology/blog/introducing-polygon-id-zero-knowledge-own-your-identity-for-web3.

Polygon Labs. Plonky3. GitHub Repository, 2023. https://github.com/Plonky3/Plonky3.

Program for Humanitarian Policy and Conflict Research at Harvard University. Rule 55. access for humanitarian relief to civilians in need / practice relating to rule 70. relief personnel. https://perma.cc/4S3L-9Q4W. The guide is a product of collaboration; content primarily based on ICRC study. Accessed: 2025-05-05.

RAILGUN Project. Private proofs of innocence - railgun docs. https://docs.railgun.org/wiki/assurance/private-proofs-of-innocence, a. Accessed: May 10, 2025.

RAILGUN Project. Private proofs of innocence - list providers section - railgun docs. https://docs.railgun.org/wiki/assurance/private-proofs-of-innocence, b. Accessed: May 10, 2025. Information regarding List Providers is detailed on this page.

RAILGUN Project. Private proofs of innocence - private proofs of innocence nodes section - railgun docs. https://docs.railgun.org/wiki/assurance/private-proofs-of-innocence, c. Accessed: May 10, 2025. Information regarding Private Proofs of Innocence Nodes is detailed on this page.

RAILGUN Project. Private proofs of innocence - zk cryptography (section discussing recursion) - railgun docs. https://docs.railgun.org/wiki/assurance/private-proofs-of-innocence, d. Accessed: May 10, 2025. Specifically referring to the discussion on recursive SNARKs carrying proofs forward.

RAILGUN Project. Wallets and keys - railgun docs. https://docs.railgun.org/wiki/learn/wallets-and-keys, e. Accessed: May 10, 2025.

Railgun Project DAO. Having your privacy & eating it too — railgun private proofs of innocence. https://medium.com/@Railgun_Project/having-your-privacy-eating-it-too-railgun-proof-of-innocence-efcba557aac4, May 2023. Accessed: April 30, 2025.

Uma Roy. Introducing sp1: A performant, 100% open-source, contributor-friendly zkvm. Succinct Blog, Feb 2024. https://blog.succinct.xyz/introducing-sp1/.

Uma Roy, John Guibas, Kshitij Kulkarni, Mallesh Pai, and Dan Robinson. Succinct network: Prove the world's software. Whitepaper, 2024. Available: https://perma.cc/FWA8-LP8M.

Srinath Setty, Justin Thaler, and Riad S. Wahby. Customizable constraint systems for succinct arguments. Cryptology ePrint Archive, Report 2023/552, 2023. Available: https://eprint.iacr.org/2023/552.

Adi Shamir. How to share a secret. *Commun. ACM*, 22(11):612–613, November 1979. ISSN 0001-0782. doi: 10.1145/359168.359176. URL https://doi.org/10.1145/359168.359176.

Manu Sporny, Dave Longley, and David Chadwick. Verifiable credentials data model v1.1. W3C Recommendation, March 2022a. Available: https://www.w3.org/TR/vc-data-model/.

Manu Sporny, Dave Longley, Markus Sabadello, Drummond Reed, Orie Steele, and Christopher Allen. Decentralized identifiers (dids) v1.0. W3C Recommendation, July 2022b. Available: https://www.w3.org/TR/did-core/.

Succinct Labs. Proof aggregation | succinct docs. https://docs.succinct.xyz/docs/sp1/writing-programs/proof-aggregation, a. Accessed: 2025-05-05.

Succinct Labs. What is a zkvm? | succinct docs. https://docs.succinct.xyz/docs/sp1/what-is-a-zkvm, b. Accessed: 2025-05-05.

TOI Staff. Right-wing activists vandalize aid shipment headed to gaza, set 2 trucks alight. https://perma.cc/DDF8-UNTX, May 2024a. Accessed: 2025-05-05.

TOI Staff. Six Israelis arrested for attacking, damaging aid convoy heading to Gaza. https://perma.cc/CWS5-GSCM, May 2024b. Accessed: 2025-05-05.

UN News Staff. Gaza: Aid access still key amid reports of families surviving on one meal a day. https://perma.cc/4PCV-AGH9, April 2025. Accessed: 2025-05-05.

WAFA News Agency. Israeli settlers set up new colonial outpost near nablus. https://perma.cc/EG4Q-S2UV, January 2025. Accessed: 2025-05-05.

E. Glen Weyl, Audrey Tang, and Community. *Plurality: The Future of Collaborative Technology and Democracy*. Plurality Institute Example, Taipei, 2024. Illustrative reference for concepts on pluralistic governance and collaborative technology.